

Kubo

ERP + CRM autoalojable para PYMES

Documentación técnica y funcional

Versión 0.3.0 · Armenia, Quindío · Licencia MIT

Generado el 2026-10-21

Documentación de Kubo

[![CI](https://github.com/NFGS/kubo-docs/actions/workflows/ci.yml/badge.svg)](https://github.com/NFGS/kubo-docs/actions/workflows/ci.yml)

> Parte del proyecto **Kubo** — [kubo-workspace](https://github.com/NFGS/kubo-workspace) (ERP + CRM autoalojable para PYMES).

Documentación técnica y funcional del sistema. Está pensada para leerse en orden

la primera vez y para consultarse por secciones después.

Contenido

Documento	Para quién	Qué responde
[01 — Arquitectura](01-arquitectura.md)	Técnico	¿Cómo está construido y por qué así? Diagramas C4, flujos y atributos de calidad
[02 — Modelo de datos](02-modelo-datos.md)	Técnico	¿Qué se guarda, dónde y con qué reglas de integridad?
[03 — API](03-api.md)	Desarrollador	¿Qué endpoints existen y cómo se usan? Ejemplos con `curl`
[04 — Seguridad](04-seguridad.md)	Técnico · Auditoría	¿Cómo se protege la información y qué falta?
[05 — Despliegue](05-despliegue.md)	Operación	¿Cómo se instala, opera y respalda?
[06 — Manual de usuario](06-manual-usuario.md)	Dueño del negocio	¿Cómo se usa el sistema día a día?
[07 — Pruebas](07-pruebas.md)	Calidad	¿Qué se probó, cómo y con qué resultado?
[08 — Trazabilidad](08-trazabilidad.md)	Evaluación	¿Qué requisito se cumplió y dónde se comprueba?
[09 — Guion de demostración](09-demo-guion.md)	Presentación	Guion del video de 6 minutos
[10 — Auditoría técnica](10-auditoria.md)	Arquitectura · Calidad	Hallazgos, mediciones, correcciones aplicadas y catálogo de pendientes
[11 — Plan de cierre](11-plan-de-cierre.md)	Gestión · Arquitectura	¿Qué falta, en qué orden y con qué criterio se cierra cada fase?
[12 — Runbook de operación](12-runbook-operacion.md)	Operación	La semana uno del negocio: chequeo diario, playbooks y métricas
[13 — Guía del adaptador de facturación](13-guia-adaptador-facturacion.md)	Desarrollo · Integración	Cómo enchufar un proveedor tecnológico DIAN sin tocar el núcleo
[14 — Plan de la app móvil](14-plan-app-movil.md)	Producto · Móvil	Fases y criterios de aceptación; **Fase 1 completada**: APK y suite móvil en el emulador (CI)
[ADRs](adr/)	Arquitectura	Las treinta y una decisiones que definen el sistema

Decisiones de arquitectura (ADR)

ADR	Decisión
[0001](adr/ADR-0001-microservicios-monolito-modular.md)	Microservicios con monolito modular interno
[0002](adr/ADR-0002-polyrepo-contratos.md)	Polyrepo con contratos como fuente de verdad
[0003](adr/ADR-0003-multitenancy-hibrido.md)	Multi-tenancy híbrido: `tenant_id` + RLS activo
[0004](adr/ADR-0004-poliglotismo.md)	Poliglotismo deliberado: un lenguaje por contexto
[0005](adr/ADR-0005-eventos-rabbitmq.md)	Eventos con RabbitMQ y publicador tolerante a fallos
[0006](adr/ADR-0006-cifrado-campos.md)

Cifrado de campos personales con AES-256-GCM | | [0007](adr/ADR-0007-jwt-rs256-jwks.md) | JWT RS256 con JWKS: la llave privada no sale del IAM | | [0008](adr/ADR-0008-alcance-mvp.md) | Alcance declarado del MVP y recortes conscientes | | [0009](adr/ADR-0009-outbox-transaccional.md) | Outbox transaccional para la entrega de eventos | | [0010](adr/ADR-0010-rls-activo.md) | RLS activo con interceptor de transacción | | [0011](adr/ADR-0011-bff-capa-formal.md) | BFF como capa formal del gateway | | [0012](adr/ADR-0012-zona-horaria-por-negocio.md) | Zona horaria por negocio | | [0013](adr/ADR-0013-vertical-packs.md) | Vertical Packs: el vertical como dato del negocio | | [0014](adr/ADR-0014-puerto-facturacion-dian.md) | Facturación electrónica DIAN como puerto enchufable | | [0015](adr/ADR-0015-segundo-factor-totp.md) | Segundo factor TOTP del propietario | | [0016](adr/ADR-0016-stock-por-bodega.md) | Stock por bodega y transferencias | | [0017](adr/ADR-0017-puerto-notificaciones.md) | Puerto de notificaciones y buzón del negocio | | [0018](adr/ADR-0018-documentos.md) | Documentos en el ERP con puerto de almacenamiento | | [0019](adr/ADR-0019-rotacion-de-claves.md) | Rotación de claves de cifrado de campo (KEK/DEK) | | [0020](adr/ADR-0020-mtls-interno.md) | mTLS en la malla interna | | [0021](adr/ADR-0021-multi-tenant-saas.md) | Multi-tenant SaaS como modo de despliegue | | [0022](adr/ADR-0022-operador-de-respaldos.md) | Operador de respaldos con verificación de restauración | | [0023](adr/ADR-0023-app-movil.md) | App móvil: la PWA primero, Capacitor cuando el negocio lo pida | | [0024](adr/ADR-0024-superficie-del-operador.md) | Superficie del operador: script hoy, panel cuando haya rol | | [0025](adr/ADR-0025-rol-de-plataforma.md) | Rol de plataforma y panel del operador | | [0026](adr/ADR-0026-puerto-de-cobro.md) | Puerto de cobro y pasarela de pago | | [0027](adr/ADR-0027-sincronizacion-4-entornos.md) | Sincronización de los 4 entornos con huella | | [0028](adr/ADR-0028-despliegue-publico-oss.md) | Despliegue público de demostración 100 % open source | | [0029](adr/ADR-0029-repositorios-publicos-y-ci.md) | Repositorios públicos, licencia MIT y CI en GitHub Actions | | [0030](adr/ADR-0030-sincronizacion-bidireccional.md) | Sincronización bidireccional de los 4 entornos con reconciliación | | [0031](adr/ADR-0031-empaquetado-movil.md) | Empaquetado móvil: assets locales y servidor configurable |

Las 31 ADR están aceptadas; su estado y la fase en que se cerraron se detallan

en [`08-trazabilidad.md`](08-trazabilidad.md) §4.

Documentación por servicio

Cada repositorio tiene su propio README con lo específico:

- [`kubo-gateway`](../kubo-gateway/README.md) — enrutamiento, JWT, límites de tasa

- [`kubo-iam`](../kubo-iam/README.md) — identidad, tokens, auditoría

- [`kubo-crm`](../kubo-crm/README.md) — clientes y cifrado de campos

- [`kubo-erp`](../kubo-erp/README.md) — catálogo, kardex, ventas y eventos

- [`kubo-analytics`](../kubo-analytics/README.md) — proyección e indicadores

- [`kubo-web`](../kubo-web/README.md) — PWA, app Android y experiencia de usuario

- [`kubo-infra`](../kubo-infra/README.md) — orquestación, semilla y pruebas

Generar el PDF consolidado

```
./kubo-docs/scripts/build-pdf.sh
```

Genera `Kubo-Documentacion.pdf` en la raíz del workspace, con todos los

documentos y los ADR unidos. Requiere Google Chrome o Chromium instalado.

01 — Arquitectura

Kubo es un ERP + CRM autoalojable para PYMES. Se instala en el local del negocio

con un solo comando y opera como aplicación web instalable (PWA), incluso sin

conexión a internet.

1. Vista de contexto (C4 nivel 1)

```
flowchart LR
  DUENO["Dueño / Administrador"]
  VENDEDOR["Vendedor en mostrador"]
  KUBO["Kubo<br/>ERP + CRM autoalojable"]
  CLIENTE["Cliente del negocio<br/>(WhatsApp / correo)"]

  DUENO -->|"tablero, catálogo,<br/>clientes"| KUBO
  VENDEDOR -->|"punto de venta"| KUBO
  KUBO -.->|"comprobantes y avisos<br/>(Fase 4)"| CLIENTE
```

2. Vista de contenedores (C4 nivel 2)

```
flowchart TB
  subgraph NAVEGADOR["Navegador del negocio"]
    PWA["PWA instalada<br/>React 19 + Vite<br/>cola offline en IndexedDB"]
  end

  subgraph SERVIDOR["Servidor del negocio (Docker Compose)"]
    TLS["kubo-tls<br/>Caddy · HTTPS"]
    WEB["kubo-web<br/>nginx · activos + proxy /api"]
    GW["kubo-gateway<br/>NestJS · API Gateway<br/>JWT RS256 · límites · BFF"]
    IAM["kubo-iam<br/>Java 21 · Spring Boot 4"]
    CRM["kubo-crm<br/>Ruby 3.4 · Rails 8"]
    ERP["kubo-erp<br/>Elixir 1.17 · Phoenix 1.8"]
    ANA["kubo-analytics<br/>Python 3.13 · FastAPI"]

    PG[("PostgreSQL 17<br/>kubo_iam · kubo_crm · kubo_erp")]
    MG[("MongoDB 8<br/>kubo_analytics")]
    MQ[["RabbitMQ 4<br/>kubo.events"]]
    RD[("Redis 7<br/>límites de tasa")]
    OTEL["kubo-otel<br/>collector OTLP"]
  end

  PWA -->|"HTTPS 3443"| TLS
  TLS --> WEB
  WEB -->|"/api/v1"| GW
  GW -->|"verifica JWKS"| IAM
  GW --> IAM
  GW --> CRM
  GW --> ERP
  GW --> ANA
  GW --> RD

  IAM --> PG
  CRM --> PG
  ERP --> PG
  ANA --> MG

  ERP -.->|"outbox: sale.created"| MQ
  MQ -.-> ANA
```

Responsabilidad de cada contenedor

| Contenedor | Responsabilidad | Base de datos | | --- | --- | --- | | `kubo-gateway` | Único punto de entrada: valida el JWT contra el JWKS, aplica límites de tasa, propaga la identidad y la trazabilidad, enruta | Redis (límites) | | `kubo-iam` | Negocios, usuarios, roles, tokens, auditoría | `kubo_iam` | | `kubo-crm` | Clientes, cifrado de datos personales, cartera | `kubo_crm` | | `kubo-erp` | Catálogo, inventario (kardex), ventas, anulación y **outbox** de eventos | `kubo_erp` | | `kubo-analytics` | Modelo de lectura de ventas, indicadores del tablero | `kubo_analytics` (MongoDB) | | `kubo-web` | Interfaz PWA; sirve los activos y proxea `/api` al gateway | — | | `kubo-tls` | Terminación TLS (Caddy):

HTTPS, redirección de HTTP y HSTS | — | | `kubo-otel` | Collector OpenTelemetry: recibe las trazas OTLP de los cinco servicios (opcional: Tempo + Grafana) | — |

3. Vista de componentes (C4 nivel 3, ejemplo: `kubo-erp`)

```
flowchart TB
  subgraph WEB["kubo-erp · capa de presentación"]
    ROUTER["Router /api/v1"]
    PLUG["Plug Identity<br/>(identidad del gateway)"]
    CTRL["ProductController<br/>SaleController<br/>StockController"]
    JSON["ProductJSON · SaleJSON"]
  end

  subgraph APP["Capa de aplicación"]
    CATALOG["Catalog<br/>catálogo e inventario"]
    SALES["Sales<br/>venta transaccional"]
    PUB["Events.Publisher<br/>(RabbitMQ)"]
    EVT["Events.SaleCreated"]
  end

  subgraph DOM["Capa de dominio"]
    PROD["Product"]
    MOVE["StockMovement"]
    SALE["Sale · SaleItem"]
  end

  subgraph INFRA["Infraestructura"]
    REPO["Ecto.Repo<br/>PostgreSQL"]
    BUS["AMQP"]
  end

  ROUTER --> PLUG --> CTRL --> JSON
  CTRL --> CATALOG
  CTRL --> SALES
  SALES --> PROD
  SALES --> SALE
  CATALOG --> PROD
  CATALOG --> MOVE
  SALES --> PUB
  PUB --> EVT
  PUB --> BUS
  CATALOG --> REPO
  SALES --> REPO
```

Los otros servicios siguen la misma separación. La regla es que el dominio no

conoce HTTP, ni la base de datos, ni el bus de eventos.

4. Flujo crítico: registrar una venta

```
sequenceDiagram
  autonumber
  participant V as Vendedor (PWA)
  participant G as API Gateway
  participant E as kubo-erp
  participant P as PostgreSQL
  participant M as RabbitMQ
  participant A as kubo-analytics
  participant N as MongoDB

  V->>G: POST /api/v1/sales (Bearer JWT)
  G->>G: verifica firma RS256 contra el JWKS
  G->>E: POST /sales + X-User-Id, X-Tenant-Id
  E->>P: BEGIN
  E->>P: SELECT ... FOR UPDATE (productos)
  E->>P: INSERT sale, sale_items
  E->>P: UPDATE products.stock
  E->>P: INSERT stock_movements (kardex)
  E->>P: INSERT outbox_events (sale.created)
  E->>P: COMMIT
  E-->>G: 201 Created (venta)
  G-->>V: 201 Created
  Note over E,P: La venta y su evento se confirman juntos (outbox)
  E-)M: el publicador de barrido entrega sale.created
  M-)A: entrega del evento
  A->>N: deduplica por event_id y proyecta la venta
  Note over V,A: El tablero refleja la venta en menos de un segundo,<br/>sin que la caja
```

> Si el bus está caído, el evento **espera en la bandeja de salida**

> (`outbox_events`) y se entrega al volver; la venta ya está confirmada y el

> negocio sigue vendiendo. `make bus-drill` lo verifica (ver ADR-0009).

Si no hay internet en el local

```
sequenceDiagram
  autonumber
  participant V as Vendedor (PWA)
  participant Q as Cola IndexedDB
  participant G as API Gateway
  participant E as kubo-erp

  V->>Q: cobra sin conexión (venta guardada)
  Note over V,Q: La venta queda "pendiente de sincronizar"
  V->>V: vuelve la conexión (evento online)
  Q->>G: reintenta POST /sales con el token vigente
  G->>E: registra la venta
  E-->>Q: 201 Created
  Q->>Q: elimina la venta de la cola
```

5. Atributos de calidad y cómo se sostienen

| Atributo | Mecanismo | | --- | --- | | **Escalabilidad** | Servicios sin estado (excepto las bases); el gateway escala horizontalmente porque los límites de tasa viven en Redis. Cada base puede migrar a su propia instancia sin tocar código. | | **Disponibilidad** | Tolerancia a fallos en cadena: la venta no depende del bus ni de analítica; el gateway opera sin Redis (fail-open); cada contenedor tiene *healthcheck* y política de reinicio. | | **Seguridad** | JWT asimétrico, cifrado de campos, aislamiento por negocio, auditoría encadenada, límites de tasa y validación en cada frontera. | | **Mantenibilidad** | Un lenguaje por contexto con su ecosistema natural; arquitectura limpia dentro de cada servicio; contratos explícitos entre repos. | | **Portabilidad** | Todo corre en Docker Compose: el mismo artefacto sirve para el mini-PC del local y para un servidor en la nube. | | **Observabilidad** | Sonda `/health` en cada servicio con estado de sus dependencias, `correlation-id` propagado por el gateway y logs JSON estructurados. | | **Costo** | Consumo en reposo cercano a 1 GB de RAM y 12 contenedores; cabe en un VPS de USD 6–12 al mes o en un equipo modesto del local. El stack visual de trazas (Tempo + Grafana) es opcional. |

6. Decisiones arquitectónicas

Cada decisión relevante está registrada como ADR:

| ADR | Decisión | | --- | --- | | [0001](adr/ADR-0001-microservicios-monolito-modular.md) | Microservicios con monolito modular interno | | [0002](adr/ADR-0002-polyrepo-contratos.md) | Polyrepo con contratos como fuente de verdad | | [0003](adr/ADR-0003-multitenancy-hibrido.md) | Multi-tenancy híbrido (`tenant_id` + RLS activo) | | [0004](adr/ADR-0004-poliglotismo.md) | Poliglotismo deliberado: un lenguaje por contexto | | [0005](adr/ADR-0005-eventos-rabbitmq.md) | Eventos con RabbitMQ y publicador tolerante a fallos | | [0006](adr/ADR-0006-cifrado-campos.md) | Cifrado de campos personales con AES-256-GCM | | [0007](adr/ADR-0007-jwt-rs256-jwks.md) | JWT RS256 con JWKS en el gateway | | [0008](adr/ADR-0008-alcance-mvp.md) | Alcance declarado del MVP y recortes conscientes | | [0009](adr/ADR-0009-outbox-transaccional.md) | Outbox

transaccional para la entrega de eventos | | [0010](adr/ADR-0010-rls-activo.md) | RLS activo con interceptor de transacción | | [0011](adr/ADR-0011-bff-capa-formal.md) | BFF como capa formal del gateway | | [0012](adr/ADR-0012-zona-horaria-por-negocio.md) | Zona horaria por negocio | | [0013](adr/ADR-0013-vertical-packs.md) | Vertical Packs: el vertical como dato del negocio | | [0014](adr/ADR-0014-puerto-facturacion-dian.md) | Facturación electrónica DIAN como puerto enchufable | | [0015](adr/ADR-0015-segundo-factor-totp.md) | Segundo factor TOTP del propietario | | [0016](adr/ADR-0016-stock-por-bodega.md) | Stock por bodega y transferencias | | [0017](adr/ADR-0017-puerto-notificaciones.md) | Puerto de notificaciones y buzón del negocio | | [0018](adr/ADR-0018-documentos.md) | Documentos en el ERP con puerto de almacenamiento | | [0019](adr/ADR-0019-rotacion-de-claves.md) | Rotación de claves de cifrado de campo (KEK/DEK) | | [0020](adr/ADR-0020-mtls-interno.md) | mTLS en la malla interna | | [0021](adr/ADR-0021-multi-tenant-saas.md) | Multi-tenant SaaS como modo de despliegue | | [0022](adr/ADR-0022-operador-de-respaldos.md) | Operador de respaldos con verificación de restauración | | [0023](adr/ADR-0023-app-movil.md) | App móvil: la PWA primero, Capacitor cuando el negocio lo pida | | [0024](adr/ADR-0024-superficie-del-operador.md) | Superficie del operador: script hoy, panel cuando haya rol | | [0025](adr/ADR-0025-rol-de-plataforma.md) | Rol de plataforma y panel del operador | | [0026](adr/ADR-0026-puerto-de-cobro.md) | Puerto de cobro y pasarela de pago |

7. Estructura del workspace

```
Kubo/
├── README.md            índice y arranque rápido
├── Makefile             up · down · seed · smoke · pdf
├── kubo-gateway/        API Gateway (TypeScript + NestJS)
├── kubo-iam/            identidad (Java + Spring Boot)
├── kubo-crm/            clientes (Ruby + Rails)
├── kubo-erp/            catálogo, inventario y ventas (Elixir + Phoenix)
├── kubo-analytics/      tablero (Python + FastAPI + MongoDB)
├── kubo-web/            PWA (React + Vite + Tailwind)
├── kubo-infra/          Docker Compose, semilla y prueba de humo
└── kubo-docs/           esta documentación
```

02 — Modelo de datos

Kubo aplica **una base de datos por servicio**: ningún servicio consulta el

esquema de otro. Las referencias entre dominios son identificadores lógicos, no

llaves foráneas.

1. Panorama

```
flowchart LR
  subgraph PG["PostgreSQL 17 · una instancia, tres bases aisladas"]
    subgraph IAMDB["kubo_iam"]
      TENANTS["tenants"]
      USERS["users"]
      REFRESH["refresh_tokens"]
      RESET["password_reset_tokens"]
      MAIL["mail_outbox"]
      AUDIT["audit_logs"]
    end
    subgraph CRMDB["kubo_crm"]
      CUSTOMERS["customers"]
    end
    subgraph ERPDB["kubo_erp"]
      PRODUCTS["products"]
      MOVEMENTS["stock_movements"]
      SALES["sales"]
      ITEMS["sale_items"]
      OUTBOX["outbox_events"]
      COUNTERS["tenant_counters"]
    end
  end

  subgraph MONGO["MongoDB 8"]
    EVENTS["events<br/>(bitácora)"]
    PROJ["sales<br/>(proyección)"]
  end

  USERS -->|"tenant_id"| TENANTS
  REFRESH -->|"user_id"| USERS
  RESET -->|"user_id"| USERS
  CUSTOMERS -.->|"tenant_id (lógico)"| TENANTS
  PRODUCTS -.->|"tenant_id (lógico)"| TENANTS
  SALES -->|"sale_id"| ITEMS
  ITEMS -->|"product_id"| PRODUCTS
  MOVEMENTS -->|"product_id"| PRODUCTS
  SALES -.->|"customer_id (lógico, sin FK)"| CUSTOMERS
  SALES -.->|"sale.created (misma tx)"| OUTBOX
  OUTBOX -.->|"publicador de barrido"| EVENTS
  EVENTS -.->|"proyección"| PROJ
```

2. `kubo_iam` — identidad y auditoría

```
erDiagram
  TENANTS ||--o{ USERS : "tiene"
  USERS ||--o{ REFRESH_TOKENS : "emite"
  USERS ||--o{ PASSWORD_RESET_TOKENS : "recupera"

  TENANTS {
    uuid id PK
    varchar name
    varchar slug UK
    varchar plan
    timestamptz created_at
  }
  USERS {
    uuid id PK
    uuid tenant_id FK
    varchar email "único por negocio"
    varchar password_hash "BCrypt"
    varchar full_name
    varchar role "OWNER|ADMIN|SELLER|ACCOUNTANT|VIEWER"
    varchar status "ACTIVE|DISABLED"
    timestamptz last_login_at
    integer failed_login_attempts "bloqueo por intentos"
    timestamptz locked_until "temporal"
  }
  REFRESH_TOKENS {
    uuid id PK
    uuid user_id FK
    varchar token_hash UK "SHA-256, nunca el valor"
    timestamptz expires_at
    timestamptz revoked_at
    uuid replaced_by "rotación"
  }
  PASSWORD_RESET_TOKENS {
    uuid id PK
    uuid user_id FK
    varchar token_hash UK "SHA-256, un solo uso"
    timestamptz expires_at "30 minutos"
    timestamptz used_at
  }
  MAIL_OUTBOX {
    uuid id PK
    varchar recipient
    varchar subject
    text body "solo transporte log"
    varchar transport "log|smtp"
    timestamptz created_at
  }
  AUDIT_LOGS {
    uuid id PK
    uuid tenant_id
```

```
    uuid user_id
    varchar action
    varchar entity
    varchar entity_id
    varchar prev_hash
    varchar hash "cadena SHA-256"
    smallint hash_version "2 = vigente"
    timestamptz created_at
  }
```

Decisiones del modelo

- `refresh_tokens.token_hash`: si la base se filtra, los tokens no se pueden reutilizar (solo se guarda su SHA-256).
- `replaced_by`: al rotar un token se enlaza con el que lo reemplaza. Si vuelve a usarse uno revocado, se invalida toda la familia (detección de robo).
- `audit_logs.prev_hash` + `hash`: cada registro encadena el anterior; alterar un registro intermedio rompe la cadena y es detectable.
- `users.email` es único **por negocio**, no global.
- `password_reset_tokens`: solo el SHA-256 del enlace, con expiración y uso único; al consumirse se revocan todas las sesiones del usuario.
- `users.failed_login_attempts` + `locked_until`: el bloqueo por intentos vive en la misma fila y se levanta con un acceso correcto o una recuperación.
- `mail_outbox`: buzón de demostración (transporte `log`); con `smtp` el cuerpo no se persiste.
- `audit_logs.hash_version`: cada fila declara con qué algoritmo se calculó su hash; las de la versión 1 solo se verifican por enlace.

3. `kubo_crm` — clientes con datos personales cifrados

```
customers
├── id                          uuid  PK
├── tenant_id                   uuid  (aislamiento por negocio)
├── name                        varchar(160)
├── email                       varchar(180)
├── document_number_encrypted   text      ← AES-256-GCM: base64(iv ‖ tag ‖ ciphertext)
├── document_number_bidx        varchar(64) ← HMAC-SHA256 normalizado (búsqueda exacta)
├── phone_encrypted             text      ← AES-256-GCM
├── phone_bidx                  varchar(64) ← HMAC-SHA256
├── city, address, notes
├── stage                       LEAD | PROSPECT | CUSTOMER
├── credit_limit                numeric(14,2)  ≥ 0
├── deleted_at                  timestamptz    (borrado lógico)
└── created_at / updated_at
```

Índices y restricciones

| Objeto | Propósito | | --- | --- | | `idx_customers_tenant_document_unique` (único, parcial) | Un documento no puede repetirse dentro del mismo negocio; ignora borrados | | `idx_customers_tenant_stage` | Filtros del tablero por etapa | | `customers_stage_check` | Solo los tres estados previstos | | `customers_credit_limit_check` | El cupo no puede ser negativo |

4. `kubo_erp` — catálogo, inventario y ventas

```
erDiagram
  PRODUCTS ||--o{ STOCK_MOVEMENTS : "registra"
  PRODUCTS ||--o{ SALE_ITEMS : "se vende en"
  SALES ||--|{ SALE_ITEMS : "contiene"

  PRODUCTS {
    uuid id PK
    uuid tenant_id
    varchar sku "único por negocio"
    varchar name
    numeric price "precio final con IVA"
    numeric cost
    numeric tax_rate "19.00 por defecto"
    integer stock "proyección del kardex"
    integer min_stock "alerta de reposición"
    boolean active
    timestamptz deleted_at
  }
  STOCK_MOVEMENTS {
    uuid id PK
    uuid tenant_id
    uuid product_id FK
    varchar kind "IN|OUT|ADJUST"
    integer quantity "> 0"
    integer stock_after
    varchar reason
    varchar reference_type "SALE|VOID|MANUAL"
    uuid reference_id
    uuid created_by
    timestamptz inserted_at
  }
  SALES {
    uuid id PK
    uuid tenant_id
    varchar number "V-000001, único por negocio"
    uuid customer_id "referencia lógica al CRM"
    varchar customer_name "copia histórica"
    varchar status "COMPLETED|VOIDED"
    varchar payment_method "CASH|CARD|TRANSFER|CREDIT"
    numeric subtotal
    numeric tax
    numeric total
    uuid sold_by
    timestamptz voided_at
  }
  SALE_ITEMS {
    uuid id PK
    uuid sale_id FK
    uuid tenant_id "denormalizado para RLS"
    uuid product_id FK
```

```
    varchar product_name "copia histórica"
    integer quantity "> 0"
    numeric unit_price "copia histórica"
    numeric tax_rate
    numeric tax_amount
    numeric total
  }
  SUPPLIERS {
    uuid id PK
    uuid tenant_id
    varchar name
    varchar tax_id "NIT único por negocio"
    varchar contact_name
    varchar phone
    varchar email
    boolean active
    timestamptz deleted_at
  }
  PURCHASES {
    uuid id PK
    uuid tenant_id
    varchar number "C-000001, único por negocio"
    uuid supplier_id FK
    varchar supplier_name "copia histórica"
    varchar status "RECEIVED|VOIDED"
    numeric subtotal
    numeric tax
    numeric total
    uuid received_by
    timestamptz received_at
    timestamptz voided_at
  }
  PURCHASE_ITEMS {
    uuid id PK
    uuid purchase_id FK
    uuid tenant_id "denormalizado para RLS"
    uuid product_id FK
    varchar product_name "copia histórica"
    integer quantity "> 0"
    numeric unit_cost "con IVA incluido"
    numeric tax_rate
    numeric tax_amount
    numeric total
  }
```

Reglas de integridad en el motor

| Restricción | Qué garantiza | | --- | --- | | `products_stock_not_negative` | El motor rechaza stock negativo aunque el código falle | | `stock_movements_reference_unique` (único parcial) | Una venta no puede descontar dos veces el mismo producto (idempotencia) | | `sale_items_quantity_positive` |

No existen líneas con cantidad cero o negativa | | `sales_total_not_negative` | No existen ventas con total negativo | | `products_tenant_sku_unique` (único parcial) | Un SKU no se repite dentro del negocio |

Por qué el stock es una proyección: la verdad está en `stock_movements`. `products.stock` se mantiene por velocidad de lectura y se actualiza **dentro de la misma transacción** que inserta el movimiento, con el producto bloqueado (`SELECT ... FOR UPDATE`). Ante cualquier duda se puede reconstruir el saldo sumando el kardex y compararlo con la proyección.

Bandeja de salida (`outbox_events`): el evento `sale.created` se guarda en la misma transacción de la venta (`status PENDING`), con `event_id` único, el `payload` completo y `attempts`/`available_at` para reintentos. El publicador de barrido lo marca `PUBLISHED` (o `FAILED` tras 10 intentos). Queda fuera de RLS a propósito: se lee cruzando negocios para entregar los eventos (ADR-0009).

Numeración atómica (`tenant_counters`): los consecutivos de venta (`sale_seq`) y de compra (`purchase_seq`) se obtienen con un UPSERT (`ON CONFLICT DO UPDATE … RETURNING`), sin leer el máximo ni reintentar; dos cajas nunca reciben el mismo número. La tabla no lleva RLS: es numeración operativa, sin datos de negocio. `sale_items.tenant_id` y `purchase_items.tenant_id` se denormalizan desde su documento para que las políticas de RLS sean comparaciones por índice (la subconsulta anterior fallaba de forma intermitente bajo concurrencia).

Compras y proveedores (Fase 3): `suppliers` guarda el proveedor con su NIT (único por negocio y borrado lógico); `purchases` la cabecera (número `C-000001`, proveedor con copia del nombre, totales, `warehouse_id` —la bodega de entrada, obligatoria desde la migración de compras por bodega— y estado `RECEIVED|VOIDED`) y `purchase_items` el detalle con el costo unitario. La compra suma inventario en la bodega elegida, actualiza el costo del producto sin

IVA y emite `purchase.received`; anularla revierte el stock en esa misma bodega.

5. `kubo_analytics` — modelo de lectura en MongoDB

```
events (bitácora cruda)
├── _id           = event_id del evento  ← índice único: deduplicación
├── event_type    "sale.created"
├── version       1
├── tenant_id
├── occurred_at   date
├── payload       documento original completo
└── received_at   date

sales (proyección consultable)
├── _id            ObjectId
├── sale_id        índice único
├── tenant_id      + sold_at  (índice compuesto)
├── number, status, payment_method
├── customer_id, customer_name
├── subtotal, tax, total        (double: analítica, no contabilidad)
├── items[]        {product_id, product_name, quantity, unit_price, tax_amount, total}
├── sold_at, created_at, updated_at
```

Idempotencia: `events._id` es el `event_id`. Si el mismo evento llega dos

veces, la segunda inserción falla con clave duplicada y se descarta: una venta

nunca se cuenta dos veces.

Precisión numérica: el sistema de registro contable es `kubo-erp`, con

aritmética decimal. Aquí los importes se guardan como punto flotante porque las

agregaciones sobre miles de documentos no necesitan precisión de centavo y así

se aprovechan los operadores nativos de MongoDB. Es una decisión consciente: el

tablero es para decidir, la factura es para cobrar.

6. Convenciones transversales

| Convención | Decisión | | --- | --- | | Identificadores | UUID (v4) en todas las tablas; nunca enteros autoincrementales expuestos | | Fechas | `timestamptz` en PostgreSQL, almacenadas en UTC; la interfaz muestra `America/Bogota` | | Dinero | `numeric(14,2)` y aritmética decimal en el ERP; nunca coma flotante | | Borrado | Lógico (`deleted_at`), nunca físico: la historia comercial se conserva | | Aislamiento | `tenant_id` en toda tabla de negocio + **RLS activo con `FORCE`** en las tres bases

(ADR-0010) | | Migraciones | Versionadas y **inmutables** una vez aplicadas; nunca se edita una migración ya ejecutada |

03 — API

Todas las peticiones entran por el **API Gateway** (`http://localhost:9080` en

desarrollo). El gateway valida el JWT, aplica límites de tasa y enruta al

servicio correspondiente. Ningún servicio de negocio está expuesto directamente.

- **Base**: `/api/v1`

- **Formato**: JSON UTF-8

- **Autenticación**: `Authorization: Bearer <access token>`

- **Trazabilidad**: cabecera `X-Correlation-Id` (si no llega, el gateway la crea)

1. Rutas públicas y protegidas

| Ruta | Acceso | | --- | --- | | `GET /api/v1/health` | Público (lo atiende el gateway) | | `POST /api/v1/auth/login` | Público | | `POST /api/v1/auth/register` | Público | | `POST /api/v1/auth/refresh` | Público (cookie de refresco) | | `POST /api/v1/auth/logout` | Público (cookie de refresco) | | `POST /api/v1/auth/forgot-password` | Público | | `POST /api/v1/auth/reset-password` | Público | | `GET /api/v1/auth/.well-known/jwks.json` | Público | | Todo lo demás | Requiere `Bearer` válido |

2. Enrutamiento

| Prefijo | Servicio | | --- | --- | | `/api/v1/auth`, `/api/v1/users`, `/api/v1/audit` | `kubo-iam` | | `/api/v1/customers` | `kubo-crm` | | `/api/v1/products`, `/api/v1/sales`, `/api/v1/stock` | `kubo-erp` | | `/api/v1/dashboard`, `/api/v1/events` | `kubo-analytics` |

3. Identidad (`kubo-iam`)

`POST /auth/register`

Crea un negocio y su usuario propietario.

```
curl -X POST http://localhost:9080/api/v1/auth/register \
  -H 'Content-Type: application/json' \
  -d '{
    "tenantName": "Tienda La Esquina",
    "fullName": "Administrador",
    "email": "admin@kubo.local",
    "password": "Admin123!"
  }'
```

`POST /auth/login`

```
curl -X POST http://localhost:9080/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@kubo.local","password":"Admin123!"}'
```

Respuesta:

```
{
  "accessToken": "eyJhbGciOiJSUzI1NiIsImtpZCI6...",
  "tokenType": "Bearer",
  "expiresInSeconds": 900,
  "user": {
    "id": "…", "email": "admin@kubo.local", "fullName": "Administrador Kubo",
    "role": "OWNER", "tenantId": "…", "tenantName": "Tienda La Esquina"
  }
}
```

El **access token** dura 15 minutos y se firma con RS256. El **refresh token**

dura 7 días, se guarda hasheado y **rota en cada uso**; reutilizar uno revocado

invalida toda la familia de tokens del usuario. El gateway (BFF) lo entrega en una

cookie `httpOnly` + `SameSite=Strict` (`Path=/api/v1/auth`) y **nunca** aparece

en el cuerpo de la respuesta: un XSS no puede robarlo.

`POST /auth/forgot-password`

Solicita el enlace de recuperación. Responde siempre `204`, exista o no la cuenta

(no permite enumerar usuarios).

```
curl -X POST http://localhost:9080/api/v1/auth/forgot-password \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@kubo.local"}'
```

Con `KUBO_MAIL_TRANSPORT=log` (demostración) el correo queda en la tabla

`mail_outbox`; con `smtp` sale por el servidor configurado.

`POST /auth/reset-password`

Consume el token del enlace (un solo uso, vence en 30 minutos) y cambia la

contraseña; además revoca todas las sesiones y levanta el bloqueo por intentos.

```
curl -X POST http://localhost:9080/api/v1/auth/reset-password \
  -H 'Content-Type: application/json' \
  -d '{"token":"<token del enlace>","newPassword":"NuevaClave123!"}'
```

`GET /auth/me`

Devuelve el perfil del usuario autenticado.

`GET /auth/.well-known/jwks.json`

Llave pública RSA que el gateway usa para verificar firmas.

4. Clientes (`kubo-crm`)

| Método | Ruta | Descripción | | --- | --- | --- | | GET | `/customers?q=&stage=` | Listado (documento y teléfono **enmascarados**) | | GET | `/customers/{id}` | Detalle (valores descifrados) | | GET | `/customers/by-document/{documento}` | Búsqueda exacta por documento (índice ciego) | | POST | `/customers` | Crear | | PATCH | `/customers/{id}` | Actualizar | | DELETE | `/customers/{id}` | Borrado lógico | | GET | `/customers/stats` | Conteos por etapa y cartera |

```
TOKEN=$(curl -s -X POST http://localhost:9080/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@kubo.local","password":"Admin123!"}' | jq -r .accessToken)

curl -X POST http://localhost:9080/api/v1/customers \
  -H "Authorization: Bearer ${TOKEN}" -H 'Content-Type: application/json' \
  -d '{"name":"Panadería El Trigal","document_number":"900123456-7",
       "phone":"3105551234","city":"Armenia","stage":"CUSTOMER"}'
```

5. Catálogo, inventario y ventas (`kubo-erp`)

| Método | Ruta | Descripción | | --- | --- | --- | | GET | `/products?q=&low_stock=` | Catálogo | | POST | `/products` | Crear producto | | PATCH | `/products/{id}` | Actualizar | | DELETE | `/products/{id}` | Borrado lógico | | GET | `/products/stats` | Totales y valor del inventario | | POST | `/products/{id}/stock` | Movimiento de inventario | | GET | `/stock/movements?product_id=` | Kardex |

| GET | `/sales?status=` | Ventas | | POST | `/sales` | Registrar venta | | GET | `/sales/{id}` | Detalle | | POST | `/sales/{id}/void` | Anular venta (devuelve inventario) | | GET | `/sales/stats` | Totales del día | | GET | `/suppliers?q=&active=` | Proveedores (paginado) | | POST | `/suppliers` | Crear proveedor | | PATCH | `/suppliers/{id}` | Actualizar proveedor | | DELETE | `/suppliers/{id}` | Borrado lógico | | GET | `/purchases?status=&supplier_id=` | Compras (paginadas) | | POST | `/purchases` | Registrar compra (suma inventario) | | GET | `/purchases/{id}` | Detalle de compra | | POST | `/purchases/{id}/void` | Anular compra (revierte inventario) | | GET | `/purchases/stats` | Totales de compras y proveedores |

Registrar una compra

```
curl -X POST http://localhost:9080/api/v1/purchases \
  -H "Authorization: Bearer ${TOKEN}" -H 'Content-Type: application/json' \
  -d '{
    "supplier_id": "…",
    "items": [{"product_id": "…", "quantity": 5, "unit_cost": 5000}]
  }'
```

La compra **suma** el inventario, deja el movimiento en el kardex

(`reference_type = PURCHASE`) y actualiza `products.cost` con el valor **sin

IVA** (el costo unitario se interpreta con IVA incluido, igual que el precio de

venta). `POST /purchases/{id}/void` la anula y revierte el stock.

Registrar una venta

```
curl -X POST http://localhost:9080/api/v1/sales \
  -H "Authorization: Bearer ${TOKEN}" -H 'Content-Type: application/json' \
  -d '{
    "items": [
      {"product_id": "…", "quantity": 2},
      {"product_id": "…", "quantity": 1}
    ],
    "customer_id": "…",
    "customer_name": "Panadería El Trigal",
    "payment_method": "CASH"
  }'
```

Respuesta `201`:

```
{
  "data": {
    "id": "…", "number": "V-000004", "status": "COMPLETED",
    "subtotal": "31680.67", "tax": "6019.33", "total": "37700.00",
    "items": [{ "product_name": "Cafe molido 250 g", "quantity": 2,
                "unit_price": "9500.00", "tax_amount": "3033.61", "total": "19000.00" }]
  }
}
```

Reglas de negocio aplicadas

| Situación | Respuesta | | --- | --- | | Venta sin líneas | `422 EMPTY_SALE` | | Producto inexistente o de otro negocio | `422 PRODUCT_NOT_FOUND` | | Cantidad mayor al stock | `409 INSUFFICIENT_STOCK` (con el stock disponible) | | Venta ya anulada | `409 ALREADY_VOIDED` | | Colisión del número de venta | reintento automático (hasta 3) |

Ajustar inventario

```
curl -X POST http://localhost:9080/api/v1/products/{id}/stock \
  -H "Authorization: Bearer ${TOKEN}" -H 'Content-Type: application/json' \
  -d '{"kind":"IN","quantity":40,"reason":"Compra a proveedor"}'
```

`kind` acepta `IN` (entrada), `OUT` (salida) y `ADJUST` (fijar el stock exacto).

6. Tablero (`kubo-analytics`)

| Método | Ruta | Descripción | | --- | --- | --- | | GET | `/dashboard/overview` | **Vista compuesta (BFF)**: el gateway consulta en paralelo las 7 vistas y devuelve una sola respuesta | | GET | `/dashboard/summary` | Ventas, ingreso, IVA, ticket promedio, unidades, hoy | | GET | `/dashboard/sales-by-day?days=14` | Serie diaria (en la zona horaria del negocio) | | GET | `/dashboard/top-products?limit=10` | Productos más vendidos | | GET | `/dashboard/top-customers?limit=10` | Mejores clientes | | GET | `/dashboard/payment-methods` | Distribución por medio de pago | | GET | `/dashboard/recent-sales?limit=10` | Últimas ventas | | GET | `/dashboard/rotation` | Productos con más salida en 7 días | | POST | `/events` | Vía de respaldo para reenviar eventos por HTTP |

Vista compuesta (patrón BFF)

```
curl http://localhost:9080/api/v1/dashboard/overview -H "Authorization: Bearer ${TOKEN}"
```

```
{
  "data": {
    "summary": { "sales_count": 10, "revenue": 141900.0, "today": { "revenue": 0, "sales_
    "sales_by_day": [ { "date": "2026-09-23", "revenue": 141900.0, "sales_count": 10 } ],
    "top_products": [ { "product_name": "Cafe molido 250 g", "quantity": 6, "revenue": 57
    "payment_methods": [ { "payment_method": "CASH", "revenue": 45300.0, "sales_count": 3
    "recent_sales": [ { "number": "V-000010", "total": 35700.0 } ],
    "rotation": [ { "product_name": "Panela 500 g", "quantity_7d": 4 } ],
    "customers": { "total": 11, "by_stage": { "LEAD": 8, "PROSPECT": 1, "CUSTOMER": 2 } }
  }
}
```


Por qué existe: sin esta capa la PWA necesitaba **siete viajes de red** para pintar el tablero (108 ms medidos). El gateway consulta los servicios en paralelo dentro de la red privada y responde en ~25 ms. Si una vista falla, la respuesta incluye `unavailable: ["top_products"]` y el tablero se muestra parcial en lugar de caer.

Zona horaria del negocio

Los indicadores de «hoy» y la serie diaria se calculan en la zona horaria del negocio (`KUBO_TIMEZONE`, por defecto `America/Bogota`), no en UTC. Sin esto, en Colombia las ventas de 19:00 a 23:59 se atribuirían al día siguiente. La respuesta de `/sales/stats` incluye `timezone` y `business_date` para poder auditarlo.

7. Eventos de dominio

El ERP publica en el exchange *topic* `kubo.events`. Sobre del evento:

```
{
  "event_id": "9f1c…",
  "event_type": "sale.created",
  "version": 1,
  "occurred_at": "2026-09-23T17:50:12.480Z",
  "tenant_id": "…",
  "data": {
    "sale_id": "…", "number": "V-000004", "status": "COMPLETED",
    "customer_id": "…", "customer_name": "Panadería El Trigal",
    "payment_method": "CASH",
    "subtotal": "31680.67", "tax": "6019.33", "total": "37700.00",
    "sold_by": "…", "sold_at": "2026-09-23T17:50:12.470Z",
    "items": [
      { "product_id": "…", "product_name": "Cafe molido 250 g", "quantity": 2,
        "unit_price": "9500.00", "tax_rate": "19.00",
        "tax_amount": "3033.61", "total": "19000.00" }
    ]
  }
}
```

Contrato de consumo: el consumidor debe deduplicar por `event_id` y tolerar

campos nuevos (versionado aditivo). Los mensajes fallidos van a la cola

`kubo.analytics.sale_created.dlq`.

8. Formato de errores

Todos los servicios responden con la misma forma:

```
{
  "code": "INSUFFICIENT_STOCK",
  "message": "Stock insuficiente de Cafe molido 250 g: disponible 7"
}
```

| Código HTTP | Cuándo | | --- | --- | | `400` | Datos inválidos (`VALIDATION_ERROR`, `INVALID_QUANTITY`) | | `401` | Sin token, token inválido/expirado, identidad ausente | | `404` | Recurso inexistente o de otro negocio | | `409` | Conflicto de estado (`INSUFFICIENT_STOCK`, `ALREADY_VOIDED`, `EMAIL_ALREADY_EXISTS`) | | `422` | Regla de negocio incumplida (`EMPTY_SALE`, `PRODUCT_NOT_FOUND`) | | `429` | Límite de tasa excedido | | `502` | Microservicio no disponible (`UPSTREAM_UNAVAILABLE`) |

Los mensajes nunca incluyen trazas ni detalles internos; los detalles técnicos

quedan en los logs con el `correlation-id` de la petición.

9. Límites de tasa

| Ámbito | Límite por defecto | | --- | --- | | Rutas de negocio | 600 peticiones/minuto por negocio (o por IP si no hay token) | | Rutas de autenticación | 40 peticiones/minuto (frena fuerza bruta) |

Las respuestas incluyen `X-RateLimit-Limit` y `X-RateLimit-Remaining`. Al

excederse: `429 RATE_LIMIT_EXCEEDED`.

04 — Seguridad

Kubo maneja datos personales de clientes, precios de compra y cartera: la

seguridad no es un añadido, es parte del diseño. Este documento explica **qué se

implementó, cómo se verifica y qué queda pendiente**.

1. Modelo de confianza

```
flowchart LR
  EXT["Red del local<br/>(posible red compartida)"]
  GW["API Gateway<br/>único punto expuesto"]
  NET["Red privada de contenedores<br/>kubo-net"]
  SVC["Microservicios<br/>sin puertos públicos"]

  EXT -->|"HTTPS + JWT"| GW
  GW -->|"cabeceras de identidad<br/>X-User-Id · X-Tenant-Id"| NET
  NET --> SVC
```

- Solo el gateway (9080) y la PWA (3000) se publican. Los servicios 9081–9084

solo se mapean al host para diagnóstico local; **en producción no se publican**.

- La identidad se establece **una sola vez**, en el gateway. Los servicios

internos confían en las cabeceras porque no son alcanzables desde fuera.

- El gateway **elimina cualquier cabecera `X-User-*` que envíe el cliente**

antes de inyectar la identidad verificada: sin esto, cualquiera podría

suplantar a otro usuario con un `curl`.

2. Autenticación

| Control | Implementación | | --- | --- | | Contraseñas | **BCrypt** (coste 10). Nunca se persisten ni se registran en logs | | Access token | **JWT RS256**, 15 minutos, con `iss`, `aud`, `jti`, `exp` y claims de negocio (`tenant_id`, `role`) | | Verificación | Solo el gateway, contra el **JWKS** del servicio de

identidad, con caché y refresco automático | | Refresh token | Valor aleatorio de 384 bits; se guarda **solo su SHA-256** | | Rotación | Cada refresh emite uno nuevo y revoca el anterior | | Detección de reuso | Si se usa un token ya rotado, se **invalida toda la familia** del usuario y se audita `REFRESH_REUSE_DETECTED` | | Cierre de sesión | Revoca el refresh token en el servidor, no solo en el navegador | | Almacenamiento del refresh | **Cookie `httpOnly` + `SameSite=Strict`** emitida por el gateway (BFF); el cuerpo de la respuesta nunca lo incluye | | Bloqueo de cuenta | Tras 5 intentos fallidos la cuenta se bloquea 15 minutos; el acceso correcto o la recuperación lo levantan | | Recuperación de contraseña | Enlace de un solo uso, hasheado en base, vence en 30 minutos; al usarse revoca todas las sesiones |

Por qué asimétrico: con un secreto compartido, comprometer cualquier servicio

permitiría *emitir* tokens válidos para cualquier usuario. Con RS256, la llave

privada vive solo en `kubo-iam`.

3. Cifrado

| Ámbito | Mecanismo | Dónde se verifica | | --- | --- | --- | | En tránsito (externo) | TLS terminado en Caddy (`kubo-tls`, puertos 3080/3443) con HSTS y redirección de HTTP | `make smoke` bloque 10 · `05-despliegue.md` §3 | | En tránsito (interno) | Red privada de contenedores; mTLS previsto para clúster | — | | **Datos personales** | **AES-256-GCM** campo a campo (documento, teléfono) | `make smoke` inspecciona el texto cifrado en PostgreSQL | | Búsqueda sobre cifrado | **Índice ciego** HMAC-SHA256 normalizado | `make smoke` verifica que el índice no contiene el valor | | Contraseñas | BCrypt con sal por usuario | — | | Respaldos | Cifrado con `age` antes de salir del servidor | Implementado: `make backup` / `make restore-drill` |

Formato almacenado: `base64(iv ‖ tag ‖ ciphertext)`. El **tag de GCM** hace que

cualquier manipulación del texto cifrado sea detectable: el descifrado falla y el

servicio devuelve `null` en lugar de un valor corrupto.

Gestión de claves

- Las claves llegan por variable de entorno y **nunca** están en el repositorio

(`.env` está en `.gitignore`; solo se versiona `.env.example` con ceros).

- `KUBO_FIELD_ENCRYPTION_KEY` y `KUBO_BLIND_INDEX_KEY` son de 32 bytes en hex;

el servicio **falla al arrancar** si no cumplen el formato, en lugar de cifrar

con una clave débil.

- El servicio **rechaza explícitamente las claves de ejemplo** de `.env.example`:

desplegar copiando la plantilla sin generar claves produce un error claro (`«conserva el valor de ejemplo; genere una clave real»`) en vez de proteger los datos con una clave conocida por cualquiera. Cubierto por dos pruebas unitarias.

- La llave de firma JWT puede llegar como PEM; si no, el servicio genera un par **efímero** en memoria y lo advierte en los logs (solo desarrollo).
- **Pendiente**: envoltura KEK/DEK por negocio con KMS para la versión en nube, y rotación de claves de campo (exige re-cifrado por lotes).

4. Aislamiento entre negocios

1. Toda tabla de negocio lleva `tenant_id` y toda consulta lo filtra.
2. El `tenant_id` **nunca** viene del cuerpo ni de un parámetro: se toma del claim del JWT, que el gateway propaga como cabecera.
3. **RLS activo con `FORCE` en las tres bases** (P-02, ADR-0010). Cada petición abre una transacción y fija `app.tenant_id` con `set_config(..., true)`; sin contexto, una consulta devuelve **cero filas**. Las operaciones que cruzan negocios por diseño (autenticación por correo, cadena de auditoría, semilla, respuesta a incidentes) usan la marca `app.system`, deliberada y acotada.
4. **Verificación automática**: `make smoke` comprueba que sin contexto no hay filas y con contexto sí, en IAM, CRM y ERP, además de registrar un segundo negocio y comprobar que no ve ni un cliente ni un producto del primero.

5. Autorización

Roles definidos: `OWNER`, `ADMIN`, `SELLER`, `ACCOUNTANT`, `VIEWER`. El rol viaja en el token y el gateway lo propaga; cada servicio decide qué permite. En el MVP el control es por rol y negocio (no hay permisos finos por recurso), lo que se documenta como alcance.

6. Auditoría

`kubo-iam` mantiene `audit_logs` **append-only con cadena de hash**: cada registro incluye el hash del anterior, de modo que alterar o borrar un registro intermedio rompe la cadena y es detectable. Se auditan, entre otros: registro de negocio, inicio de sesión correcto y fallido, rotación de token, reuso detectado.

Serialización de la cadena: leer el último hash e insertar el nuevo debe ser atómico. Sin un bloqueo, dos peticiones concurrentes leerían el mismo hash anterior y la cadena quedaría partida en dos ramas, lo que haría indetectable una manipulación posterior. `AuditService` toma un `pg_advisory_xact_lock` al inicio de la transacción: se libera solo al terminar y no bloquea ninguna tabla.

Los logs son **JSON estructurados** y el gateway **redacta la cabecera `Authorization`** y las cookies. Nunca se registran contraseñas, tokens ni datos personales: Rails filtra `document_number`, `phone` y `email` en sus logs.

7. Protección de la interfaz y la API

| Amenaza | Mitigación | | --- | --- | | Fuerza bruta de contraseñas | Límite de 40 peticiones/minuto en rutas de autenticación + auditoría de intentos fallidos | | Abuso de la API | Límite de 600 peticiones/minuto por negocio o IP | | XSS | React escapa por defecto; no se usa `dangerouslySetInnerHTML`; cabeceras de seguridad con Helmet | | Clickjacking | `X-Frame-Options` de Helmet | | MIME sniffing | `X-Content-Type-Options: nosniff` | | Cabeceras de la PWA | nginx sirve `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin` y `Permissions-Policy` (cámara, micrófono y ubicación denegados) en todas las respuestas, incluidos los activos con caché | | Inyección SQL | Consultas parametrizadas en los cuatro lenguajes (JPA, ActiveRecord, Ecto, PyMongo); **no hay SQL concatenado** | | Inyección en búsquedas | Los comodines `%` y `_` se eliminan del término de búsqueda | | CSRF | La API no usa cookies de sesión: la autenticación es por cabecera `Authorization` | | Enumeración de usuarios | El login responde el mismo mensaje para correo inexistente y contraseña incorrecta | | Fuga de datos entre negocios | Aislamiento por `tenant_id` + prueba automática |

Los diálogos de la PWA atrapan el foco (Tab y Shift+Tab no salen del panel), cierran con Escape y devuelven el foco al elemento de origen: el comportamiento está cubierto por una prueba unitaria y las páginas se auditan con axe en E2E

(P-26).

Sobre el almacenamiento del refresh token

El refresh token **no toca el navegador**: el gateway (BFF) lo recibe del IAM y lo

deja en una cookie `httpOnly` + `SameSite=Strict` con `Path=/api/v1/auth`, y lo

elimina del cuerpo de la respuesta (P-03, verificado en el humo). El access token

(15 minutos) vive solo en memoria. El refresh rota en cada uso y el reuso se

detecta invalidando la familia completa: el humo comprueba que, tras un reuso, ni

la cookie rotada sigue sirviendo. En producción con HTTPS, `KUBO_COOKIE_SECURE=true`

marca además la cookie como `Secure`.

8. Cumplimiento normativo

| Norma | Cómo se aborda | | --- | --- | | **Ley 1581 de 2012** (protección de datos personales, Colombia) | Cifrado de datos personales, minimización (enmascarado en listados), borrado lógico reversible, trazabilidad de accesos. **Pendiente**: registro de autorizaciones del titular, política de retención y procedimiento de supresión definitiva | | **Facturación electrónica DIAN** | Fuera del alcance del MVP (requiere ser Proveedor Tecnológico autorizado). El dominio deja preparado el puerto de facturación para emitir UBL 2.1 con CUFE y QR en la Fase 4 (P-18) | | **Habeas data en la interfaz** | El manual de usuario documenta qué datos se guardan, para qué y cómo se solicitan al cliente |

9. Verificación

```
make smoke
```

La prueba de humo comprueba, entre otras cosas:

- Petición sin token → `401`.

- Token con firma falsificada → `401`.

- Cabecera `X-User-Id` inyectada por el cliente → **ignorada** (`401`).

- Documento cifrado en PostgreSQL y legible solo por el detalle autorizado.

- Listado con el documento enmascarado.

- Índice ciego sin el valor en claro.

- Segundo negocio que no ve datos del primero.

10. Pendientes de seguridad

| Pendiente | ID · Fase | Riesgo que cierra | | --- | --- | --- | | Segundo factor (TOTP) para el propietario | P-30 · Fase 4 | Robo de credenciales | | mTLS entre gateway y servicios | P-28 · Fase 5 | Movimiento lateral dentro del clúster | | Rotación de claves de cifrado de campo | P-29 · Fase 5 | Compromiso de una clave a largo plazo |

> **Resueltos**: en la Fase 1, cookie `httpOnly` (P-03), RLS activo (P-02) y

> límite de tasa por usuario (P-31); en la Fase 2, SAST/escaneo de secretos y

> dependencias en CI (P-06) y contratos ejecutables (P-09). Ver

> [`11-plan-de-cierre.md`](11-plan-de-cierre.md).

05 — Despliegue y operación

1. Requisitos

| Recurso | Mínimo | Recomendado | | --- | --- | --- | | CPU | 2 núcleos | 4 núcleos | | RAM | 4 GB | 8 GB | | Disco | 20 GB | 60 GB SSD | | Sistema | Linux con Docker Engine 24+ y el plugin `docker compose` | Ubuntu 24.04 LTS | | Red | Salida a internet solo para construir/actualizar | — |

El sistema completo consume **menos de 2 GB de RAM en reposo** (límites por

contenedor definidos en `kubo-infra/docker-compose.yml`).

2. Instalación en el local del negocio

```
git clone <url-del-workspace> kubo && cd kubo
cp kubo-infra/.env.example kubo-infra/.env
```

Editar `kubo-infra/.env` y **cambiar todos los valores**. Generar las claves:

```
openssl rand -hex 32   # KUBO_FIELD_ENCRYPTION_KEY
openssl rand -hex 32   # KUBO_BLIND_INDEX_KEY
openssl rand -hex 64   # KUBO_CRM_SECRET_KEY_BASE
openssl rand -base64 48 # KUBO_ERP_SECRET_KEY_BASE
```

Levantar:

```
make up      # construye y arranca los 12 contenedores
make seed    # datos de demostración (omitir en producción)
make smoke   # verificación end-to-end
```

Abrir `http://<ip-del-servidor>:3000` e ingresar con el administrador creado.

Puertos

| Servicio | Host | Contenedor | | --- | --- | --- | | PWA | 3000 | 80 | | **TLS (Caddy): HTTPS · HTTP** | **3443 · 3080** | 443 · 80 | | **OTel collector: gRPC · HTTP** | **4317 · 4318** | 4317 · 4318 | | Grafana (perfil `observability`) | 3001 | 3000 | | API Gateway | 9080 | 8080 | | IAM · CRM · ERP · Analítica | 9081–9084 | 8081–8084 | | PostgreSQL · MongoDB | 5433 · 27018 | 5432 · 27017 | | RabbitMQ (AMQP · consola) | 5673 · 15673 | 5672 · 15672 | | Redis | 6380 | 6379 |

Los puertos se eligieron **fuera del rango 8000–8100** para no chocar con otros

servicios que ya corran en la máquina del negocio.

3. Puesta en producción (checklist)

- [ ] `kubo-infra/.env` con todas las claves cambiadas y el archivo **fuera del control de versiones**.

- [ ] `KUBO_JWT_PRIVATE_KEY` con una llave RSA real (si queda vacía, el servicio genera una efímera y los tokens se invalidan al reiniciar).

- [ ] TLS: `KUBO_TLS_DOMAIN` con el dominio del negocio (ver abajo). El servicio `kubo-tls` ya está en el compose.

- [ ] `KUBO_COOKIE_SECURE=true` cuando el acceso sea siempre por HTTPS.

- [ ] Correo real: `KUBO_MAIL_TRANSPORT=smtp` con `KUBO_SMTP_HOST` y credenciales (si se deja `log`, el enlace de recuperación queda en el buzón de demostración).

- [ ] **No publicar** los puertos 9080–9084 en producción: la PWA llega al gateway por la red interna de contenedores (nginx proxea `/api`), y los servicios de negocio solo deben ser alcanzables por el gateway. Publicar el 9080 permitiría inyectar `X-Forwarded-For` y saltarse el límite de autenticación.

- [ ] `KUBO_SEED_ENABLED=false` (no sembrar datos de demostración).

- [ ] Respaldos programados (sección 5) y un simulacro de restauración ejecutado.

- [ ] Monitoreo de las sondas `/api/v1/health` de los cinco servicios.

TLS con Caddy (incluido)

El servicio `kubo-tls` (Caddy) termina HTTPS en el local y redirige HTTP a HTTPS:

- `https://<dominio>:3443` sirve la PWA; `http://<dominio>:3080` redirige.

- Con `KUBO_TLS_DOMAIN=localhost` (por defecto) Caddy usa su CA interna

(`tls internal`): válido para la red del local, el navegador pedirá aceptar el

certificado una vez.

- Para un **dominio público**: ponga `KUBO_TLS_DOMAIN=kubo.minegocio.co` en el

`.env`, quite `tls internal` de `kubo-infra/caddy/Caddyfile` y Caddy gestionará

el certificado Let's Encrypt automáticamente.

La PWA ya proxea `/api` al gateway, así que un único dominio con HTTPS cubre todo

el sistema. La cabecera `Strict-Transport-Security` se envía en todas las

respuestas.

Trazas (OpenTelemetry)

El collector `kubo-otel` recibe trazas OTLP de los cinco servicios y las escribe

en su log: `docker logs kubo-otel` es la comprobación básica (el humo la verifica).

Para el stack visual:

```
make observability   # agrega Tempo y Grafana; Grafana en http://localhost:3001
```

No forma parte de la instalación por defecto: un local de barrio no necesita el

stack visual y así el consumo se mantiene bajo. En producción, fije la versión de

las imágenes de observabilidad (hoy usan etiquetas).

4. Operación diaria

| Comando | Para qué | | --- | --- | | `make ps` | Ver el estado y la salud de cada contenedor | | `make logs` | Logs en vivo de todos los servicios | | `make restart` | Reiniciar los servicios de aplicación | | `make smoke` | Verificar el flujo completo (útil tras una actualización) | | `make down` | Detener el sistema conservando los datos | | `make clean` | Detener **y borrar** los volúmenes (pierde los datos) |

Actualización

```
git pull
make up      # reconstruye solo lo que cambió
make smoke   # confirma que todo sigue funcionando
```

Las migraciones se aplican solas al arrancar cada servicio

(`KUBO_RUN_MIGRATIONS=true`): Flyway en IAM, `db:prepare` en Rails y

`ecto.migrate` en Phoenix. Una migración aplicada **nunca se edita**; se crea una

nueva.

5. Respaldos

Automatizados con `make backup` (`kubo-infra/scripts/backup.sh`):

| Qué | Cómo | Frecuencia | | --- | --- | --- | | PostgreSQL | `pg_dump -Fc` por base (`kubo_iam`, `kubo_crm`, `kubo_erp`) | Diaria | | MongoDB | `mongodump --archive` de `kubo_analytics` | Diaria | | Configuración | `kubo-infra/.env` cifrado con `age` (si hay destinatario) o `chmod 600` | Diaria | | Retención | `KUBO_BACKUP_RETENTION_DAYS` (14 por defecto) | Automática |

```
make backup          # deja un directorio con fecha en kubo-infra/backups/
make restore-drill   # restaura en bases kubo_drill_* y compara filas
```

Programación diaria con systemd (copiar a `/etc/systemd/system/`):

```
sudo cp kubo-infra/systemd/kubo-backup.{service,timer} /etc/systemd/system/
sudo systemctl enable --now kubo-backup.timer   # 03:30 con retraso aleatorio
```

Objetivos: RPO 24 h · RTO 4 h. **Un respaldo que nunca se restauró no es un

respaldo**: `make restore-drill` toma un respaldo nuevo, lo restaura en bases de

prueba, compara el número de filas de las tablas clave y mide el tiempo (en este

equipo: 14/14 en 6 segundos). Ejecútalo mensualmente y tras cada cambio de

esquema.

Importante: MongoDB guarda una **proyección** reconstruible desde los eventos

del ERP. PostgreSQL es la fuente de verdad y es lo que no puede perderse.

6. Diagnóstico rápido

| Síntoma | Primera revisión | | --- | --- | | La PWA carga pero no hay datos | `make ps` — ¿el gateway está *healthy*? | | `401` en todas las llamadas | El gateway no puede leer el JWKS: revisa `kubo-iam` | | `502 UPSTREAM_UNAVAILABLE` | El microservicio destino está caído: `docker logs kubo-erp` | | El tablero no refleja ventas | `docker logs kubo-analytics` — ¿el consumidor está conectado? | | `429` en operación normal | Ajusta `KUBO_RATE_LIMIT_PER_MINUTE` en el gateway | | El inventario no cuadra | Compara `products.stock` con la suma del kardex (consulta en `02-modelo-datos.md`) |

7. Escalado

El sistema está diseñado para crecer sin reescribir:

1. **Más memoria/CPU**: subir los `mem_limit` del compose.

2. **Más carga en la caja**: escalar el ERP horizontalmente (`docker compose up -d --scale kubo-erp=3`) porque es sin estado; el gateway reparte.

3. **Bases separadas**: mover cada base a su propia instancia cambiando solo la URL de conexión.

4. **Clúster**: los mismos artefactos corren en Kubernetes; la malla mTLS

(ADR-0020) y el RLS activo (ADR-0010) ya están implementados; el empaquetado

(manifiestos/Helm) queda como backlog declarado (ADR-0028).

8. Lo que queda (declarado)

| Pendiente | Tipo | Impacto | | --- | --- | --- | | Adaptador del proveedor tecnológico DIAN | Externo (habilitación) | La firma XAdES y el envío los hace el PT; el puerto ya está listo (guía `13`) | | Adaptadores reales de notificaciones (WhatsApp/SMTP) | Proveedor externo | El puerto y el buzón ya funcionan; falta la entrega real | | Empaquetado Kubernetes (manifiestos/Helm) | Backlog | Necesario solo para multi-nodo (>50 negocios); el compose es el despliegue soportado | | App móvil nativa (Capacitor) | Decisión tomada | Solo cuando el negocio pida cámara, push o impresión (ADR-0023) | | Capturas internas y video narrado | Tarea del usuario | Evidencia de demostración (`09-demo-guion.md`) |

> Las fases 0–6 están cerradas y verificadas; ver

> [`11-plan-de-cierre.md`](11-plan-de-cierre.md).

9. Demo público 100 % open source (ADR-0028)

9.1 Con acceso al router (ruta directa)

1. **Dominio gratis**: crear cuenta en [DuckDNS](https://www.duckdns.org)

(login con GitHub) y el subdominio (p. ej. `kubo-app`); copiar el token y

guardarlo en `.env` como `KUBO_DUCK_TOKEN`.

2. **Perfil público** en `kubo-infra/.env`: `KUBO_TLS_DOMAIN=kubo-app.duckdns.org`, `KUBO_CADDYFILE=Caddyfile.public`, `KUBO_TLS_HTTP_PORT=80`, `KUBO_TLS_HTTPS_PORT=443` y `KUBO_COOKIE_SECURE=true`.

3. **IP dinámica**: `./kubo-infra/scripts/ddns-duckdns.sh` (cron cada 5 minutos) mantiene el registro A al día.

4. **Router**: reenviar 80 y 443 a la IP del portátil (reservarla antes en el DHCP del router) y no abrir nada más.

5. `make up` y verificar con `./kubo-infra/scripts/demo-check.sh`: Caddy emite el certificado Let's Encrypt (HTTP-01) y `https://kubo-app.duckdns.org` sirve la PWA. Los puertos 9080–9084 y las bases no deben responder desde internet (siguen en loopback).

6. **Respaldo offsite**: Storj (S3, 25 GB gratis) configurando `KUBO_BACKUP_OFFSITE_DIR=storj:kubo-backups` y las credenciales `KUBO_STORJ_ACCESS_KEY`/`KUBO_STORJ_SECRET_KEY` en `.env` (el operador trae rclone y verifica la copia); luego `make backup-operator` + `make restore-drill`.

9.2 Sin acceso al router (túnel 100 % OSS con zrok)

Si el router no permite administración (ni UPnP), el borde es un túnel open source; el compose no cambia:

1. Instalar el cliente `zrok2` (Apache-2.0) desde [zrok releases](https://github.com/openziti/zrok/releases).

2. Crear la cuenta: `zrok2 invite` (el token de invitación se obtiene en zrok.io) y `zrok2 enable <token>`.

3. Compartir: `./kubo-infra/scripts/demo-tunnel.sh start` (URL estable `https://kubo.shares.zrok.io`; `status` y `stop` para operarlo). Manualmente:

`zrok2 share public --headless -n public:kubo http://localhost:3000`.

4. `KUBO_COOKIE_SECURE=true` en `.env` y

`docker compose up -d kubo-gateway`.

5. Detrás del túnel todos los visitantes comparten IP: el límite de

autenticación se vuelve global (súbelo temporalmente si hay muchos

espectadores).

6. Verificar: `./kubo-infra/scripts/demo-check.sh` comprueba el stack local,

el DNS, el servicio del túnel y la URL pública del demo. La ruta directa

(§9.1) se reporta como informativa hasta que exista reenvío de puertos.

06 — Manual de usuario

Guía para el dueño del negocio y sus vendedores. No requiere conocimientos

técnicos.

1. Ingresar

1. Abre el navegador en la dirección que te entregó el instalador

(por ejemplo `http://192.168.1.50:3000`).

2. Escribe tu correo y contraseña.

3. Presiona **Ingresar**.

Instala Kubo como aplicación (recomendado): en el navegador busca el ícono

de instalar (o el menú ⋮ → *Instalar aplicación*). Queda un acceso directo como

cualquier otro programa y abre a pantalla completa.

> La primera vez recibirás el usuario administrador. Cámbialo apenas puedas.

2. El tablero

Es la primera pantalla y responde una pregunta: **¿cómo va el negocio hoy?**

| Indicador | Qué significa | | --- | --- | | **Ventas de hoy** | Cuánto se vendió hoy y cuántas transacciones | | **Ingreso acumulado** | Total vendido desde que usas Kubo | | **Ticket promedio**

| Lo que gasta un cliente en promedio | | **Unidades vendidas** | Cuántos productos salieron | | **Ventas de los últimos 14 días** | La tendencia: ¿sube o baja? | | **Medios de pago** | Cuánto entra en efectivo, tarjeta o transferencia | | **Productos más vendidos** | En qué se está yendo la mercancía | | **Últimas ventas** | El detalle reciente, venta por venta |

Si el local se queda sin internet, verás la etiqueta **Sin internet**: el

tablero muestra lo último que se sincronizó y puedes seguir vendiendo.

3. Vender (punto de venta)

1. Entra a **Vender**.

2. Toca los productos para agregarlos al carrito (usa el buscador si tienes

muchos).

3. Ajusta cantidades con **+** y **–**.

4. Elige el **cliente** (o deja *Consumidor final*).

5. Elige el **medio de pago**: efectivo, tarjeta, transferencia o crédito.

6. Presiona **Cobrar**.

Lo que ocurre al cobrar:

- Se descuenta el inventario en el mismo instante.

- Se asigna un número de venta (V-000001, V-000002…).

- El tablero se actualiza solo, en segundos.

Vender sin internet

Si se cae la conexión, Kubo **no te deja tirado**: la venta se guarda en el

navegador y aparece la etiqueta *«1 venta por sincronizar»*. Cuando vuelva la

conexión:

- La aplicación la envía automáticamente, o

- Presiona **Sincronizar** en la barra superior.

> Mientras no haya conexión no podrás consultar el stock más reciente de otros

> equipos, pero las ventas nunca se pierden.

4. Productos e inventario

En **Productos** ves el catálogo con precio, IVA y existencias.

- **Nuevo producto**: pide código (SKU), nombre, precio de venta, costo, tarifa de IVA y stock mínimo.
- El **precio de venta es el precio final que paga el cliente** (con IVA incluido). Kubo desagrega el impuesto al facturar.
- El **stock mínimo** dispara la alerta de reposición.
- **Ajustar inventario** (ícono de cajas): registra entradas por compra, salidas por merma o conteos físicos.

Tipo	Cuándo usarlo
Entrada (compra)	Llegó mercancía del proveedor
Salida (merma, consumo)	Se dañó, se venció o se usó internamente
Fijar stock exacto	Después de un conteo físico

Escribe siempre el **motivo**: queda en el kardex con fecha y usuario.

- **Archivar** (papelera): el producto deja de venderse pero su historia se conserva. Nunca se borra nada de verdad.

Cuando algo está en o por debajo del mínimo, aparece un aviso amarillo arriba: es el momento de programar una compra.

Registrar una compra

En **Compras** registras la mercancía que llega del proveedor: eliges el proveedor, la **bodega de entrada** (la principal viene preseleccionada) y una o más líneas con producto, cantidad y costo con IVA. Al guardar, el stock entra a esa bodega, el costo del producto se actualiza al valor sin IVA y el movimiento queda en el kardex. **Anular** una compra revierte el stock en la misma bodega.

5. Clientes

En **Clientes** está tu cartera.

- **Nuevo cliente**: nombre o razón social, documento (NIT o cédula), teléfono, correo, ciudad, dirección, etapa y cupo de crédito.
- **Etapas**: *Prospecto nuevo* → *En negociación* → *Cliente*. Sirve para saber a quién hay que llamar.
- El **documento y el teléfono se guardan cifrados**: si alguien se lleva una copia de la base de datos, no puede leerlos.
- En la lista solo se ven los últimos dígitos (`*******432`); el dato completo aparece al abrir el cliente.

> **Protección de datos**: Kubo guarda los datos personales únicamente para gestionar la relación comercial. Informa a tus clientes qué guardas y para qué, y atiende sus solicitudes de corrección o eliminación.

6. Preguntas frecuentes

¿Necesito internet para vender?

No. Solo para sincronizar; las ventas se guardan localmente.

¿Puedo tener varias cajas al mismo tiempo?

Sí. Cada equipo entra con su propio usuario. El stock se descuenta en el servidor, así que dos cajas no pueden vender la misma unidad.

¿Dónde están mis datos?

En el equipo donde se instaló Kubo, no en la nube de terceros.

¿Cuánto cuesta?

El software es tuyo (licencia MIT). Solo pagas el equipo donde corre.

Me equivoqué en una venta.

Pídele al administrador que la **anule**: el inventario vuelve y queda registrada

la anulación (no se borra la historia).

¿Puedo usar mi lector de código de barras?

Sí, si escribe el código como texto. Escribe o escanea en el buscador del POS.

Olvidé mi contraseña.

En la pantalla de ingreso pulsa **«¿Olvidaste tu contraseña?»** y escribe tu correo: recibirás un enlace para crear una nueva. El enlace vence en 30 minutos y solo sirve una vez. Si el correo del servidor aún no está configurado, pídele al administrador que la restablezca.

7. Buenas prácticas

1. **Registra el costo** de cada producto: es lo que permite saber tu margen.
2. **Haz un conteo físico** una vez al mes y ajusta el stock con el motivo *«Conteo físico»*: así el kardex cuenta la historia real.
3. **Usa las etapas de cliente**: es tu lista de trabajo comercial.
4. **Revisa el tablero cada mañana**: cinco segundos para saber cómo arrancó el día.
5. **No compartas el usuario administrador**: crea un usuario por vendedor.

07 — Pruebas

La verificación del sistema tiene cuatro niveles: pruebas unitarias dentro de cada servicio, una prueba de humo end-to-end, la suite móvil en el emulador y verificaciones manuales de operación.

1. Prueba de humo end-to-end (la principal)

```
make smoke
```

Ejecuta **192 comprobaciones** contra el sistema en ejecución, usando el API Gateway como un cliente real. Resultado esperado: `192 pruebas exitosas, 0 fallidas`.

| Bloque | Qué verifica | Comprobaciones | | --- | --- | --- | | 1. Salud | Los cinco servicios de aplicación responden `UP` | 7 | | 2. Autenticación y cookie | Login; **RS256**; negocio en el token; **JWKS**; el refresh **no viaja en el cuerpo**; cookie **httpOnly + SameSite=Strict**; rotación; el token refrescado autentica; **reuso → `401`**; **el reuso revoca la familia completa**; logout 204 y borra la cookie | 14 | | 3. Acceso, auditoría y bloqueo | Sin token → `401`; token falsificado → `401`; cabecera inyectada → **ignorada**; negocio malformado → `400`; **5 intentos fallidos bloquean la cuenta**; **cadena de auditoría intacta** y hash versionado; usuarios y roles; **segundo factor TOTP completo** (secreto, URI, activación, desafío, código válido e inválido); deshabilitado no ingresa; eventos de seguridad en la bitácora | 23 | | 4. Clientes, cifrado y RLS | Crear cliente; detalle completo; listado **enmascarado**; texto **cifrado**; índice ciego; **búsqueda por documento**; **RLS: sin contexto 0 filas, con contexto > 0** | 8 | | 5. Inventario, venta y outbox | Bodega por defecto y segunda bodega; **transferencia con kardex en ambas**; cupo de bodegas por plan; venta desde la bodega elegida; aviso de stock bajo; venta por `35700.00`; IVA `5700.00`; kardex; **importación CSV**; **paginación con total real**; sobreventa → `409`; **evento en la bandeja** y publicado; **RLS en ERP** | 30 | | 6. Evento, tablero y verticales | Proyección en analítica y MongoDB; tablero; **vista compuesta con las 7 vistas**; zona horaria del negocio; **cuatro verticales** y catálogo de arranque idempotente; servicio sin kardex y venta con mesa; reportes CSV; **factura DIAN con CUFE y XML**; comprobante PDF con hash; anti-inyección de fórmulas; outbox sin fallidos | 42 | | 7. Anulación y nota crédito | La venta queda `VOIDED`; el inventario vuelve; una venta anulada no se factura; **nota crédito con CUDE, referencia y XML UBL**; reemisión idempotente | 10 | | 8. Aislamiento, plataforma y cobro | Un segundo negocio no ve clientes ni catálogo; **RLS en IAM**; suspensión y reactivación; renovación del plan; **panel de plataforma con TOTP obligatorio**; uso por negocio; planes y cupos; intención de pago al precio del catálogo; **webhook firmado e idempotente** | 26 | | 9. Recuperación de contraseña | Solicitud 204; enlace en el buzón; consumo 204; **la cuenta se desbloquea con la clave nueva**; el enlace **no se reutiliza**; sin enumeración de usuarios | 6 | | 10. Compras, proveedores y caja | Proveedor creado; compra por `25000.00`; IVA desagregado; **entra a la bodega elegida**; bodega inexistente → `404`; soporte documental; **suma 5 unidades**; **el costo se actualiza sin IVA**; kardex `PURCHASE`; **la anulación revierte en la misma bodega**; **RLS en compras**; apertura, venta en efectivo, cierre y arqueo de caja | 20 | | 11. Tasa, TLS y trazas | **Límite por usuario** activo; HTTPS 200; HTTP → HTTPS; HSTS; **collector de trazas arriba y recibiendo spans** | 6 |

Además del humo: **23 contratos OpenAPI** (`make contracts`), **5 pruebas de

navegador con axe** (`make e2e`), la **carga a 50 cajas** (`make load`: 100 % de ventas exitosas; p95

107 ms con 10 cajas y ~540 ms en el host de desarrollo con presión de memoria), los simulacros de bus (`make bus-drill`, 4/4) y restauración

(`make restore-drill`, 14/14), la **suite móvil** (4 flujos en el emulador Android), los **enlaces de la documentación** (102 resueltos, sin rotos) y el **PDF consolidado al día** (huella de las fuentes, sin abrir Chrome), y el gate `make ci` (13 verificaciones) que los reúne.

La prueba es **idempotente**: crea sus propios datos con marcas de tiempo y puede ejecutarse tantas veces como haga falta.

2. Pruebas unitarias por servicio

```
# kubo-iam — 83 pruebas (tokens, identidad, TOTP, plataforma, cobro y datos fiscales)
# Incluye Testcontainers: PostgreSQL real, migraciones Flyway y RLS.
# Gate de cobertura JaCoCo ≥ 80 % en application/domain (hoy 85.2 %).
cd kubo-iam && mvn verify

# kubo-gateway — 19 pruebas (rutas, middleware de acceso, límite de tasa con Redis real,
# Gate de Node ≥ 80 % en líneas y funciones (hoy 95.6 / 85.2).
cd kubo-gateway && npm test

# kubo-crm — 34 pruebas (13 de cifrado + 21 de integración con PostgreSQL)
# Gate SimpleCov ≥ 80 % de líneas (hoy 100 %).
./kubo-infra/scripts/crm-tests.sh

# kubo-erp — 57 pruebas ExUnit (35 puras + 22 de integración/HTTP)
# Ratchet de ExUnit ≥ 35 % (hoy 37.68 %); la capa web la cubre el humo.
./kubo-infra/scripts/erp-tests.sh

# kubo-analytics — 8 pruebas (6 unitarias + 2 de integración con MongoDB)
./kubo-infra/scripts/analytics-tests.sh

# kubo-web — 45 pruebas vitest (cola offline, politica de errores, formato,
# sistema de diseno —con focus trap y Escape del dialogo—, pantalla de ingreso
# y modo nativo —servidor configurable y HTTP nativo—)
# Gate v8: líneas/funciones/statements ≥ 85 y ramas ≥ 65.
cd kubo-web && npm test
```

Total: 246 pruebas de servicio (unitarias y de integración) más 192 comprobaciones end-to-end, 23 contratos, 5 pruebas de navegador con axe y los simulacros de bus y restauración. La cobertura se vigila en CI con gates: IAM ≥ 80 % (JaCoCo, hoy 85.2 %), analítica ≥ 80 % en su módulo de procesamiento, gateway ≥ 80 % en líneas y funciones (hoy 95.6 / 85.2), CRM ≥ 80 % de líneas (hoy 100 %), web ≥ 85 % de líneas/funciones/statements (hoy 97.3 / 95.3 / 95.9) y ERP con ratchet ≥ 35 % (hoy 37.68 %; la capa web la cubre el humo).

Cómo ejecutar cada suite

```
make smoke                                  # 192 comprobaciones end-to-end
make bus-drill                              # 4 comprobaciones: caida del bus
make restore-drill                          # 14 comprobaciones: restauracion
make contracts                              # 23 contratos OpenAPI
make e2e                                    # 5 pruebas de navegador + axe
make load                                   # 50 cajas, p95 < 300 ms
make ci                                     # gate completo (13 verificaciones)
make pdf                                    # regenera el PDF consolidado y su huella
./kubo-web/e2e-android/run-sesion.sh        # suite móvil: 4 flujos en el emulador (Mae

./kubo-infra/scripts/crm-tests.sh           # 34 pruebas del CRM
./kubo-infra/scripts/erp-tests.sh           # 57 pruebas del ERP
./kubo-infra/scripts/analytics-tests.sh     # 8 pruebas de analitica
cd kubo-web && npm test                     # 45 pruebas vitest de la PWA
```

Las pruebas del ERP que no tocan base de datos son puras (aritmética decimal,

outbox, packs, facturación, documentos) y **no pueden ejecutarse dentro del

contenedor de producción**: al compilar el entorno de pruebas el contenedor se

queda sin memoria (límite de 512 MB), y además la imagen de ejecución no incluye

`test/`. Se ejecutan con más memoria y el directorio de pruebas montado

(`erp-tests.sh`, integrado en `make ci`).

Qué cubren las pruebas unitarias

| Servicio | Casos | | --- | --- | | IAM | El token se firma con RS256 y tiene 3 segmentos; el JWKS **no** expone la llave privada; los refresh tokens son aleatorios y su hash es SHA-256 hex | | Gateway | La tabla cubre los cuatro microservicios; los prefijos anidados resuelven al servicio correcto; una ruta protegida no se confunde con una pública | | CRM | Ida y vuelta del cifrado; dos cifrados del mismo valor difieren (IV aleatorio); **GCM detecta manipulación**; el índice ciego es determinista y normalizado; enmascarado correcto; valores vacíos no se cifran | | ERP | IVA desagregado de un precio con impuesto incluido; suma de varias líneas; producto exento; el total siempre es subtotal + impuesto | | Analítica | Conversión de importes y cantidades desde el evento; tolerancia a entradas inválidas; normalización del detalle; fechas a UTC |

3. Suite móvil (emulador Android)

```
./kubo-web/e2e-android/run-sesion.sh    # requiere un emulador o un teléfono conectado po
```

Cuatro flujos de **Maestro** (`kubo-web/.maestro/`) sobre el APK real contra el

servidor de demostración: sesión (primer arranque, conexión y **restauración por cookie**), **venta de POS**, **venta sin conexión** (la red se corta y se verifica con `ping` desde el propio emulador; la venta queda en la cola local) y **sincronización automática al reconectar** (la cola reintenta cada 30 s).

Corren en el flujo `Android` de GitHub Actions (emulador API 34 + Maestro; el APK queda como artefacto de cada push). Resultado vigente: **4/4 en verde** (2026-10-07). La instalación del APK en un equipo físico es la verificación manual opcional (ver [`14-plan-app-movil.md`](14-plan-app-movil.md)).

4. Verificación manual de la interfaz

| # | Paso | Resultado esperado | | --- | --- | --- | | 1 | Abrir `http://localhost:3000` | Pantalla de ingreso con la marca Kubo | | 2 | Ingresar con `admin@kubo.local` / `Admin123!` | Tablero con indicadores | | 3 | Instalar la aplicación desde el navegador | Se abre a pantalla completa, sin barra del navegador | | 4 | Registrar una venta en el POS | Aparece el número de venta y el tablero se actualiza | | 5 | Activar el modo avión y vender | Etiqueta «Sin internet» y la venta queda en cola | | 6 | Recuperar la conexión | La venta se sincroniza y el contador desaparece | | 7 | Abrir un cliente | Documento completo; en el listado, enmascarado | | 8 | Registrar una entrada de inventario | El stock sube y el movimiento queda en el kardex | | 9 | Intentar vender más unidades que el stock | Mensaje «Stock insuficiente» con la cantidad disponible | | 10 | En el ingreso, «¿Olvidaste tu contraseña?» → pedir el enlace | El correo queda en el buzón de demostración (`mail_outbox`) y el enlace de `/recuperar?token=…` permite elegir una contraseña nueva |

5. Verificación de seguridad

```
# Sin token
curl -o /dev/null -w '%{http_code}\n' http://localhost:9080/api/v1/customers        # 401

# Suplantación por cabecera
curl -o /dev/null -w '%{http_code}\n' http://localhost:9080/api/v1/customers \
  -H 'X-User-Id: 00000000-0000-0000-0000-000000000000'                              # 401

# El texto cifrado no contiene el documento
docker exec kubo-postgres psql -U kubo_root -d kubo_crm -c \
  "select document_number_encrypted from customers limit 1"

# La llave privada no se publica
curl -s http://localhost:9080/api/v1/auth/.well-known/jwks.json | grep -c '"d"'    # 0

# Límite de tasa en autenticación
for i in $(seq 1 50); do
  curl -s -o /dev/null -w '%{http_code} ' -X POST http://localhost:9080/api/v1/auth/login
    -H 'Content-Type: application/json' -d '{"email":"x@x.co","password":"malo"}'
done; echo                                                                          # inc
```

6. Rendimiento observado

Medido en un equipo de 16 núcleos, a través del gateway (la latencia que percibe

el usuario):

| Operación | Tiempo | | --- | --- | | Login (BCrypt + firma RSA) | ~75 ms | | Registrar venta (transacción + kardex) | ~45 ms | | Consultar catálogo (300 productos) | ~6 ms | | **Tablero completo (vista compuesta BFF)** | **~25 ms** (antes 108 ms en 7 viajes) | | Búsqueda por texto con 50.000 registros (ERP) | ~5.7 ms (antes 46 ms) | | Búsqueda por texto con 50.000 registros (CRM) | ~4.4 ms (antes 28 ms) | | Propagación del evento a MongoDB | < 1 s | | Arranque completo del sistema | ~50 s | | Consumo en reposo | ~930 MB |

El detalle de las mediciones, los planes de ejecución y su análisis están en

[`10-auditoria.md`](10-auditoria.md).

7. Cobertura y pendientes vivos

Los pendientes de las fases 1 y 2 (P-06, P-08, P-09, P-10 y P-26) quedaron

cerrados: el CI por repositorio, las pruebas de integración contra el motor

real en los cinco servicios, los contratos ejecutables, la carga a 50 cajas y la

accesibilidad automatizada corren en `make ci`.

Lo que sigue abierto, sin bloquear el cierre:

| Tema | Estado | | --- | --- | | Gates de cobertura | IAM (85.2 %, JaCoCo), analítica (≥ 80 %), gateway (95.6 %), CRM (100 %), web (97.3 % de líneas) y ERP (ratchet 37.68 %; la capa web la cubre el humo 192/192) | | Pruebas unitarias de la PWA | Vitest: cola offline, política de errores, formato, sistema de diseño —incluido el focus trap y Escape del diálogo—, pantalla de ingreso y modo nativo —servidor configurable y HTTP nativo— (45 pruebas); los flujos completos se cubren con E2E y axe | | Proveedor tecnológico DIAN | **Proyecto listo para enchufarlo**: puerto configurable (`KUBO_BILLING_ADAPTER`), datos fiscales en el token, errores tipados y guía [`13`](13-guia-adaptador-facturacion.md); falta el adaptador del PT elegido (externo) | | Contratos del lado del consumidor (Pact) | **Implementado**: 4 interacciones del consumidor (PWA) generan el pact y `make pact` lo verifica contra el sistema vivo; integrado en `make ci` | | Carga con datos voluminosos | **Implementado**: `make load-big` siembra 50.000 productos y mide la busqueda (p95 233 ms a 10 cajas; 6 ms por peticion en frio) |

08 — Matriz de trazabilidad

Cada requisito del enunciado, cómo se implementó y **dónde se comprueba**. Si un

requisito no tiene evidencia verificable, aparece marcado como pendiente.

1. Requisitos del proyecto

| # | Requisito | Implementación | Evidencia | Estado | | --- | --- | --- | --- | --- | | R1 | ERP + CRM para PYMES sin acceso a tecnología, sin pagar SaaS | Instalación de un comando en el local, licencia MIT, sin dependencias de nube | `README.md` · `05-despliegue.md` · `make up` | Cumplido | | R2 | Mínimo 4 microservicios | 4 servicios desplegados (IAM, CRM, ERP y Analítica) más el gateway; notificaciones y documentos viven como **puertos** dentro del ERP (ADR-0017 y ADR-0018) | `docker compose ps` (12 contenedores) · `01-arquitectura.md` | Cumplido | | R3 | Mínimo 3 tecnologías distintas | Java, Ruby, Elixir, Python, TypeScript (5) | `01-arquitectura.md` · ADR-0004 | Cumplido | | R4 | Front dinámico y actual, nivel senior | React 19 + Vite 8 + Tailwind 4, design system con tokens, TanStack Query, gráficas, accesibilidad | `kubo-web/` · `kubo-web/src/components/ui.tsx` | Cumplido | | R5 | Conexión con PostgreSQL y MongoDB | PostgreSQL 17 (IAM, CRM, ERP) + MongoDB 8 (analítica) | `docker compose ps` · `02-modelo-datos.md` | Cumplido | | R6 | Interfaz UI/UX | 13 rutas (POS, inventario, compras, caja, bodegas, usuarios, configuración, notificaciones, panel de plataforma), estados de carga/vacío/error, toasts, modales accesibles, español, modo offline visible | `06-manual-usuario.md` · verificación manual en `07-pruebas.md` §3 | Cumplido | | R7 | API Gateway | `kubo-gateway` (NestJS): validación JWT, límites de tasa, correlación, enrutamiento, BFF | `kubo-gateway/README.md` · `make smoke` bloque 3 | Cumplido | | R8 | Cada microservicio con su base de datos | `database-per-service`: `kubo_iam`, `kubo_crm`, `kubo_erp` con roles aislados; `kubo_analytics` en MongoDB | `kubo-infra/scripts/init-db.sh` · ADR-0003 | Cumplido | | R9 | Repositorio separado por parte del software | 9 repositorios: 5 componentes + infra + docs + web + raíz | `ls -d kubo-*/` · ADR-0002 | Cumplido | | R10 |

Documentación detallada del funcionamiento | 15 documentos + 31 ADRs + README por servicio + OpenAPI ejecutable + guion de video | `kubo-docs/` · `Kubo-Documentacion.pdf` | Cumplido; capturas y recorrido base en `kubo-docs/evidencia/`; video narrado de demostración pendiente (tarea del usuario) | | R11 | Encriptado de datos | AES-256-GCM campo a campo + índice ciego HMAC + BCrypt + SHA-256 de refresh tokens + auditoría con cadena de hash versionada + RLS en las tres bases + outbox transaccional | `make smoke` bloques 3, 4 y 5 · `04-seguridad.md` | Cumplido | | R12 | PWA | Service worker, manifest, instalable, cola offline en IndexedDB, sincronización automática | `curl localhost:3000/manifest.webmanifest` · `kubo-web/vite.config.ts` | Cumplido | | R13 | JWT | RS256 con JWKS, access 15 min, refresh rotativo con detección de reuso y **cookie `httpOnly`** (BFF); segundo factor TOTP con desafío `typ=totp` | `make smoke` bloques 2 y 3 · ADR-0007 · ADR-0015 | Cumplido | | R14 | Arquitectura idónea al tipo de proyecto | Microservicios con monolito modular, polyrepo, eventos, multi-tenancy híbrido | `01-arquitectura.md` · ADR-0001 a 0008 | Cumplido | | R15 | Diseño y seguridad del proyecto | Modelo de confianza, STRIDE por servicio, OWASP, Ley 1581, límites de tasa, anti-suplantación | `04-seguridad.md` · `make smoke` bloque 3 y 8 | Cumplido |

2. Requisitos funcionales del dominio

| # | Requisito | Implementación | Evidencia | | --- | --- | --- | --- | | RF-01 | Autenticar usuarios y aislar negocios | `kubo-iam` + `tenant_id` en el token | `make smoke` bloques 2 y 8 | | RF-02 | Gestionar clientes con datos personales protegidos | `kubo-crm` con cifrado de campo | `make smoke` bloque 4 | | RF-03 | Gestionar catálogo de productos | `kubo-erp` CRUD + borrado lógico | `make smoke` bloque 5 | | RF-04 | Controlar inventario con trazabilidad | Kardex `stock_movements` + proyección de stock | `make smoke` bloque 5 y 7 | | RF-05 | Registrar ventas con descuento de inventario | Transacción con bloqueo pesimista | `make smoke` bloque 5 | | RF-06 | Anular ventas devolviendo el inventario | `POST /sales/:id/void` | `make smoke` bloque 7 | | RF-07 | Calcular impuestos (IVA) | Desagregación con `Decimal` en el ERP | `make smoke` bloque 5 | | RF-08 | Tablero de indicadores del negocio | `kubo-analytics` sobre MongoDB | `make smoke` bloque 6 | | RF-09 | Operar sin conexión | Cola IndexedDB + sincronización | Verificación manual `07-pruebas.md` §3 | | RF-10 | Auditar las acciones sensibles | `audit_logs` con cadena de hash | `GET /api/v1/audit` | | RF-11 | Gestionar proveedores y compras (Fase 3) | `kubo-erp`: compra suma inventario, kardex `PURCHASE` y costo sin IVA | `make smoke` bloque 10 · `make contracts` |

3. Requisitos no funcionales

| # | Atributo | Objetivo | Cómo se sostiene | Evidencia | | --- | --- | --- | --- | --- | | RNF-01 | Rendimiento | Venta < 100 ms | Transacción única + Elixir | `07-pruebas.md` §5 | | RNF-02 | Disponibilidad | El bus caído no bloquea la caja | Publicación asíncrona tolerante a fallos | ADR-0005 | | RNF-03 | Escalabilidad | Escalar la caja sin reescribir | Servicios sin estado | `05-despliegue.md` §7 | | RNF-04 | Seguridad | Sin datos personales legibles en la base | AES-256-GCM + índice ciego | `make smoke` bloque 4 | | RNF-05 | Mantenibilidad | Un lenguaje por contexto con arquitectura limpia | ADR-0004 · README por servicio | | RNF-06 | Portabilidad | Mismo artefacto en local y nube | Docker Compose | `05-despliegue.md` | | RNF-07 | Consumo | < 2 GB de RAM en reposo | Límites por contenedor |

`docker stats --no-stream` | | RNF-08 | Usabilidad | Dueño sin conocimientos técnicos | Manual + modo simple + español | `06-manual-usuario.md` | | RNF-09 | Observabilidad | Estado de cada servicio y su dependencia | Sondas `/health` | `make smoke` bloque 1 | | RNF-10 | Trazabilidad | Reconstruir una operación completa | `X-Correlation-Id` propagado | Logs del gateway |

4. Decisiones de arquitectura

| ADR | Decisión | Estado | | --- | --- | --- | | 0001 | Microservicios con monolito modular interno | Aceptada | | 0002 | Polyrepo con contratos como fuente de verdad | Aceptada | | 0003 | Multi-tenancy híbrido (`tenant_id` + RLS activo) | Aceptada | | 0004 | Poliglotismo deliberado: un lenguaje por contexto | Aceptada | | 0005 | Eventos con RabbitMQ y publicador tolerante a fallos | Aceptada (outbox diferido) | | 0006 | Cifrado de campos personales con AES-256-GCM | Aceptada | | 0007 | JWT RS256 con JWKS en el gateway | Aceptada | | 0008 | Alcance declarado del MVP y recortes conscientes | Aceptada | | 0009 | Outbox transaccional para la entrega de eventos | Aceptada (Fase 1) | | 0010 | RLS activo con interceptor de transacción | Aceptada (Fase 1) | | 0011 | BFF como capa formal del gateway | Aceptada (Fase 2) | | 0012 | Zona horaria por negocio | Aceptada (Fase 4) | | 0013 | Vertical Packs: el vertical como dato del negocio | Aceptada (Fase 4) | | 0014 | Facturación electrónica DIAN como puerto enchufable | Aceptada; habilitación externa pendiente | | 0015 | Segundo factor TOTP del propietario | Aceptada (Fase 4) | | 0016 | Stock por bodega y transferencias | Aceptada (Fase 4) | | 0017 | Puerto de notificaciones y buzón del negocio | Aceptada (Fase 4) | | 0018 | Documentos en el ERP con puerto de almacenamiento | Aceptada (Fase 4) | | 0019 | Rotación de claves de cifrado de campo (KEK/DEK) | Aceptada (Fase 5) | | 0020 | mTLS en la malla interna | Aceptada (Fase 5) | | 0021 | Multi-tenant SaaS como modo de despliegue | Aceptada (Fase 5) | | 0022 | Operador de respaldos con verificación de restauración | Aceptada (Fase 5) | | 0023 | App móvil: la PWA primero, Capacitor cuando el negocio lo pida | Aceptada; empaquetado ejecutado en la fase 1 del plan móvil (ADR-0031) | | 0024 | Superficie del operador: script hoy, panel cuando haya rol | Aceptada (Fase 5) | | 0025 | Rol de plataforma y panel del operador | Aceptada (Fase 6) | | 0026 | Puerto de cobro y pasarela de pago | Aceptada (Fase 6) |

5. Requisitos no cubiertos (declarados)

> Los recortes de las fases 1 a 6 quedaron implementados y verificados (ver

> [`11-plan-de-cierre.md`](11-plan-de-cierre.md)). Lo que sigue fuera del

> alcance actual:

| Requisito | Motivo | Ruta | | --- | --- | --- | | Habilitación DIAN y firma XAdES | Trámite externo como Proveedor Tecnológico; el puerto UBL 2.1, el CUFE y la nota crédito ya están implementados y probados | Operación (ADR-0014) | | Capacidades nativas (cámara, push, impresión térmica) | El empaquetado base está hecho y verificado (fase 1 del plan móvil: APK, assets locales y suite en el emulador); las capacidades nativas se activan cuando el negocio las pida | ADR-0023 · ADR-0031 · [`14`](14-plan-app-movil.md) | | Kubernetes | El despliegue objetivo es un local con Docker Compose;

k3s es el siguiente nivel (> 50 negocios) | Fase 5 (backlog declarado) | | Importación desde Dolibarr | La importación CSV (P-24) está implementada; la ruta específica de Dolibarr es una mejora de adopción | Backlog comercial | | Capturas y video de demostración | Requieren grabación manual del usuario | `evidencia/README.md` |

> Ningún requisito se declara cumplido sin evidencia verificable. Los recortes

> están documentados en el ADR-0008 y en esta matriz.

09 — Guion de demostración (video)

**Duración objetivo: 6 minutos 25 segundos de base visual + 3 minutos de

sustentación técnica (opcional).** La base visual

[`evidencia/demo-kubo.webm`](evidencia/demo-kubo.webm) (6:25) recorre el sistema

con un subtítulo por bloque: narra sobre ella, o graba tu propia pantalla

siguiendo los mismos bloques y tiempos.

| Campo | Valor | | --- | --- | | Versión del guion | 2.0 — aligned con la fase de cierre 0–6 y la Fase 1 móvil | | Base visual | `evidencia/demo-kubo.webm` (6:25, 15 secciones) | | Estado del producto | v0.3.0 · fases 0–6 completadas · 31 ADR · app Android en Fase 1 completada | | Verificación vigente | humo **192/192** (reverificado en vivo) · `make ci` 13/13 · 246 pruebas de servicio · contratos 23/23 · E2E 5/5 · bus 4/4 · restauración 14/14 · móvil 4/4 | | Duración total si narras todo | ≈ 9–10 minutos |

> **Qué cambió respecto de la versión anterior del guion.** (1) La Fase 1 de la

> app móvil **ya está completada** y verificada en emulador: el bloque 15 decía

> "primera fase completada" y hoy se puede mostrar el APK funcionando. (2) Se

> incorpora la **sustentación técnica** con los dos diagramas nuevos —mapa de

> arquitectura y stack, y la secuencia animada del outbox— que son el respaldo

> visual del diseño. (3) Se narran las capacidades operativas que ya existen y

> no se narraban: mTLS en la malla, trazas OpenTelemetry, operador de respaldos

> con simulacro de restauración, CI/CD, carga a 50 cajas, sincronización de 4

> entornos y demo pública 100 % open source. (4) Los conteos de pruebas se

> actualizaron a los vigentes.

Preparación (antes de grabar)

```
cd "Documents/Proyectos de Programación/Kubo"
make foreign-stop        # libera RAM (detiene Dolibarr, Guacamole, Open Notebook)
make up                  # levanta los 12 contenedores
make smoke               # verifica 192 comprobaciones end-to-end
make reset-demo          # deja la demostración limpia y presentable (borra lo que deja e
```

Deja abiertas estas ventanas:

1. Terminal en la carpeta del proyecto.

2. Navegador en `http://localhost:3000` (sin sesión iniciada).

3. Terminal secundaria para consultar PostgreSQL.

4. Explorador de archivos con la carpeta `Kubo/`.

5. (Opcional) Teléfono con el APK de Kubo instalado, si quieres mostrar la app

Android (ver `kubo-docs/14-plan-app-movil.md`).

6. (Opcional) Un segundo monitor o pestaña con

`kubo-docs/diagramas/arquitectura-stack.png` y

`kubo-docs/diagramas/flujo-venta-outbox.webm` para la sustentación técnica.

> **Antes de grabar, confirma el estado real** (esto también te da material para

> preguntas del jurado):

>

> ```bash

> make ps # los 12 contenedores en healthy

> make ci # gate completo: 13/13

> make observability # opcional: levanta Tempo + Grafana (Grafana en :3001)

> make sync-status # el estado de los 4 entornos (local, GitHub, Notion, Obsidian)

> ```

Bloque 1 · Ingreso (0:00 – 0:27)

En pantalla: la pantalla de ingreso con la marca y sus beneficios.

> "Hola, soy [nombre], estudiante del Tecnólogo en Análisis y Desarrollo de
> Software. Este es **Kubo**, un ERP + CRM para pequeñas y medianas empresas,
> que se instala en el propio local —en un computador pequeño o un servidor
> económico— y funciona incluso sin internet. Un solo comando levanta todo el
> sistema. Ingreso con la cuenta del administrador: la contraseña se guarda con
> BCrypt y la sesión se firma con RSA."

Bloque 2 · Tablero (0:27 – 0:53)

En pantalla: el tablero con KPIs, la serie de 14 días, los productos más vendidos y los medios de pago.

> "Este es el tablero: todos los indicadores vienen de MongoDB, construidos a
> partir de los eventos de venta que publica el ERP —ventas de hoy, ingreso
> acumulado, ticket promedio, la serie de los últimos catorce días, los
> productos más vendidos y los medios de pago—. El navegador solo habla con el
> API Gateway, que valida el token JWT y enruta cada petición a los
> microservicios. Hace un momento no había nada aquí: los datos llegaron solos."

Bloque 3 · Clientes (0:53 – 1:18)

En pantalla: el listado de clientes con documento y teléfono enmascarados.

> "En Clientes, el documento y el teléfono aparecen enmascarados en el listado:
> solo se muestran los últimos dígitos. El CRM está escrito en Ruby on Rails y
> los datos personales están protegidos según la Ley 1581 de habeas data; cada
> consulta queda auditada."

Bloque 4 · Detalle del cliente (1:18 – 1:43)

En pantalla: el detalle de un cliente con documento y teléfono completos.

> "Al abrir un cliente, el detalle sí trae el documento completo. En la base de
> datos no se puede leer: está cifrado con AES-256-GCM, que además detecta
> cualquier manipulación. La columna de al lado es un índice ciego con HMAC:
> permite buscar por documento exacto sin descifrarlo."

Bloque 5 · Catálogo (1:43 – 2:08)

En pantalla: el catálogo con precio, IVA, stock y alerta de stock bajo.

> "En Productos está el catálogo, con precio, IVA, stock y alerta de stock
> bajo. El ERP está escrito en Elixir y cada movimiento queda en el kardex con
> fecha, usuario y motivo. El vertical del negocio cambia la terminología: una
> tienda ve Productos y un restaurante, Platillos."

Bloque 6 · Compras (2:08 – 2:34)

En pantalla: proveedores, registro de compras y últimas compras.

> "En Compras se registran los proveedores y las entradas de mercancía: al
> recibir una compra, el inventario sube y el costo del producto se actualiza
> sin IVA. Si algo llega mal, la anulación revierte todo en la misma bodega."

Bloque 7 · Caja (2:34 – 2:59)

En pantalla: la caja del turno con apertura, ventas en efectivo y arqueo.

> "En Caja se abre el turno, se registran las ventas en efectivo y al cerrar se

> hace el arqueo: el sistema muestra la diferencia entre lo contado y lo

> esperado, y todo queda en la auditoría."

Bloque 8 · Bodegas (2:59 – 3:24)

En pantalla: las bodegas del negocio y una transferencia entre locales.

> "En Bodegas se mueve mercancía entre locales con una transferencia: el stock

> baja en una y sube en la otra, con su movimiento de kardex en ambas; el total

> del negocio no cambia."

Bloque 9 · Punto de venta (3:24 – 3:49)

En pantalla: el POS con un producto en el carrito, IVA desagregado y total.

> "Este es el punto de venta. Busco un producto, lo agrego al carrito y el

> total aparece con el IVA desagregado. Al cobrar, la venta se registra en una

> sola transacción: el inventario se descuenta y el evento ya viajó a

> analítica."

Bloque 10 · Sin conexión (3:49 – 4:15)

En pantalla: «Sin internet» y la venta guardada en la cola local.

> "Si se cae el internet, la caja no se detiene: la venta queda en una cola

> local del navegador y la barra avisa que hay ventas esperando sincronización.

> Nunca se pierde una venta."

Bloque 11 · Sincronización (4:15 – 4:40)

En pantalla: «En línea» y la cola vaciándose sola.

> "Al volver la conexión, Kubo envía las ventas pendientes sola, en orden, y el

> tablero se actualiza. Los errores de negocio se descartan para no bloquear la

> cola; los de red se reintentan."

Bloque 12 · Documentos (4:40 – 5:05)

*En pantalla: facturas electrónicas, nota crédito y comprobantes PDF con

descarga autenticada.*

> "En Documentos quedan las facturas electrónicas con su CUFE, las notas

> crédito, los comprobantes PDF y los soportes, todos con descarga

> autenticada. El adaptador de la DIAN es un puerto enchufable: se conecta un

> proveedor tecnológico sin tocar el núcleo."

Bloque 13 · Notificaciones (5:05 – 5:30)

En pantalla: el buzón del negocio con avisos de stock bajo, compras y resumen.

> "El buzón del negocio avisa lo que hay que saber sin mirar el tablero: stock

> bajo, compras recibidas y el resumen del día. Las notificaciones salen por un

> puerto que admite correo o WhatsApp."

Bloque 14 · Usuarios y roles (5:30 – 5:56)

En pantalla: la gestión de usuarios y roles del negocio.

> "En Usuarios, el propietario crea, edita y deshabilita cuentas y asigna

> roles —vendedor o propietario—; los permisos se aplican en todos los

> servicios. El segundo factor con aplicación autenticadora protege las

> acciones sensibles."

Bloque 15 · Configuración y cierre (5:56 – 6:25)

En pantalla: vertical, zona horaria, datos fiscales, plan y segundo factor.

> "En Configuración se define el vertical, la zona horaria, los datos fiscales

> para la DIAN, el uso contra el plan y el segundo factor. La documentación

> incluye arquitectura con diagramas C4, modelo de datos, seguridad, despliegue,

> trazabilidad, treinta y una decisiones registradas como ADR y el plan de la

> app móvil. Kubo corre en cerca de un gigabyte de memoria repartido en doce

> contenedores, se instala con un comando y está pensado para que un negocio de

> barrio digitalice sus ventas sin pagar suscripciones. Gracias."

Sustentación técnica (opcional, ~3 minutos)

> **Cuándo usarla.** Es la parte que responde "¿cómo está construido y por

> qué?" ante un jurado técnico. No está en la base visual: grábala con tu

> pantalla o muéstrala en vivo. Si el tiempo es corto, el bloque 15 + el bloque

> extra de arquitectura bastan.

S1 · El mapa de arquitectura y el stack (≈1:20)

*Muestra `kubo-docs/diagramas/arquitectura-stack.png` (o el interactivo

`kubo-docs/diagramas/html/arquitectura-stack.html`, que trae vistas guiadas

por capa).*

> "Este es el mapa completo del sistema. Arriba, el cliente: una sola PWA en

> React 19 con Vite y Tailwind, que además se empaqueta como app Android con

> Capacitor —el mismo código, un solo binario por versión—. Sigue el borde:

> Caddy termina el HTTPS con HSTS y nginx sirve los activos y hace de proxy de

> `/api` hacia el gateway, de modo que todo es un solo origen y no hay CORS.

>

> En el centro, la puerta de entrada: `kubo-gateway` en TypeScript con NestJS.
> Es el único punto de entrada y aplica tres cosas: valida el JWT contra el
> JWKS del servicio de identidad, aplica límites de tasa con Redis y enruta.
> Detrás están los cinco servicios de aplicación, y deliberadamente cada uno
> está en un lenguaje distinto: **Java con Spring Boot** para identidad y
> auditoría, **Ruby con Rails** para clientes, **Elixir con Phoenix** para
> catálogo, inventario y ventas, y **Python con FastAPI** para analítica. No es
> arbitrario: cada servicio usa su ecosistema natural, y el dominio de ninguno
> conoce HTTP, la base de datos ni el bus —lo decide el ADR 0004—.
>
> Abajo, los cuatro almacenes: una instancia de PostgreSQL 17 con tres bases
> separadas —identidad, clientes y ERP—, MongoDB 8 para el modelo de lectura,
> Redis 7 para los límites y RabbitMQ 4 como bus de eventos. Y a la derecha, la
> plataforma: Docker Compose con doce contenedores, el collector de
> OpenTelemetry, Tempo y Grafana, el CI de GitHub Actions, las pruebas de carga
> con k6 y el operador de respaldos."

Si te preguntan por el costo: "En reposo el conjunto consume cerca de un gigabyte: cabe en un VPS de seis a doce dólares al mes o en el mini-PC del propio local, y no depende de ninguna nube."

S2 · La venta y su outbox, en movimiento (≈1:00)

Muestra `kubo-docs/diagramas/flujo-venta-outbox.webm` (6 s en bucle) o el estático `flujo-venta-outbox.png`.

> "Este diagrama es la respuesta al punto más difícil del diseño: cómo se
> registra una venta sin que la caja espere al tablero. La caja envía la venta
> al gateway, el gateway verifica la firma contra el JWKS de identidad y
> reenvía al ERP. Ahí empieza la parte importante: el ERP abre una transacción

> y escribe tres cosas —la venta, sus líneas y el movimiento de inventario— y

> en la *misma* transacción inserta el evento en una bandeja de salida, el

> *outbox*.

>

> Cuando la transacción confirma, el evento y la venta ya están garantizados

> juntos: es imposible vender sin registrar el evento. Después, y solo después,

> un publicador entrega el evento a RabbitMQ, la analítica lo consume y proyecta

> la venta en MongoDB. La caja recibió su respuesta 201 mucho antes de que

> eso ocurra.

>

> Esto es lo que hace que el sistema sea tolerante a fallos: si el bus se cae,

> el evento espera en la bandeja y se entrega cuando vuelve. La venta ya está

> confirmada y el negocio sigue vendiendo. Hay un simulacro automatizado que

> lo comprueba, `make bus-drill`, y da cuatro de cuatro."

Si te preguntan por el modo sin conexión: "El mismo patrón resuelve el

internet caído: la venta se guarda en una cola local del navegador y se

reintenta sola al volver la conexión."

S3 · Qué respalda cada afirmación (≈0:40)

Muestra la terminal con `make smoke` en verde.

> "Nada de esto es una captura de pantalla bonita: cada afirmación tiene una

> prueba. La prueba de humo ejecuta ciento noventa y dos comprobaciones

> contra el sistema en ejecución, usando el API Gateway como un cliente real.

> A eso se suman las doscientos cuarenta y seis pruebas unitarias y de

> integración de los cinco servicios, veintitrés contratos OpenAPI validados

> contra la API viva, cinco pruebas de navegador con auditoría de

> accesibilidad, la carga a cincuenta cajas —ciento por ciento de ventas

> exitosas— y los simulacros de caída del bus y de restauración. Todo eso

> converge en un único gate, `make ci`, que es trece de trece."

Bloques extra (si narras con tu propia pantalla)

Estos bloques no están en la base visual; sirven si grabas tu propia pantalla y

quieres profundizar (agregan ~2 minutos).

La app Android (Fase 1 completada)

Muestra el teléfono con el APK, o el emulador con la suite de Maestro.

> "Esta es la misma PWA empaquetada con Capacitor. No reescribimos nada:

> primero se probó la PWA y solo cuando el negocio lo pidió se envolvió. La

> Fase 1 ya está completada y verificada: el APK se construye en el CI, la

> sesión funciona sobre la cookie httpOnly, y la suite móvil cubre el ingreso,

> la venta en el punto de venta, la cola sin conexión y la sincronización —

> cuatro de cuatro en verde en el emulador."

Operación, escala y continuidad

*Muestra `make ps`, `docker stats --no-stream` y, si levantaste el perfil, el

panel de Grafana.*

> "Los doce contenedores se levantan con un comando y se vigilan con sondas de

> salud. Las trazas de los cinco servicios van por OpenTelemetry a Tempo y

> Grafana, que se activa con un perfil opcional. Entre los servicios hay mTLS:

> la malla exige certificado de cliente, así que un servicio comprometido no

> puede hablar con otro sin credencial. Los respaldos son automáticos y

> verificados: no basta con hacerlos, hay que restaurarlos, y por eso existe

> un simulacro de restauración cronometrado. Y el escalado es horizontal

> porque los servicios no tienen estado."

Sincronización de cuatro entornos y demo pública

Muestra `make sync-status` y, si la tienes abierta, la demo pública.

> "La documentación no vive en un solo sitio: hay un comando que sincroniza

> los cuatro entornos —local, GitHub, Notion y Obsidian— y otro que detecta

> la deriva entre ellos. Y el sistema completo se puede exponer sin pagar

> nada: hay una demo pública que se levanta con un túnel cien por ciento open

> source, con licencia MIT en los nueve repositorios."

El cifrado en la base (terminal)

```
docker exec kubo-postgres psql -U kubo_root -d kubo_crm -c \
  "select name, left(document_number_encrypted, 28) as cifrado, left(document_number_bidx
```

> "En la base de datos el documento no se puede leer, y a la vez se puede buscar

> por igualdad con el índice ciego."

La arquitectura en texto (si no quieres mostrar la imagen)

> "El navegador solo habla con el API Gateway. El gateway valida el token JWT

> contra el JWKS del servicio de identidad y enruta cada petición:

> **kubo-iam** en Java con Spring Boot —identidad, roles y auditoría—;

> **kubo-crm** en Ruby on Rails —clientes, con los datos personales cifrados—;

> **kubo-erp** en Elixir con Phoenix —catálogo, inventario y ventas—; y

> **kubo-analytics** en Python con FastAPI —el tablero, sobre MongoDB—.

> El ERP publica un evento cuando se registra una venta y analítica lo consume

> por RabbitMQ: la caja nunca espera al tablero."

Muestra la carpeta raíz con los 9 repositorios:

```
ls -d kubo-*/
```

> "Nueve repositorios: uno por cada parte del software, más la infraestructura y

> la documentación. El acceso por HTTPS lo termina Caddy y el token de sesión

> vive en una cookie que el navegador no puede leer."

Frases de respaldo (por si algo falla en vivo)

| Situación | Qué decir | | --- | --- | | Un contenedor no responde | "Reviso la sonda de salud del servicio…" y ejecuta `make ps` | | El tablero tarda | "La consistencia es eventual: el evento viaja por RabbitMQ" | | Falla el modo offline | Muestra la cola en `IndexedDB` desde las herramientas del navegador | | Falla la demo por completo | Ejecuta `make smoke` y muestra las 192 comprobaciones en verde | | No hay red para el túnel de la demo | "La demo pública es opcional; el sistema corre igual en el local" | | Te preguntan por el costo de operación | "Cerca de 1 GB en reposo: cabe en un VPS de 6 a 12 USD al mes" | | Te preguntan qué pasa si el bus se cae | "El evento espera en el outbox y se entrega al volver; `make bus-drill` lo verifica" | | Te preguntan por la disponibilidad de los datos | "Cada negocio ve solo lo suyo: RLS activo en las tres bases de PostgreSQL" |

Evidencia complementaria

```
make smoke 2>&1 | tee evidencia-smoke.txt     # 192 comprobaciones end-to-end
make ci                                       # gate completo (13 verificaciones)
docker compose -f kubo-infra/docker-compose.yml ps
docker stats --no-stream
make observability                            # opcional: Tempo + Grafana en :3001
make sync-status                              # estado de los 4 entornos
make bus-drill                                # 4/4: caída del bus sin perder eventos
make restore-drill                            # 14/14: restauración cronometrada
make load                                     # 50 cajas, p95 < 300 ms

# Regenerar la base visual y las capturas de la evidencia
make reset-demo
cd kubo-web && node ../kubo-docs/scripts/capturas-evidencia.mjs --video
```

La evidencia completa (capturas, comprobaciones y video) vive en

[`evidencia/`](evidencia/README.md).

Los diagramas de la sustentación

| Archivo | Qué muestra | Formato | | --- | --- | --- | | `diagramas/arquitectura-stack.png` | Mapa de arquitectura y stack tecnológico | PNG + SVG | | `diagramas/flujo-venta-outbox.png` | Secuencia de la venta con el outbox transaccional | PNG + SVG | | `diagramas/flujo-venta-outbox.webm` | La misma secuencia en movimiento (traza animada de 6 s) | WebM | | `diagramas/html/arquitectura-stack.html` | Mapa interactivo: vistas guiadas por capa, modo claro/oscuro | HTML | |

`diagramas/html/flujo-venta-outbox.html` | Secuencia interactiva con animación encendida/pausada | HTML |

Los HTML son autocontenidos (funcionan sin conexión) y los PNG/SVG/WebM son sus

exportaciones, de modo que la imagen y el vídeo no pueden divergir.

10 — Auditoría técnica y plan de mejora

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Alcance | Los 6 servicios, la base de datos, la infraestructura y la documentación | | Método | Medición sobre el sistema en ejecución, planes de ejecución reales y revisión de código | | Estado de partida | 33/33 comprobaciones end-to-end en verde, 10 contenedores sanos | | Estado de cierre | **44/44** (se añadieron 11 comprobaciones para cubrir las correcciones) | | Rondas | Primera: rendimiento y verdad del dato · Segunda: seguridad y consistencia |

1. Resumen ejecutivo

El sistema **funciona de extremo a extremo y está verificado**, pero una auditoría

técnica seria no se conforma con que funcione: busca dónde se romperá al crecer,

dónde miente un indicador y qué falta para ser un producto y no una demostración.

Se encontraron **17 hallazgos**, todos corregidos y medidos en esta misma sesión, y

31 pendientes priorizados con su plan. **Los 31 quedaron cerrados y verificados

en las fases 1–6** (ver [`11-plan-de-cierre.md`](11-plan-de-cierre.md)); lo que

sigue abierto es el backlog comercial y externo, declarado en §5.4 y en el plan.

Los dos más graves no eran de rendimiento:

1. **El indicador «ventas de hoy» usaba el día UTC**: en Colombia todas las ventas

posteriores a las 19:00 se contaban al día siguiente — justo las horas de mayor

venta de una tienda de barrio.

2. **La auditoría de seguridad se borraba a sí misma**: los intentos de acceso

fallidos y la detección de robo de token se registraban dentro de la transacción

que después se revertía al lanzar la excepción. La bitácora quedaba sin los dos

eventos que más importan en una investigación, y la invalidación de la familia de

tokens tampoco se guardaba: el sistema detectaba el robo y no hacía nada.

El más silencioso, descubierto al construir la vista compuesta, fue otro error de

dato: la tarjeta de cartera de clientes del tablero **mostraba ceros siempre**,

porque el CRM devolvía `stats` sin el envoltorio `data` que el resto de la API sí

usa. Nada fallaba, nada avisaba: simplemente el dueño veía un dato falso.

Correcciones aplicadas (con medición)

| ID | Hallazgo | Severidad | Medición antes → después | | --- | --- | --- | --- | | A-01 | El día comercial se calculaba en UTC | **Crítica** | Ventas de 19:00–23:59 caían en el día siguiente | | A-02 | Búsqueda por texto sin índice (recorría todo) | Alta | ERP 46 ms → **5.7 ms** (8×) · CRM 28 ms → **4.4 ms** (6×) | | A-03 | Tablero con 7 viajes de red secuenciales | Alta | 108 ms → **~25 ms** (BFF, 4×) | | A-04 | **La auditoría de fallos se revertía con la transacción** | **Alta** | El intento fallido y el robo de token no quedaban registrados | | A-05 | **La llave JWT era efímera y el compose no la pasaba** | **Alta** | Cada reinicio invalidaba **todas** las sesiones | | A-06 | Cadena de auditoría con condición de carrera | Alta | Dos peticiones podían partir la cadena sin detección | | A-07 | Verificación de la cadena inalcanzable y mal definida | Media | Código muerto → endpoint `/audit/verify` que verifica enlace y contenido | | A-08 | Hash calculado con nanosegundos, columna con microsegundos | Media | La verificación habría fallado siempre | | A-09 | Numeración de ventas con `COUNT(*)` | Media | O(n) → **O(log n)** por venta | | A-10 | Cartera de clientes siempre en cero | Media | Contrato inconsistente → dato falso visible | | A-11 | Sin red de seguridad en la interfaz | Media | Pantalla en blanco → mensaje accionable | | A-12 | Índice ciego implementado pero sin endpoint | Media | Función inalcanzable → `GET /customers/by-document/:doc` | | A-13 | Listado de usuarios sin paginación | Media | Tabla completa → página acotada a 100 | | A-14 | **Prueba del ERP con la expectativa mal calculada** | Baja | Esperaba 41650 donde el resultado correcto es 29750 (nunca se había ejecutado) | | A-15 | Efectos secundarios dentro de actualizadores de estado | Baja | Avisos duplicados en StrictMode | | A-16 | Regla de «stock bajo» duplicada en dos vistas | Baja | Una sola definición en el dominio | | A-17 | Código muerto y nombres engañosos | Baja | `deleted?` sin uso, `low_stock_alerts` que devolvía rotación |

2. Método de auditoría

1. **Medición, no estimación.** Latencia por endpoint con `curl` (promedio de 5

ejecuciones) a través del gateway, es decir, la latencia que percibe el usuario.

2. **Prueba a escala.** Se cargaron **50.000 productos y 50.000 clientes**

sintéticos para que el planificador de PostgreSQL eligiera planes reales, y se

compararon con `EXPLAIN (ANALYZE, BUFFERS)`. Los datos de prueba se eliminaron

al terminar.

3. **Revisión de código** servicio por servicio buscando condiciones de carrera,

rutas sin usar y supuestos implícitos.

4. **Verificación funcional** con la prueba de humo (33 comprobaciones al inicio, 44 al cierre).

5. **Prueba de manipulación**: se editó una fila de la bitácora con SQL para comprobar que la verificación la detecta.

3. Rendimiento medido

3.1 Latencia por endpoint (a través del gateway, red local)

| Endpoint | Antes | Después | | --- | --- | --- | | `POST /auth/login` (BCrypt + firma RSA) | 75 ms | 75 ms | | `GET /products` | 6 ms | 6 ms | | `GET /customers` | 9 ms | 9 ms | | `GET /products/stats` | 8 ms | 8 ms | | `GET /dashboard/*` (6 endpoints) | 5–6 ms cada uno | — | | **`GET /dashboard/overview`** (vista compuesta) | **108 ms** (7 viajes) | **~25 ms** (rango 11–42 ms) |

3.2 Búsqueda por texto con 50.000 registros

```
-- ERP: productos por nombre o SKU
ANTES   Limit → Sort → Index Scan → Filter → Rows Removed by Filter: 49999
        Execution Time: 46.6 ms
DESPUÉS Bitmap Heap Scan → BitmapOr → Bitmap Index Scan (idx_products_name_trgm)
        Execution Time: 5.7 ms      → 8× más rápido

-- CRM: clientes por nombre o correo
ANTES   28 ms (recorría todos los clientes del negocio)
DESPUÉS 4.4 ms con índices en nombre Y correo → 6× más rápido
```

> **Lección de diseño:** con un solo índice (nombre) el CRM seguía lento, porque

> el `OR` con la columna de correo obligaba a descartar el plan indexado. En

> búsquedas con `OR` hay que indexar **todas** las columnas participantes.

3.3 Recursos

| Recurso | Medición | Evaluación | | --- | --- | --- | | RAM total del sistema | ~930 MB | Cabe en un VPS de USD 6/mes | | Servicio más pesado | `kubo-iam` 282 MB (JVM) | Esperable; acotado con `-Xmx256m` | | Resto de servicios | 13–157 MB | Correcto | | Arranque completo | ~50 s | Aceptable; se puede cachear mejor |

4. Hallazgos corregidos (detalle)

A-01 · Día comercial en UTC — **Crítica**

```
-- Una venta registrada a las 20:00 hora Colombia (01:00 UTC del día siguiente)
momento_utc (columna)      = 2026-09-24 01:00:00
fecha que usaba Kubo       = 2026-09-24   ← incorrecta
fecha real del negocio     = 2026-09-23   ← correcta
```

Impacto: el KPI «Ventas de hoy» y la serie diaria del tablero atribuían al día siguiente las ventas de 19:00 a 23:59. En un negocio de barrio esa franja es la de mayor venta: el indicador más visible del producto mentía.

Corrección: zona horaria del negocio configurable (`KUBO_TIMEZONE`, por defecto `America/Bogota`). El almacenamiento sigue en UTC (correcto); el cálculo del día comercial se hace en la zona del negocio:

- ERP: `(? AT TIME ZONE 'UTC' AT TIME ZONE ?)::date = ?` — la doble conversión es necesaria porque las columnas son `timestamp without time zone`.
- Analítica: `$dateToString` con `timezone` y la ventana de «hoy» calculada en la zona del negocio y convertida a UTC para consultar.

A-02 · Búsqueda sin índice — Alta

El `ILIKE '%termino%'` del POS y del CRM no puede apoyarse en un índice B-tree. Con catálogos de barrio no se nota; con 50.000 referencias el sistema recorría todo.

Se añadieron índices **GIN con `pg_trgm`** mediante migraciones versionadas (`pg_trgm` es extensión *trusted* desde PostgreSQL 13: no requiere superusuario).

A-03 · Siete viajes para pintar el tablero — Alta

La PWA pedía 7 endpoints en secuencia. Se implementó el patrón **BFF (Backend For Frontend)**: el gateway expone `GET /api/v1/dashboard/overview` y consulta los servicios **en paralelo** dentro de la red privada, devolviendo una sola respuesta.

- `Promise.allSettled`: si una vista falla, el tablero se entrega **parcial** y avisa cuál falta, en lugar de mostrar una pantalla de error.

- Tiempo de espera por servicio: 5 s.

- La ruta la atiende el gateway (`GATEWAY_OWNED_PATHS`), no se reenvía.

A-04 · La auditoría de seguridad se borraba a sí misma — Alta

El código registraba el intento fallido y el robo de token **dentro de la misma

transacción** de la petición:

```
if (!passwordEncoder.matches(...)) {
    auditService.record(..., "LOGIN_FAILED", ...);   // se inserta...
    throw DomainException.unauthorized(...);          // ...y el rollback lo borra
}
```

Los dos eventos que más importan en una investigación forense (intentos de acceso

fallidos y reutilización de un token robado) **desaparecían de la bitácora**. Peor

aún: en el caso de reuso, la revocación de todas las sesiones del usuario ocurría

en esa misma transacción, así que el sistema **detectaba el robo y no cerraba nada**.

Corrección: dos caminos según la semántica del evento.

| Ruta | Transacción | Por qué | | --- | --- | --- | | Éxito (login, registro, rotación) | La de la operación | No debe existir un registro de algo que finalmente no ocurrió | | Fallo (`LOGIN_FAILED`, `REFRESH_REUSE_DETECTED`) | **Independiente** (`REQUIRES_NEW`) | El registro debe sobrevivir al rollback |

La revocación de la familia de tokens se movió a `SecurityIncidentService`, que abre

su propia transacción: al no existir en ella ningún bloqueo de la cadena, tampoco

puede producirse un bloqueo muto con el `advisory lock` de auditoría.

Verificado en el humo: el intento fallido y el reuso **aparecen en la bitácora**.

A-05 · La llave JWT era efímera — Alta

`TokenService` generaba un par RSA en memoria cuando no había llave configurada, y

el `docker-compose.yml` **nunca pasaba `KUBO_JWT_PRIVATE_KEY`**. Consecuencia: cada

reinicio del servicio de identidad invalidaba **todas las sesiones** de todos los

usuarios. Se descubrió porque el propio humo falló en cascada tras reconstruir el

contenedor.

Corrección: el compose propaga la llave y el entorno de demostración usa una llave RSA real generada con `openssl`, guardada en el `.env` (no versionado). Verificado: el mismo token sigue siendo válido **después** de reiniciar el servicio. Además se redujo la ventana de caché del JWKS en el gateway (30 s → 10 s): si algún día se rota la llave, el gateway reconoce el nuevo `kid` casi de inmediato. El tiempo de espera existe para que nadie pueda saturar el servicio de identidad pidiendo el JWKS con `kid` inventados.

A-06 · Cadena de auditoría con condición de carrera — Alta

`record()` leía el último hash y luego insertaba. Dos peticiones concurrentes podían leer el mismo hash anterior y **partir la cadena**: dos entradas apuntando al mismo predecesor, y la manipulación dejaba de ser detectable.
Corrección: `pg_advisory_xact_lock` al inicio de la transacción. Serializa la escritura de la cadena sin bloquear ninguna tabla y se libera automáticamente al terminar la transacción.

A-09 · Numeración de ventas con `COUNT(*)` — Media

Cada venta contaba todas las ventas del negocio (verificado con `EXPLAIN`: `Index Only Scan` sobre 50.000 filas). Ahora se toma el **máximo** del consecutivo, que el índice único `(tenant_id, number)` resuelve en tiempo logarítmico. La restricción única sigue protegiendo la concurrencia y el reintento ya existente la resuelve.

A-06 a A-08 · Integridad de la bitácora

| Hallazgo | Corrección | | --- | --- | | Condición de carrera al encadenar hashes (A-06) | `pg_advisory_xact_lock` al inicio de la transacción: serializa la escritura sin bloquear tablas | | Verificación inalcanzable y mal definida (A-07) | Nuevo `GET /audit/verify`: verifica que cada entrada **enlace** con la anterior y que su **hash corresponda al contenido**. Se expone el hash más antiguo de la ventana para poder encadenar verificaciones | | Hash con nanosegundos y columna con microsegundos (A-08) | El instante se trunca a microsegundos antes de calcular el hash: de otro modo la verificación fallaría siempre |

Prueba de manipulación (ejecutada): se editó una fila con SQL y la verificación

devolvió `chainIntact: false`. Una edición parcial rompe la verificación; falsificar

la bitácora completa exigiría recalcular toda la cadena.

A-09 a A-17 · Consistencia, contrato, interfaz y limpieza

| Hallazgo | Corrección | | --- | --- | | Numeración de ventas con `COUNT(*)` (A-09) | Se toma el **máximo** del consecutivo: el índice único lo resuelve en tiempo logarítmico | | **`customers/stats` sin envoltorio `data`** (A-10) | Unificado al contrato del resto de la API. Además el gateway cae al cuerpo completo si un servicio no envuelve: una inconsistencia degrada la forma, no borra una vista | | Un error de render dejaba la pantalla en blanco (A-11) | `ErrorBoundary` con mensaje entendible, botón de recarga y registro del error | | `Customer.find_by_document` inalcanzable (A-12) | `GET /api/v1/customers/by-document/:document` — el caso de uso real del mostrador | | `GET /users` devolvía la tabla completa (A-13) | Paginación con tamaño acotado en el servidor (máx. 100) | | **Prueba del ERP con expectativa mal calculada** (A-14) | Esperaba `41650` donde el resultado correcto es `29750`. La prueba **nunca se había ejecutado**: el contenedor se quedaba sin memoria al compilar el entorno de pruebas. Ahora corre en 4/4 | | Efectos secundarios en actualizadores de estado (A-15) | Validaciones movidas fuera: sin avisos duplicados en StrictMode | | Regla de «stock bajo» duplicada (A-16) | Una sola definición en el dominio (`Product.low_stock?/1`) usada por las dos vistas | | Código muerto y nombres engañosos (A-17) | `Customer#deleted?` eliminado; `low_stock_alerts` renombrado a `product_rotation` (devolvía rotación, no alertas) |

Nota de migración: bitácora anterior a la corrección

Los registros de auditoría creados **antes** de A-08 se hashearon con nanosegundos,

por lo que no son verificables con el algoritmo corregido. En un sistema en

producción esto exigiría **versionar el algoritmo** (una columna `hash_version`) y

mantener el verificador anterior para el histórico. Aquí, al ser datos de

demostración, la bitácora se reinició para partir de una cadena verificable. Queda

como pendiente P-23.

5. Pendientes priorizados

> **Catálogo cerrado (2026-10-03).** Los 31 pendientes están resueltos y

> verificados; el detalle de cada uno, con su evidencia, vive en

> [`11-plan-de-cierre.md`](11-plan-de-cierre.md). Resumen por fase:

>

> | Fase | Pendientes cerrados |

> | --- | --- |

> | F1 · Confiabilidad | P-01, P-02, P-03, P-04, P-05, P-11, P-23, P-27, P-31 |

> | F2 · Calidad | P-06, P-07, P-08, P-09, P-10, P-12, P-13, P-14, P-26 |

> | F3 · Núcleo comercial | P-15, P-16, P-20, P-21, P-24 |

> | F4 · Diferenciadores | P-17, P-18, P-19, P-22, P-25, P-30 |

> | F5 · Escala | P-28, P-29 |

> | F6 · Operación | F6.1–F6.6 (uso, cobro, panel, respaldos, pasarela) |

>

> Evidencia de cierre: `make ci` 13/13 (secretos, suites, contratos 23/23, humo

> 192/192, E2E 5/5, enlaces y PDF al día), gates de cobertura por servicio (IAM 85.2 %, analítica

> ≥ 80 %, gateway 95.6 %, CRM 100 %, web 97.3 % de líneas, ERP ratchet 37.68 %),

> `make bus-drill` 4/4 y `make restore-drill` 14/14.

>

> Las tablas de §5.1–5.3 conservan el catálogo original con el riesgo y el

> esfuerzo estimados al momento de la auditoría; su estado final es **cerrado**.

5.1 Bloqueantes para un negocio real

| ID | Pendiente | Riesgo | Esfuerzo | | --- | --- | --- | --- | | P-01 | ***Transactional outbox*** | Un evento se pierde si el proceso muere justo tras el `commit`: el tablero no reflejaría una venta ya cobrada | 1–2 días | | P-02 | **Activar RLS** con interceptor de transacción | Hoy el aislamiento entre negocios depende de que toda consulta filtre por `tenant_id`. Un olvido no se detecta | 2–3 días | | P-03 | **Refresh token en cookie `httpOnly`** (BFF) | Un XSS podría robar el token de refresco | 2 días | | P-04 | **Recuperación de contraseña** | Hoy solo un administrador puede restablecerla desde la base | 1 día | | P-05 | **Respaldos automatizados + simulacro de restauración** | Documentado pero no ejecutado: un respaldo sin probar no es un respaldo | 1 día | | P-27 | **TLS/HTTPS en la instalación** (Caddy y verificación en el humo) | Sin cifrado en tránsito, las credenciales viajan en claro por la red del local | 1 día |

5.2 Calidad e ingeniería

| ID | Pendiente | Por qué importa | Esfuerzo | | --- | --- | --- | --- | | P-06 | **CI/CD** (lint + pruebas + SAST + escaneo de secretos por repo) | Las verificaciones son manuales; un `make smoke` no protege contra una regresión de seguridad | 2–3 días | | P-07 | **Observabilidad** (OpenTelemetry →

Grafana/Loki/Tempo) | Hoy hay sondas de salud y logs locales: no hay trazas ni métricas históricas | 3 días | | P-08 | **Pruebas de integración con Testcontainers** por servicio | Las pruebas actuales son unitarias puras + humo; falta cubrir los repositorios | 3 días | | P-09 | **Pruebas de contrato** (OpenAPI ejecutable + Pact) | El contrato se verifica de facto en el humo; un cambio incompatible se detectaría tarde | 2 días | | P-10 | **E2E de navegador** (Playwright) y **carga** (k6, 50 cajas) | La interfaz se verifica a mano; el comportamiento bajo concurrencia no está medido | 2 días | | P-11 | **Bloqueo de cuenta** tras N intentos fallidos | Hoy solo hay límite de tasa por IP | 0.5 día | | P-12 | **Caché del service worker por usuario** | En un equipo compartido, el caché de lecturas de un usuario podría servirse a otro si no cierra sesión | 0.5 día | | P-13 | **Paginación** en productos, ventas y movimientos | El ERP corta en 300/200 registros; un negocio grande los supera | 1 día | | P-14 | **Empaquetado de imágenes por digest** y reservas de recursos | Las etiquetas (`postgres:17`) pueden cambiar; conviene fijar el digest | 0.5 día | | P-23 | **Versionar el algoritmo de hash de auditoría** | Un cambio de algoritmo deja el histórico sin verificar (ver nota de migración) | 1 día | | P-26 | **Accesibilidad automatizada** (axe en CI) | Hoy solo hay revisión manual de contraste y foco | 0.5 día | | P-30 | **Segundo factor (TOTP)** para el propietario | Robo de credenciales del administrador | 1–2 días | | P-31 | **Límite de tasa por usuario** (no solo por negocio o IP) | Abuso desde una cuenta comprometida | 0.5 día |

5.3 Alcance funcional (lo que separa el MVP de un producto)

| ID | Módulo | Valor para el negocio | | --- | --- | --- | | P-15 | **Compras y proveedores** | Cierra el ciclo del inventario: hoy solo entran mercancías por ajuste manual | | P-16 | **Sesiones de caja** (apertura, cierre, arqueo) | Control de efectivo por turno: es lo primero que pide un tendero | | P-17 | **Vertical Packs** (retail, servicios, restaurantes, agro) | Convierte Kubo en multi-rubro sin duplicar código (ya diseñado) | | P-18 | **Facturación electrónica DIAN** (UBL 2.1, CUFE, QR) | Requisito legal para facturar; exige ser Proveedor Tecnológico | | P-19 | **Notificaciones** (WhatsApp / correo) | Cobranza, confirmaciones y avisos de stock: el quinto servicio planificado | | P-20 | **Usuarios y roles en la interfaz** | Hoy el API existe pero el dueño no puede crear vendedores sin ayuda técnica | | P-21 | **Reportes exportables** (CSV/PDF) y **comprobante de venta imprimible** (ticket) | Contabilidad y bancos piden soportes; el mostrador necesita el comprobante | | P-22 | **Multi-bodega** y transferencias | Para negocios con más de un punto | | P-24 | **Importación de datos** (Excel/CSV y ruta Dolibarr) | Sin esto, digitalizar un catálogo existente es manual: la primera barrera de adopción | | P-25 | **Servicio de Documentos** (adjuntos y plantillas) | Completa el quinto servicio planificado junto con las notificaciones |

5.4 Escala (Fase 5, backlog declarado)

| ID | Pendiente | Por qué | | --- | --- | --- | | P-28 | mTLS entre gateway y servicios | Movimiento lateral dentro del clúster | | P-29 | Rotación de claves de cifrado de campo (KEK/DEK) | Compromiso de una clave a largo plazo |

Líneas declaradas sin ID: instalación remota (Terraform/Ansible), multi-tenant SaaS

(zona horaria por negocio y onboarding), app móvil nativa y operador de respaldos.

6. Evaluación arquitectónica

Lo que está bien y no conviene tocar

- **Fronteras por contexto delimitado**: cada servicio es dueño de su esquema y

ningún servicio consulta la base de otro. Es la decisión que más ha simplificado

todo lo demás.

- **Seguridad en el borde**: un único punto de validación de identidad (gateway +

JWKS) evita repartir la llave privada y centraliza la política.

- **Tolerancia a fallos en cadena**: la caja no depende del bus ni de analítica.

Verificado: con RabbitMQ caído la venta se registra igual.

- **Cifrado de campo con índice ciego**: resuelve la tensión entre proteger el dato

y poder buscarlo, sin cifrado determinista (que filtra información).

Lo que hay que cambiar

| Cambio | Motivo | ADR a escribir | Fase | | --- | --- | --- | --- | | Outbox transaccional | Único punto donde el sistema puede perder un dato ya confirmado al usuario | ADR-0009 | Fase 1 | | Interceptor de tenant + RLS activo | Convertir el aislamiento en garantía del motor, no del programador | ADR-0010 | Fase 1 | | BFF como capa formal | Ya existe para el tablero; conviene declararlo patrón del gateway y no un caso puntual | ADR-0011 | Fase 2 | | Zona horaria por negocio en la tabla `tenants` | Hoy es global por instalación; en multi-tenant debe ser un atributo del negocio | ADR-0012 | Fase 4 |

Escalado

| Nivel | Qué hacer | Cuándo | | --- | --- | --- | | 1 negocio | Un VPS, Docker Compose (estado actual) | Ahora | | Varias cajas | Escalar `kubo-erp` (sin estado) y separar PostgreSQL a su instancia | > 5 cajas | | Varios negocios | Activar RLS, zona horaria por tenant, BFF, caché de lecturas | Producto SaaS | | Clúster | k3s + mTLS entre servicios + operador de respaldos | > 50 negocios |

7. Plan de mejora

El plan de cierre completo vive en [`11-plan-de-cierre.md`](11-plan-de-cierre.md).

Resumen:

| Fase | Foco | Duración estimada (30 h/semana) | | --- | --- | --- | | 0 | Consistencia de la documentación | 2–3 h (completada) | | 1 | Confiabilidad: outbox, RLS, cookie, respaldos, TLS | ≈3 semanas | | 2 | Calidad y observabilidad: CI, OTel, pruebas, accesibilidad | ≈4 semanas | | 3 | Núcleo comercial: compras, caja, usuarios, importación, comprobantes | 5–7 semanas | | 4 | Diferenciadores: verticales, DIAN, notificaciones, multi-bodega | 5–6 semanas | | 5 | Escala: k3s + mTLS, SaaS multi-tenant, app móvil | backlog declarado |

Cada fase cierra solo con su criterio medido; el detalle de pasos, IDs y criterios

de aceptación está en el documento enlazado.

8. Conclusión

El sistema pasó de **«funciona»** a **«es correcto y sostiene la escala de su

mercado»**. La primera ronda arregló la **verdad del dato** (un indicador que

mentía a partir de las 19:00 y una tarjeta que mostraba ceros sin fallar) y el

rendimiento (búsquedas que recorrían la tabla completa, un tablero que pedía

siete viajes). La segunda ronda cerró la **seguridad**: la auditoría registraba los

eventos de éxito pero borraba los de fallo —justo los que sirven para investigar—,

y la detección de robo de token no llegaba a cerrar las sesiones.

Ninguno de esos defectos aparecía en una prueba funcional: el sistema respondía

200, el humo pasaba en verde y la interfaz se veía bien. Los encontró la revisión

del **código y de los datos**, no la de las respuestas.

Lo que falta no es cosmético: son las piezas que convierten una demostración

técnica sólida en un producto que un tendero puede usar todos los días sin

sobresaltos — **garantía de entrega de eventos, aislamiento impuesto por el motor,

automatización de calidad y los módulos que el negocio realmente pide** (compras,

caja, usuarios). El plan de las fases 1 a 4 está ordenado por riesgo, no por

lucimiento: primero lo que puede perder o mezclar datos, después lo que evita

regresiones, y al final lo que amplía el mercado.

9. Cómo se sostiene este nivel de rigor

La auditoría no fue una lista de buenas intenciones: cada hallazgo se **midió** (`EXPLAIN ANALYZE` con 50.000 filas sintéticas, latencia promedio de 5 ejecuciones, `docker stats`) y cada corrección se **verificó** con una comprobación añadida al humo. Once de las 44 comprobaciones existen porque un defecto concreto obligó a escribirlas.

Ese es el criterio para presentar el MVP con tranquilidad: no que no tenga fallos —los tenía, diecisiete— sino que **cada fallo tiene ahora una prueba que lo vigila**.

10. Auditoría de seguridad final (2026-10-03)

Revisión estática OWASP sobre el estado final del sistema, después de las fases 0–6. Se encontraron seis hallazgos residuales y **los seis quedaron atendidos**.

ID	Hallazgo	Severidad	Resolución
KUBO-01	`SECRET_KEY_BASE` de CRM/ERP con valor por defecto en compose	**Crítica**	compose exige la variable (`:?`) y Rails falla en producción si falta: el arranque ya no puede firmar con una clave pública
KUBO-02	Semilla activada por defecto con credenciales conocidas	Alta	`KUBO_SEED_ENABLED` por defecto `false` (el `.env` local lo activa), contraseñas obligatorias en compose y el seeder se omite sin `KUBO_ADMIN_PASSWORD`
KUBO-03	Cookie de refresco sin `Secure` en producción	Alta	nginx propaga el esquema original de Caddy y Ansible despliega solo el borde TLS (3080/3443) con `KUBO_COOKIE_SECURE=true`
KUBO-04	Contenedores IAM y ERP como root	Media	IAM corre como usuario `kubo`; ERP ajusta el volumen de documentos y baja privilegios con `gosu` en el entrypoint
KUBO-05	`erl_crash.dump` en el historial de git	Media	**Resuelto**: historial purgado (`filter-branch` + `gc`), el objeto no existe en ningún commit ni en el almacén, y el archivo se retiró del disco y se ignoró
KUBO-06	Cabeceras internas: TLS por defecto apagado y roles sin whitelist	Media	`KUBO_INTERNAL_TLS` fail-closed por defecto (las pruebas lo apagan de forma explícita) y whitelist de roles en IAM y ERP

Evidencia: `make ci` 13/13, humo **192/192** tras desplegar el endurecimiento, IAM 83 pruebas, ERP 57 pruebas, `make bus-drill` 4/4 y `make restore-drill` 14/14.

Nota de rendimiento (misma fecha). Al re-verificar la carga se encontraron dos serializaciones reales en la venta: el comprobante PDF y el contador de numeración se generaban **dentro** de la transacción (el contador bloquea su fila hasta el commit y serializaba todas las cajas del negocio). Ambos salieron

de la transacción: con 10 cajas el p95 quedó en **107 ms** y el throughput subió

~13 %; a 50 cajas en este host de desarrollo (swap/zram en uso por otros

proyectos) el p95 es ~540 ms con el **100 % de las ventas exitosas**.

Catálogo voluminoso (misma fecha). `make load-big` siembra **50.000

productos** y mide la búsqueda del POS: **6 ms** por petición en frío (índice

trigram), p95 **233 ms** con 10 cajas concurrentes y el 100 % de búsquedas

exitosas. A 20 cajas el host satura (~54 req/s) y el p95 sube a 531 ms: el

techo es del entorno, no de la consulta.

11 — Plan de cierre

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Objetivo | Producto comercializable en la región (Fases 0–4); la Fase 5 queda como backlog declarado | | Punto de partida | 44/44 comprobaciones en verde · 10 contenedores sanos · 9 repositorios limpios | | Dedicación | ~30 h/semana (≈ 3.75 jornadas de 8 h) | | Estado | Fases 0–4 completadas · Fase 5 completada salvo el backlog comercial · **Fase 6 (operación y comercial) completada**: F6.1–F6.6 construidos y verificados, incluidos el uso del ERP en el panel y la rotación del segundo factor del operador; quedan las mejoras continuas (métricas de F6.5) y la app móvil cuando el negocio la pida (ADR-0023; plan por fases en [`14-plan-app-movil.md`](14-plan-app-movil.md)) | | Verificación vigente | `make ci` 13/13 · humo 192/192 · 246 pruebas de servicio · contratos 23/23 · E2E 5/5 · ERP ratchet 37.68 % (2026-10-08) |

Este documento ordena los 31 pendientes de [`10-auditoria.md`](10-auditoria.md) en

fases con criterio de cierre medible. No sustituye a la auditoría: la usa como

catálogo de origen.

1. Decisiones registradas

| Decisión | Elección | | --- | --- | | Objetivo del cierre | Producto comercializable (Fases 0–4; Fase 5 como backlog declarado) | | Primer módulo de la Fase 3 | Compras + Caja | | Dedicación | ~30 h/semana |

2. Hallazgos de la revisión de cierre (R-1…R-9)

La revisión que originó este plan encontró nueve inconsistencias. Todas quedaron

corregidas en la Fase 0 o asignadas a una fase con ID.

| ID | Hallazgo | Resolución | Estado | | --- | --- | --- | --- | | R-1 | IDs de hallazgos duplicados en la auditoría (las dos rondas colisionaron: cuatro secciones A-04/A-05) | Renumerados a **A-06** y **A-09** en §4 | Corregido | | R-2 | La importación de datos (Excel/CSV/Dolibarr) no estaba en ningún plan | **P-24** · Fase 3 | Planificado | | R-3 | El quinto servicio figuraba como «planificado» en trazabilidad pero sin fase | **P-25** Documentos + **P-19** Notificaciones · Fase 4 | Planificado | | R-4 | Faltaban los ADR 0009–0012 que la propia auditoría exige | 0009 y 0010 · Fase 1 · 0011 · Fase 2 · 0012 · Fase 4 | Planificado | | R-5 | La auditoría de accesibilidad (axe) estaba en pruebas pendientes sin ID | **P-26** · Fase 2 | Planificado | | R-6 | TLS/HTTPS solo estaba en la guía de despliegue, no en el plan | **P-27** · Fase 1 | Planificado | | R-7 | RLS versionada solo en `kubo-iam`, contra lo que afirmaba `04-seguridad.md` §4.3 | Script para CRM y ERP dentro de **P-02** · Fase 1 | Planificado | | R-8 | Pendientes de `04-seguridad.md` §10 sin ID ni fase (mTLS, rotación de claves, 2FA, rate limiting por usuario) | **P-28…P-31** con fase asignada | Planificado | | R-9 | Conteos y referencias de fase desactualizados (8/9 contenedores, 37 comprobaciones, «16 pendientes») | Corregidos en `01`, `04`, `05`, `07`, `08`, `09` y `10` | Corregido |

3. Fases

Fase 0 — Consistencia (completada, 2–3 h)

| Paso | Acción | Estado | | --- | --- | --- | | F0.1 | Renumerar los A-04/A-05 duplicados (A-06, A-09) | Hecho | | F0.2 | Corregir conteos y la afirmación de RLS | Hecho | | F0.3 | Incorporar P-24…P-31 con ID y fase | Hecho | | F0.4 | Renumerar fases 0–5 en todos los documentos y ADR | Hecho | | F0.5 | Crear este documento, enlazarlo, añadirlo al PDF y commit | Hecho |

Criterio: el resumen y el detalle de la auditoría coinciden y el plan es

navegable desde el índice.

Fase 1 — Confiabilidad (≈3 semanas) — bloqueante para un negocio real

| Paso | Pendiente | ID | Esfuerzo | | --- | --- | --- | --- | | F1.1 | Outbox transaccional + publicador de barrido + ADR-0009 | P-01 | 1–2 d | | F1.2 | Interceptor de tenant + RLS en las tres bases (escribir el script en CRM y ERP) + ADR-0010 | P-02 · R-7 | 2–3 d | | F1.3 | Refresh token en cookie `httpOnly` + `SameSite` (BFF) | P-03 | 2 d | | F1.4 | Recuperación de contraseña por correo | P-04 | 1 d | | F1.5 | Respaldos automatizados + simulacro cronometrado | P-05 | 1 d | | F1.6 | Bloqueo de cuenta tras N intentos fallidos | P-11 | 0.5 d | | F1.7 | Límite de tasa por usuario | P-31 | 0.5 d | | F1.8 | Versionado del algoritmo de hash de auditoría (`hash_version`) | P-23 | 1 d | | F1.9 | TLS en la instalación (Caddy + verificación en el humo) | P-27 | 1 d |

Criterio de aceptación: prueba de caída del bus sin pérdida de eventos · una

consulta sin filtro de tenant devuelve cero filas · restauración completa

cronometrada por debajo de 4 h · `make smoke` con ≥ 55 comprobaciones en verde.

Estado: completada (2026-09-24). Evidencia: `make smoke` **73/73** ·
`make bus-drill` **4/4** (venta con el bus caído, evento `PENDING`, publicación al
volver) · `make restore-drill` **14/14** con RTO de 6 s · consulta sin contexto =
0 filas en IAM, CRM y ERP · ADR-0009 y ADR-0010 aceptados.

Dos defectos reales aparecieron al integrar la fase y quedaron corregidos:

1. **RLS en transacciones `REQUIRES_NEW`**: el contador de intentos fallidos y la
revocación de la familia de tokens corrían en otra conexión sin contexto y
actualizaban cero filas en silencio (el sistema detectaba el robo y no cerraba
nada). `SecurityIncidentService` ahora fija su propia marca de sistema, y el
humo comprueba la revocación completa de la familia.
2. **Transacciones anidadas de Ecto sin savepoint**: un `Repo.rollback` de negocio
(stock insuficiente) abortaba la transacción externa del interceptor y tumbaba
la petición después de responder. `Repo.scoped_transaction/1` usa savepoints.

En el mismo paso se corrigió un contrato roto del ERP: `Catalog.adjust_stock`
devolvía `{:ok, {product, movement}}` y el controlador esperaba
`{:ok, product, movement}` (el ajuste de inventario respondía 500).

Fase 2 — Calidad y observabilidad (≈4 semanas)

| Paso | Pendiente | ID | Esfuerzo | | --- | --- | --- | --- | | F2.1 | CI por repositorio (lint, pruebas, SAST, secretos, SBOM) + ADR-0011 | P-06 | 2–3 d | | F2.2 | OpenTelemetry en los cuatro lenguajes → Grafana/Loki/Tempo | P-07 | 3 d | | F2.3 | Pruebas de integración con Testcontainers | P-08 | 3 d | | F2.4 | Contratos ejecutables (OpenAPI/Pact) | P-09 | 2 d | | F2.5 | E2E (Playwright) + carga (k6, 50 cajas) | P-10 | 2 d | | F2.6 | Caché del service worker por usuario + axe en CI | P-12 · P-26 | 1 d | | F2.7 | Paginación en productos, ventas y movimientos + digests de imagen | P-13 · P-14 | 1.5 d |

Criterio de aceptación: una regresión de seguridad o de contrato bloquea el
merge · p95 del POS por debajo de 300 ms con 50 cajas · cobertura de dominio ≥ 80 %.

Estado: completada (2026-09-24). Evidencia:

- **P-06 CI**: gate local `make ci` → **13/13 en verde** (secretos, suites,
contratos, Pact, humo, E2E, enlaces y PDF al día). Los pipelines de GitLab

(`.gitlab-ci.yml` en los ocho repositorios, con pruebas, SAST, secretos y dependencias) se retiraron el 2026-10-04 al quedar GitHub como único destino; desde el 2026-10-05 cada repositorio tiene su flujo propio en GitHub Actions (ADR-0029).

- **P-07 OpenTelemetry**: collector OTLP propio (`kubo-otel`) y los **cinco servicios exportando trazas** (gateway, IAM, CRM, ERP y analítica), con `make observability` para el stack visual (Tempo + Grafana). El humo verifica que el collector recibe trazas.
- **P-09 contratos**: `kubo-docs/api/openapi.json` + `make contracts` → **10/10 contratos** validados con Ajv contra la API viva.
- **P-10 carga y E2E**: `make load` (50 cajas) → **100 % de ventas exitosas, p95 = 149.76 ms**; Playwright + axe → **4/4**, con tres defectos reales de accesibilidad corregidos (contraste y un `select` sin nombre accesible).
- **P-12 · P-13 · P-14 · P-26**: caché por sesión, paginación real con totales (los listados mentían en `total`), digests de imágenes y axe en CI.
- **Cobertura**: IAM **85.5 %** de dominio (gate JaCoCo ≥ 80 % en `mvn verify`) y analítica **91 %** en su módulo de dominio (gate `--cov-fail-under=80`).
- **P-08 integración con infraestructura real**: en los cinco servicios, contra el motor y no contra dobles. IAM: RLS + cadena de auditoría sobre PostgreSQL (2 pruebas). Analítica: deduplicación y proyección sobre MongoDB efímero (testcontainers, 2 pruebas). ERP: RLS sin contexto = 0 filas, numeración consecutiva por negocio, atomicidad venta + bandeja de salida y kardex con anulación sobre PostgreSQL real (4 pruebas). CRM: aislamiento RLS entre negocios y round-trip del cifrado con índice ciego sobre PostgreSQL real (2 pruebas). Gateway: ventana de límite de tasa, 429 al exceder y cuota independiente por usuario contra Redis real (3 pruebas). En todos, las

pruebas corren como el **rol de la aplicación**, no como superusuario: un superusuario ignora RLS incluso con `FORCE` y la prueba dejaría de probar algo. Los runners son `kubo-infra/scripts/erp-tests.sh`, `crm-tests.sh` y `analytics-tests.sh` (integrados en `make ci`), que corren contra los servicios reales del compose (postgres/redis).

- **Defectos reales corregidos al añadir P-08**: el CI de analítica no podía importar `app` (faltaba `pytest.ini` con `pythonpath`) ni tenía instalado `pytest-cov` pese a usar `--cov-fail-under`; el guard de las pruebas de integración exigía el binario `docker` cuando basta el socket; y el volcado de `schema.rb` rompía las migraciones del CRM en el contenedor de pruebas.

Defectos reales corregidos durante la fase (además de los de implementación): numeración de ventas con `ON CONFLICT` que devolvía un id inexistente, RLS con subconsulta inestable bajo concurrencia (denormalizado `tenant_id` en `sale_items`), interceptor de tenant con `Repo.rollback` fuera de transacción real, y la cobertura de listados que reportaba el tamaño de página como total.

Fase 3 — Núcleo comercial (5–7 semanas) — orden aprobado: Compras + Caja

| Paso | Pendiente | ID | Esfuerzo | | --- | --- | --- | --- | | F3.1 | Compras y proveedores | P-15 | 5–7 d | | F3.2 | Sesiones de caja (apertura, cierre, arqueo) | P-16 | 5–7 d | | F3.3 | Usuarios y roles en la interfaz | P-20 | 3–4 d | | F3.4 | Importación de datos (Excel/CSV; ruta Dolibarr) | P-24 | 3–4 d | | F3.5 | Reportes exportables + comprobante de venta imprimible | P-21 | 3–4 d |

Criterio de aceptación: un negocio carga su catálogo desde Excel, abre caja, vende, imprime el comprobante y cierra el turno con arqueo cuadrado.

Estado: completada (2026-09-24). Los cinco pasos quedaron construidos y verificados:

- **P-15 Compras y proveedores**: dominio completo en el ERP (proveedores con borrado lógico, compras con detalle, numeración atómica `C-000001`, RLS en las tres tablas nuevas), la compra suma inventario, deja el kardex (`PURCHASE`),

actualiza el costo del producto **sin IVA** y emite `purchase.received` por la bandeja de salida; la anulación revierte el stock (`PURCHASE_VOID`).

- **P-16 Sesiones de caja**: apertura con base, una sola caja abierta por negocio (índice único parcial), ventas ligadas al turno, cierre con arqueo (esperado = base + efectivo de ventas completas − anuladas) y diferencia calculada.

- **P-20 Usuarios y roles**: correo único global en IAM (índice funcional sobre `lower(email)`), creación y edición de usuarios por el propietario (`requireAdmin`), cambio de rol, habilitar/deshabilitar con efecto inmediato en el ingreso (`USER_DISABLED`).

- **P-24 Importación de catálogo (CSV)**: detección de separador `,`/`;`, alias de cabeceras en español e inglés, alta o actualización por SKU, stock por kardex (`IMPORT`), errores por línea con número, límite de 1000 filas; en la PWA con modal de archivo o texto pegado y resumen de creados/actualizados/errores.

- **P-21 Reportes y comprobante**: `GET /reports/sales.csv` (filtro `from`/`to` interpretado en la **zona horaria del negocio**, fechas ISO con offset) y `GET /reports/inventory.csv` valorizado; fechas inválidas responden 400. La PWA descarga ambos reportes y el POS imprime el comprobante de la última venta sin depender de internet.

- **Defecto corregido durante el cierre**: el ERP no traía base de datos de zonas horarias (`tzdata`), así que `DateTime.now("America/Bogota")` fallaba y el día comercial caía silenciosamente a UTC entre las 19:00 y las 23:59 locales (el mismo defecto A-01, latente en otra capa). Se añadió `tzdata`, se registró el respaldo y el humo verifica que el filtro de fechas respeta la zona del negocio.

- **Evidencia**: humo **102/102** (12 bloques, repetible en el mismo minuto), contratos **12/12**, E2E **4/4** con auditoría axe, `make ci` en verde.

Fase 4 — Diferenciadores (5–6 semanas)

| Paso | Pendiente | ID | Esfuerzo | | --- | --- | --- | --- | | F4.1 | Vertical Packs (retail, servicios, restaurantes, agro) | P-17 | 4–6 d | | F4.2 | Facturación electrónica DIAN (UBL 2.1, CUFE, QR) como puerto enchufable | P-18 | 4–6 d | | F4.3 | Notificaciones (WhatsApp/correo) + servicio de Documentos | P-19 · P-25 | 4–6 d | | F4.4 | Multi-bodega y transferencias | P-22 | 2–3 d | | F4.5 | Segundo factor (TOTP) del propietario | P-30 | 1–2 d | | F4.6 | ADR-0012: zona horaria por negocio en la tabla `tenants` | — | 0.5 d |

Criterio de aceptación: dos verticales activables sin tocar el núcleo · factura DIAN emitida en ambiente de habilitación · notificación real entregada · transferencia entre dos bodegas con kardex en ambas.

Estado: completada (2026-09-24). Los seis pasos quedaron construidos y verificados (F4.1 a F4.6); el detalle de cada uno está abajo.

- **F4.6 ADR-0012 · zona horaria por negocio: completado.** `tenants.timezone` en IAM (migración `V5`), claim `tenant_timezone` en el token, cabecera `x-tenant-timezone` propagada por el gateway (reescrita siempre con el valor verificado, nunca la del cliente) y aplicada por el ERP con validación contra la base de zonas (400 `INVALID_TIMEZONE` si es desconocida) y respaldo configurado. Evidencia: IAM 35 pruebas (claim), gateway 8/8 (cabecera), ERP 5/5 de integración (precedencia), humo **109/109** (zona del negocio, zona distinta honrada y zona inválida rechazada).
- **F4.1 Vertical Packs (P-17): en curso.** El vertical es un dato del negocio (ADR-0013), no una bifurcación del producto:
- **Catálogo** en el ERP (`KuboErp.Packs`): cuatro verticales —retail, servicios, restaurantes y agro— con terminología, IVA por defecto, si lleva inventario y flujo de punto de venta; `GET /packs` y `GET /packs/current`; un vertical desconocido cae al paquete por defecto con aviso (solo afecta etiquetas, no datos).
- **Negocio**: `tenants.vertical` en IAM (migración `V6`), claim `tenant_vertical`, cabecera `x-tenant-vertical` propagada por el gateway y

`GET/PATCH /tenants/me` (solo propietario o administrador) para activar un paquete; el cambio aplica al refrescar la sesión, que la propia interfaz dispara al guardar.

- **PWA**: contexto `usePack` que adapta la terminología (una tienda ve «Productos» y un restaurante «Platillos») y pantalla **Configuración** con el selector de vertical y la zona horaria.

- **Evidencia**: IAM **36 pruebas** (claim y validación `INVALID_VERTICAL`), gateway **9/9** (cabecera y rutas), ERP puras **16/16** y 5/5 de integración, humo **114/114** (catálogo, demo en retail, vertical inválido rechazado y cambio reflejado en el token nuevo), E2E **4/4** con la pantalla nueva auditada.

- **Incremento aplicado**: el paquete ya cambia el comportamiento, no solo las etiquetas. `products.tracks_stock` (migración `20260923000010`): un servicio se vende sin existencias, no mueve kardex al vender y no lo devuelve al anular; el POS no bloquea la venta ni avisa de stock, y el formulario de productos lo propone según el paquete. `sales.table_number`: el flujo `pos_flow: "table"` muestra el campo de mesa en el POS, lo guarda en la venta y lo imprime en el comprobante. Evidencia: ERP integración **6/6** (servicio sin kardex y venta con mesa), humo **117/117**, E2E 4/4.

- **Siembra del catálogo de arranque**: `POST /packs/apply` crea los productos de arranque del paquete activo (tres por vertical, con la terminología, el IVA y la bandera de inventario del paquete) y es **idempotente**: los SKU existentes se omiten, así que aplicarlo dos veces no duplica nada. La pantalla de Configuración lo ofrece con un botón y el resultado se informa al usuario. Evidencia: ERP integración **7/7**, humo **120/120** (siembra, no duplica y omite existentes).

- **Criterio de aceptación de F4.1 cubierto**: dos verticales activables sin tocar el núcleo (probado con `retail` y `restaurantes` en el humo), con terminología, valores por defecto y comportamiento distintos.
- **F4.2 Facturación electrónica DIAN (P-18): puerto implementado.** El núcleo no conoce al proveedor: `KuboErp.Billing` es el contrato y el adaptador `Sandbox` genera UBL 2.1 con el CUFE del algoritmo DIAN (SHA-384, 96 hexadecimales), el QR del catálogo y el escapado XML. La factura se persiste inmutable (`invoices`, con RLS), emitir es idempotente, una venta anulada no se factura (409 `SALE_VOIDED`) y el nombre del emisor llega por `x-tenant-name`. Evidencia: ERP puras **32/32** y integración **16/16**, humo **192/192**, contratos **16/16** (esquema `InvoiceItem` contra la API viva). **Notas crédito: implementadas** (anular una venta facturada emite el documento que corrige la factura, con su CUDE y su XML como documento; es idempotente por venta). Pendiente externo: la firma XAdES —del lado del proveedor tecnológico, que es quien firma y envía— y el registro del NIT y la clave técnica ante la DIAN (trámite de habilitación).
- **F4.5 Segundo factor TOTP del propietario (P-30): completado.** El secreto se cifra en reposo (AES-256-GCM con `KUBO_TOTP_ENCRYPTION_KEY`; sin llave el servicio no arranca), la implementación es propia y verificada contra los vectores del RFC 6238, y el acceso se parte en dos: `login` responde un desafío firmado con `typ=totp` (el gateway solo acepta `typ=access`) y `POST /auth/totp/verify` lo cambia por la sesión. `setup` deja el secreto pendiente, `enable` lo confirma con el primer código y `disable` exige un código vigente; todo queda auditado. La PWA pide el código en el ingreso y la pantalla de Configuración permite configurarlo. Evidencia: IAM **46 pruebas** (vectores RFC, cifrado, desafío y ciclo completo), gateway **9/9**, humo

130/130 con un código real calculado en Python (implementación independiente), E2E 4/4, `make ci` 10/10.

- **F4.3 Notificaciones (P-19): corte implementado.** El aviso es un **puerto** (`KuboErp.Notifications`, ADR-0017): el núcleo dice qué pasó y el adaptador decide cómo se entrega; el de por defecto escribe en el **buzón del negocio** (`notifications`, con RLS), que sirve de demostración, auditoría y base para un proveedor real. La notificación viaja en la transacción que la provoca y el aviso de stock bajo usa histéresis (avisa al **cruzar** el mínimo, no en cada venta). Evidencia: humo **139/139** con el aviso `LOW_STOCK` y las unidades restantes. Pendiente del paso: adaptador real de WhatsApp/SMTP con entrega en segundo plano, buzón en la PWA, notificaciones de compra/resumen y el servicio de Documentos (P-25).
- **F4.4 Multi-bodega y transferencias (P-22): modelo implementado.** El stock por bodega es real (`warehouses`, `stock_levels` como fuente de verdad, `products.stock` como total denormalizado mantenido en la misma transacción), el kardex gana `warehouse_id` con índice de idempotencia por bodega y la transferencia es un **par de movimientos atómicos** con las reglas del ADR-0016 (origen ≠ destino, cantidades positivas, producto con inventario, existencia suficiente). La migración hace el backfill de bodega, niveles y kardex histórico **antes** de activar RLS, y crea la bodega por defecto de cada negocio (las nuevas se crean de forma perezosa). Evidencia: ERP integración **10/10** con PostgreSQL real, **humo 137/137** (transferencia, total invariante, kardex en ambas bodegas, 409 por existencia y 409 al borrar la bodega por defecto) y **contratos 18/18** (`WarehouseList`, `TransferList`).
El backfill de la migración resultó invisible para RLS (las tablas ya tenían `FORCE`): se suspende `FORCE` durante el backfill y se restaura al terminar,

con backfill idempotente. **Completado con la PWA de bodegas/transferencias y la elección de bodega en el POS**: la venta despacha desde la bodega elegida (el ERP la valida y descuenta su nivel; una bodega ajena responde 404), con humo **146/146** e integración **12/12**.

Fase 5 — Escala (backlog declarado)

P-29 · rotación de claves: completado (ADR-0019). Envelope encryption con anillo de llaves y formato versionado (`v1:<key_id>:…`): la llave nueva entra al frente, las viejas siguen descifrando y la rotación es una tarea de sistema por lotes, idempotente y que **nunca destruye un dato ilegible** (lo cuenta). El índice ciego se recalcula en el mismo guardado. Evidencia: CRM puras **12/12** e integración **3/3** con PostgreSQL real. Operación: `rake kubo:rotate_field_keys` (agregar la llave nueva a `KUBO_FIELD_ENCRYPTION_KEYS`, correr la tarea, retirar las viejas cuando el resumen diga 0 rotadas).

P-28 · mTLS entre gateway y servicios: completado (ADR-0020). Una CA interna propia (`make certs`, no versionada) firma un certificado por servicio; cada servicio exige el certificado del cliente (`client-auth: need`, `verify_mode: peer`, `fail_if_no_peer_cert`) y el gateway es el único cliente de la malla (agente de `undici` con CA y certificado, compartido por el proxy, el BFF de autenticación, el tablero y la descarga del JWKS). Los SAN incluyen `localhost` para que los healthchecks también ejerzan la verificación de nombre. Evidencia: humo **150/150** (incluye la comprobación negativa: sin certificado no hay handshake), `make ci` **10/10**. Un contenedor añadido a la red ya no puede suplantar identidad con cabeceras.

Instalación remota: playbook de Ansible (`kubo-infra/ansible/`). Deja el sistema corriendo en un servidor limpio con **secretos únicos generados en el host** (bases, RabbitMQ, firma, TOTP y contraseña inicial), la malla cifrada

creada allí y el cortafuegos abriendo solo la PWA y el borde TLS; Docker queda habilitado, de modo que los servicios (`restart: unless-stopped`) vuelven solos tras un reinicio. Solo necesita `ansible-core` (sin colecciones) y se verifica con `--syntax-check`. La operación —respaldar, actualizar, rotar la CA (`make rotate-ca`)— queda en el README del playbook.

Multi-tenant SaaS: ADR-0021 y primer corte implementado. El SaaS es un **modo de despliegue** del mismo producto (no un fork). El plan es un dato del negocio que viaja en el token (`tenant_plan`) y **cada servicio aplica los cupos de los recursos que posee**: IAM limita los usuarios activos del plan (5 en `community`, 25 en `pro`; los deshabilitados no ocupan asiento) y el ERP limita las bodegas (2 y 10). Un negocio **suspendido** no inicia sesión (`TENANT_SUSPENDED`) pero conserva sus datos intactos y exportables, y hay catálogo de planes (`GET /tenants/plans`). Evidencia: IAM **48 pruebas** (cupo y suspensión), ERP puras con `Plans`, humo **159/159** (cupo de usuarios en el negocio aislado, cupo de bodegas en la demo y el **ciclo de suspensión**: suspender con la herramienta del operador → el ingreso responde `TENANT_SUSPENDED` → reactivar → vuelve a entrar). **Superficie del operador (ADR-0024)**: `kubo-infra/scripts/tenant-admin.sh` (`list`, `suspend`, `activate` por slug o correo) sobre la base de identidad —sin superficie HTTP ni rol omnipotente—; el panel con rol de plataforma llegó en F6.4 (ADR-0025) con ámbito separado, segundo factor obligatorio y auditoría propia, y el uso agregado y el cobro por webhook cerraron en F6.6 (ADR-0026).

Operador de respaldos: ADR-0022 e implementado. Contenedor `kubo-backup` (perfil `backup`) que corre el ciclo **respaldo → retención → verificación → manifiesto** sin socket de Docker: habla con Postgres y Mongo por la red del compose, **incluye el volumen de documentos** (el hueco real: los XML de facturas

y comprobantes no estaban en el respaldo), **restaura en bases de usar y tirar y compara conteos** con el origen, y deja un manifiesto con el resultado y el **SHA-256** de cada archivo. Retención por días conservando un mínimo, copia fuera del sitio opcional y `make backup-operator` / `make backup-operator-loop`. Evidencia: ciclo real con `verificacion=ok` y conteos coincidentes (`users 185/185`, `customers 118/118`, `products 276/276`, `sales 310/310`).

App móvil: ADR-0023 (decisión). La PWA es la interfaz mientras el navegador alcance; cuando el negocio pida cámara/código de barras, push, biometría o impresión térmica, se **envuelve con Capacitor** en lugar de reescribir, con el mismo API y contrato. No se escribe nativo hoy.

| ID | Línea de trabajo | | --- | --- | | P-28 | mTLS entre gateway y servicios — **completado** | | — | Instalación remota (Ansible) — **completado** | | — | Multi-tenant SaaS (ADR-0021) — **primer corte implementado** | | — | Operador de respaldos (ADR-0022) — **completado** | | — | App móvil (ADR-0023) — **decisión tomada; sin trabajo hasta que el negocio lo pida** |

Estado: completada salvo el backlog comercial (2026-09-24). Todo lo que la Fase 5 declaró quedó construido y verificado: mTLS interno (ADR-0020), rotación de claves (ADR-0019), instalación remota con Ansible, primer corte de multi-tenant SaaS (ADR-0021), operador de respaldos con verificación de restauración (ADR-0022) y superficie del operador (ADR-0024). Lo que resta no es deuda técnica: es el **ciclo comercial** (medir uso, cobrar, dar soporte), que se ordena en la Fase 6.

Fase 6 — Operación y comercial (continuo)

La Fase 5 dejó el producto listo para operarse; la Fase 6 es **operarlo**: saber (F6.1–F6.6 completados; las métricas de F6.5 y las mejoras de operación siguen) cuánto usa cada negocio, cobrar, atender y decidir con datos. No tiene fecha de cierre porque es el trabajo vivo del producto.

| Paso | Pendiente | Esfuerzo | Estado | | --- | --- | --- | --- | | F6.1 | Medición de uso por negocio (usuarios, bodegas, productos, ventas del mes, documentos) | 2–3 d | **Completado** | | F6.2 | Cobro: renovación del plan (fecha) y registro manual del pago en la herramienta del operador | 1–2 d | **Completado** | | F6.3 | Adaptador de objeto para el destino de los respaldos (hoy disco + copia fuera del sitio) | 1 d | **Completado** | | F6.4 | Panel del operador (listar, suspender, renovar) — requiere el rol de plataforma (ADR-0025) | 3–5 d | **Completado** | | F6.5 | Métricas de producto y soporte (uso agregado, errores por negocio) | continuo | **Completado** | | F6.6 | Pasarela de pago automática como adaptador del puerto de cobro (ADR-0026) | 3–5 d | **Completado** |

- **F6.3 Copia fuera del sitio como puerto**: el operador usa **rclone**, que acepta tanto una ruta montada (disco, NFS) como un remoto configurado (S3, B2, Drive) con el mismo código, y **verifica la copia** (`rclone check`) antes de declararla: el manifiesto dice `fuera_del_sitio=ok|fallida|no_configurada` y una copia fallida termina el ciclo con error. Evidencia: ciclo real con `fuera_del_sitio=ok`.
- **Uso del ERP en el panel y rotación del segundo factor (cerrados, ADR-0025)**: el panel muestra los conteos por negocio (productos, bodegas, ventas del mes, documentos) uniendo IAM con el endpoint interno del ERP —sin ampliar RLS y sin leer datos de negocio— y el operador puede rotar su segundo factor con la URI entregada una sola vez (más el camino de emergencia documentado). Evidencia: humo **177/177**, contratos **23/23**, `make ci` **10/10** (IAM **69 pruebas**, gateway con 3 pruebas de la vista compuesta).
- **F6.5 Métricas de operación y soporte**: `tenant-admin.sh usage` lista, por negocio, plan, estado, usuarios activos, bodegas, productos, ventas del mes, documentos (KB) y **accesos fallidos de los últimos 7 días** (el primer síntoma de un problema de soporte). Solo cuenta filas: nunca lee datos de negocio. Evidencia: salida real con la demo y los negocios de prueba (el negocio del bloqueo muestra sus 5 accesos fallidos).
- **F6.4 Panel del operador (ADR-0025): completado.** Reino de plataforma

separado del negocio: tabla `platform_admins` (nunca un usuario de tenant), **segundo factor obligatorio** (el acceso siempre pide el código), token con `platform: true` que el gateway exige para `/platform/*` y **rechaza** en el API del negocio (y al revés), poder mínimo (listar, suspender, reactivar y renovar —jamás leer ventas, clientes ni documentos—), **auditoría propia** en `platform_audit` y el panel en `/plataforma` con su propio acceso. Evidencia: IAM **54 pruebas** (incluido el flujo positivo con el secreto bajo control), humo **167/167** (el acceso exige el código, un código inválido no abre sesión y un token de negocio responde 403 en el panel), E2E **5/5** con la página auditada.

- **F6.6 Puerto de cobro (ADR-0026): completado.** Los pagos son datos (`payment_intents` + `plan_prices`, migración V11): el negocio pide pagar su plan desde Configuración —el monto sale del catálogo, nunca del cliente—, el operador lo registra desde el panel (pagos pendientes con el nombre del negocio) y un proveedor real confirma por **webhook firmado** (HMAC-SHA256 del cuerpo crudo) que es la fuente de verdad. La renovación se deriva de un pago pagado y es **idempotente por referencia del proveedor**: un reintento no extiende dos veces el plan (índice único `(proveedor, referencia)`); un monto distinto al de la intención se rechaza y la suspensión sigue siendo humana. Evidencia: IAM **61 pruebas** (7 del puerto, con idempotencia y firma), humo **176/176** (precio del catálogo, webhook sin firma → 401, webhook firmado confirma y extiende, reintento no extiende, monto distinto → 422), contratos **23/23** (`PriceList`, `PaymentItem`).

Criterio de aceptación de F6.1–F6.2: cumplido.

- **F6.1 Uso**: el ERP expone `GET /usage` (bodegas, productos, ventas del mes en la zona del negocio y documentos con cantidad y bytes) y IAM aporta los

usuarios activos y los cupos en `/tenants/me`. La pantalla de Configuración muestra el uso contra el plan («Usuarios activos 2 de 5», «Bodegas 2 de 2», ventas del mes, documentos). Evidencia: contratos **21/21** (`UsageItem`) y humo **164/164** (uso reportado y usuarios activos en el perfil).

- **F6.2 Cobro manual**: `tenants.plan_renews_at` (migración `V9`) y `tenant-admin.sh renew <slug|correo> [días]`, que extiende desde hoy o desde la renovación vigente —**la que sea mayor**: pagar antes de vencer no regala días—; el listado del operador muestra la fecha y la PWA la enseña al negocio. El corte del servicio sigue siendo explícito (suspender), nunca automático por fecha. Evidencia: humo **164/164** (la renovación queda registrada en la fecha esperada).

4. Camino crítico y calendario

```
flowchart LR
  F0["Fase 0<br/>2-3 h"] --> F1["Fase 1 · Confiabilidad<br/>3 semanas"]
  F1 --> F2["Fase 2 · Calidad<br/>4 semanas"]
  F2 --> F3["Fase 3 · Núcleo comercial<br/>5-7 semanas"]
  F3 --> F4["Fase 4 · Diferenciadores<br/>5-6 semanas"]
  F4 --> F5["Fase 5 · Escala<br/>backlog declarado"]
```

| Hito | Duración | Semanas acumuladas | Estado | | --- | --- | --- | --- | | Fase 0 | 2–3 h | día 1 | Completada | | Fase 1 | ≈3 semanas | 1–3 | Completada | | Fase 2 | ≈4 semanas | 4–7 | Completada | | Fase 3 | 5–7 semanas | 8–14 | Completada | | Fase 4 | 5–6 semanas | 15–20 | Completada | | Fase 5 | backlog | — | Completada salvo el backlog comercial | | Fase 6 | continuo | — | **Completada** (F6.1–F6.6; las métricas siguen) | | **Producto comercializable** | **≈17–20 semanas** | **~4–5 meses a 30 h/semana** | **Alcanzado** |

Las fases 0 a 6 están completadas: el producto es operable y comercializable en la región. Lo que resta es trabajo continuo (métricas de producto, soporte, habilitación DIAN y la app móvil cuando el negocio la pida).

5. Definición de «proyecto terminado»

1. Un negocio real opera una semana completa sin soporte presencial.

2. Cero pérdida de datos: caída del bus, del proceso o del equipo → nada se

pierde (outbox + respaldos probados).

3. Aislamiento garantizado por el motor, no por el programador (RLS activo).

4. Una regresión no llega a `main`: CI con gates de seguridad y de contrato.

5. Los 5 módulos que el negocio pide (compras, caja, usuarios, comprobantes,

reportes) funcionando en la PWA.

6. Documentación viva: cada decisión con su ADR, cada requisito con su prueba.

6. Riesgos y dependencias

| Riesgo | Mitigación | | --- | --- | | Publicación de los repos | **Resuelta**: los 9 repos están publicados en GitHub (públicos, cuenta NFGS, con licencia MIT y CI en Actions, ADR-0029) y `make push-github` republica con `gh` autenticado | | Las pruebas del ERP no corren en el contenedor de producción (OOM con 512 MB) | Comando con `-m 3g` documentado en `07-pruebas.md`; el CI de la Fase 2 las ejecuta | | DIAN exige ser Proveedor Tecnológico autorizado | Trámite externo; la Fase 4 avanza con el puerto y deja la habilitación como cierre | | Capturas internas de la app y video demo pendientes | Guía en `evidencia/README.md` y guion en `09-demo-guion.md`; tarea del usuario | | La Fase 3 no debe empezar sin la Fase 1 | Orden por riesgo: no se construye producto sobre una entrega de eventos no garantizada |

7. Runbook de operación

La semana uno —el criterio de «proyecto terminado»— tiene su runbook en

[`12-runbook-operacion.md`](12-runbook-operacion.md): el chequeo diario

(`kubo-infra/scripts/operacion-check.sh`), los playbooks (bus caído, respaldo

sin verificación, disco, certificados, negocio bloqueado, autenticador del

operador perdido, pago que no cuadra, sospecha de fuga) y las métricas que se

vigilan. **Notas crédito**: implementadas (ADR-0014); el XML del puerto queda

sin firmar a propósito, porque XAdES lo firma el proveedor tecnológico.

8. Catálogo de pendientes

El catálogo completo con IDs, riesgo y esfuerzo vive en

[`10-auditoria.md` §5](10-auditoria.md); este documento lo ordena por fase y fija

el criterio de cierre de cada una.

Runbook de operación — la semana uno

> **Para quién**: quien opera Kubo en el servidor del negocio (el operador).

> **Para qué**: que la primera semana real —el criterio de «proyecto

> terminado»— transcurra sin soporte presencial. Cada procedimiento dice qué se

> mira, qué significa y qué se hace.

1. Día cero (puesta en marcha)

1. **Instalar** con el playbook de `kubo-infra/ansible/` (`README.md` del

directorio): genera los secretos en el host, levanta el stack y deja UFW

cerrado salvo 80/443/22.

2. **Certificados**: `make certs` (los internos de la malla, ADR-0020) y el

certificado público del dominio en Caddy.

3. **Negocio y dueño**: el seeder crea el negocio y el propietario

(`KUBO_ADMIN_EMAIL`/`KUBO_ADMIN_PASSWORD`). El dueño **activa su segundo

factor** en Configuración el primer día (sin él, el acceso queda en un solo

factor).

4. **Respaldo**: `make backup-operator-loop` (o los timers de

`kubo-infra/systemd/`) deja el ciclo respaldo → retención → verificación →

manifiesto corriendo. **No se abre al público sin un respaldo verificado**.

5. **Prueba de restauración** antes de abrir: `make restore-drill` restaura en

bases de usar y tirar y compara conteos.

6. **Plan**: `./kubo-infra/scripts/tenant-admin.sh renew <slug>` deja la

vigencia; el panel del operador (`/plataforma`) muestra plan, estado y uso.

2. Cada mañana (5 minutos)

```
kubo-infra/scripts/operacion-check.sh
```

Debe terminar en **0 fallos**. Qué mira y qué hacer si falla:

| Chequeo | Si falla | | --- | --- | | El gateway responde su sonda | `docker compose ps`; si el contenedor no está, `docker compose up -d kubo-gateway` | | Los servicios del núcleo corren | Levantar el que falte y mirar su log (`docker logs kubo-<servicio> --since 30m`) | | La bandeja de salida sin eventos fallidos | Ver §4.1 (bus) | | El respaldo de anoche pasó la verificación | Ver §4.2 (respaldo) | | La copia fuera del sitio quedó hecha | Revisar el destino de rclone y su credencial | | Disco por debajo del 85 % | Liberar respaldos viejos o ampliar el volumen (§4.3) | | Certificados vigentes (>15 días) | `make rotate-ca` y reiniciar (§4.4) |

3. Cada semana

- **Restauración de prueba**: `make restore-drill` (o el respaldo ya la corre

cada noche; el drill es la prueba completa).

- **Seguridad**: `./kubo-infra/scripts/tenant-admin.sh usage` muestra, por

negocio, accesos fallidos de los últimos 7 días. Un pico sostenido se

investiga con el registro de auditoría (`/auditoria` en la PWA).

- **Comercial**: revisar en `/plataforma` los planes que vencen y los pagos

pendientes de confirmar.

- **Capacidad**: uso de disco, tamaño de los documentos y ventas del mes por

negocio (el panel muestra los conteos; nunca datos de negocio).

- **Espejos**: `make sync-status` verifica la huella de los 4 entornos y

`make sync-check` detecta deriva por ediciones manuales; `make sync` propaga

el canónico, `make sync-pull` importa cambios de los espejos y `make sync-auto`

corre el ciclo completo (con staging de conflictos en `.sync/conflictos/`).

El ciclo se ejecuta solo cada 15 minutos (`kubo-sync.timer`, ADR-0030).

4. Playbooks (qué hacer cuando pasa)

4.1 El bus de eventos se cae

La bandeja de salida (`outbox`) **retiene** los eventos: no se pierde nada.

1. `docker compose ps kubo-rabbitmq`; levantarlo si está abajo.

2. Al volver, el publicador reintenta y la sonda del ERP deja

`outbox.failed = 0`.

3. Si `failed > 0`, revisar el log del ERP y republicar; los eventos están en la

tabla `outbox_events` con su carga completa.

4.2 Un respaldo no pasó la verificación

1. **No borrar** el respaldo anterior: la retención guarda los últimos

(`retencion_minima=3`).

2. Mirar el `MANIFEST` del día (`verificacion=`, `conteos=`): dice qué base no

cuadró.

3. Correr un respaldo manual: `docker compose run --rm kubo-backup`.

4. Si vuelve a fallar, restaurar el respaldo bueno anterior y avisar antes de

seguir operando.

4.3 El disco se llena

1. `du -sh kubo-infra/backups/*` — los respaldos son lo que más crece.

2. La retención (`retencion_dias=14`, `retencion_minima=3`) ya borra lo viejo;

para una urgencia, mover respaldos viejos al destino fuera del sitio y

borrarlos del servidor.

3. Los documentos del negocio **no se borran nunca** sin orden del dueño.

4.4 Un certificado interno está por vencer

`make rotate-ca` regenera la CA y los certificados de la malla y reinicia los

servicios; después, `operacion-check.sh` debe volver a pasar. La malla rechaza a

quien no presente certificado: un certificado vencido se ve como servicios que

no se hablan.

4.5 Un negocio no puede entrar

1. `./kubo-infra/scripts/tenant-admin.sh list`: ¿está suspendido?

→ `activate <slug>`.

2. Si dice `PLAN_LIMIT_REACHED`: el plan limita usuarios activos

→ desactivar usuarios o renovar/ampliar el plan.

3. Si el acceso falla con `TENANT_SUSPENDED`, es el estado del negocio, no la

clave.

4.6 El operador perdió el autenticador (o quedó bloqueado)

Si solo está **bloqueado** por intentos fallidos (el autenticador funciona, pero

la cuenta acumuló 5 códigos errados), se limpia el contador sin tocar el segundo

factor:

```
./kubo-infra/scripts/tenant-admin.sh platform-reset
```

Si de verdad **perdió el autenticador**, camino de emergencia (ADR-0025): borrar

su fila de `platform_admins` y reiniciar IAM; el seeder recrea el operador y

registra la URI `otpauth` una vez en el log:

```
docker exec kubo-postgres psql -U kubo_root -d kubo_iam -c "delete from platform_admins w
docker compose restart kubo-iam
docker logs kubo-iam --since 2m | grep otpauth
```

4.7 Un pago no cuadra

1. `/plataforma` → **Pagos por confirmar**: cada intención trae negocio, monto y

la referencia del proveedor.

2. La referencia es única por proveedor: si el proveedor reintentó el webhook,

no se extendió dos veces (idempotencia); el pago aparece una sola vez.

3. Si el monto no coincide, el webhook se rechaza (`AMOUNT_MISMATCH`) y la

intención queda pendiente: revisar con el proveedor antes de confirmar a

mano.

4.8 Sospecha de fuga entre negocios

1. **Suspender** el negocio sospechoso (`tenant-admin.sh suspend`) y preservar los registros.

2. Verificar el aislamiento: las pruebas de RLS (`make ci`, suites de integración de ERP/CRM/IAM) y el humo (§8 de `07-pruebas.md`).

3. La auditoría de IAM registra quién hizo qué y cuándo; no se borra.

5. Qué se vigila (métricas de la semana)

- **Disponibilidad**: sonda de salud del gateway (cada mañana y con el monitoreo externo si lo hay).

- **Accesos fallidos por negocio**: `tenant-admin.sh usage` (7 días).

- **Bandeja de salida**: `outbox.failed = 0`.

- **Respaldos**: verificación `ok` y copia fuera del sitio `ok`, cada día.

- **Disco y certificados**: el chequeo de la mañana.

- **Latencia**: panel de observabilidad (`make observability`, Grafana en `localhost:3001`) con las trazas OTel de los cinco servicios.

6. Criterio de salida de la semana

La semana uno se cierra bien cuando:

1. El negocio operó **sin soporte presencial** (se atendió por teléfono como máximo).

2. **Cero pérdida de datos**: ningún respaldo con verificación fallida y ningún evento perdido en el bus.

3. El **aislamiento** se mantuvo (sin hallazgos de fuga).

4. Hubo al menos **una restauración de prueba** exitosa.

5. Los incidentes de la semana quedaron escritos (qué pasó, qué se hizo, qué se cambió) — insumo directo del catálogo de mejoras.

13 — Guía para enchufar un proveedor tecnológico (DIAN)

Esta guía deja el proyecto **listo para conectar el proveedor tecnológico (PT)**

que el negocio elija, sin tocar el núcleo. Todo lo que hay que hacer es escribir

un adaptador y configurar variables de entorno.

1. Cómo está armado (recordatorio)

```
Venta (ERP) ──▶ KuboErp.Billing (puerto) ──▶ Adaptador configurado
                                              ├── Sandbox (por defecto, sin validez fis
                                              └── KuboErp.Billing.<Proveedor> (el que s
```

- **Puerto**: `kubo-erp/lib/kubo_erp/billing.ex` (contrato).

- **Error tipado**: `kubo-erp/lib/kubo_erp/billing/error.ex`.

- **Adaptador de desarrollo**: `kubo-erp/lib/kubo_erp/billing/sandbox.ex`.

- **Persistencia**: `invoices` (inmutable, con `status`, `provider_reference` y

`status_detail`), XML como documento y nota crédito con CUDE.

2. Configuración (ya cableada)

| Variable | Valores | Uso | | --- | --- | --- | | `KUBO_BILLING_ADAPTER` | módulo, por defecto `KuboErp.Billing.Sandbox` | Qué adaptador se usa | | `KUBO_BILLING_ENVIRONMENT` | `1` producción, `2` habilitación (por defecto `2`) | Ambiente DIAN |

El ERP valida al arrancar que el módulo implemente el puerto: si no, falla con

un error claro en vez de emitir documentos inválidos. El estado activo se ve en

`GET /api/v1/health` → `billing: { adapter, environment }`.

3. Datos fiscales del emisor (ya existen)

El negocio los registra una vez en **Configuración → Datos fiscales** (o por

`PATCH /api/v1/tenants/me`): NIT (el DV lo calcula el servidor), dirección,

régimen (`RESPONSABLE_IVA`, `NO_RESPONSABLE_IVA`, `SIMPLE`), resolución y

prefijo. Viajan en el **token** y el gateway los propaga como cabeceras verificadas (`x-tenant-tax-id`, `x-tenant-tax-id-dv`, `x-tenant-fiscal-address`, `x-tenant-tax-regime`, `x-tenant-invoice-resolution`, `x-tenant-invoice-prefix`). El adaptador los recibe en el mapa `tenant`; **no** debe consultar IAM.

4. Paso a paso para escribir el adaptador

1. **Crea el módulo** `kubo-erp/lib/kubo_erp/billing/<proveedor>.ex`:

```elixir
defmodule KuboErp.Billing.MiProveedor do
  @behaviour KuboErp.Billing

  alias KuboErp.Billing.Error

  @impl true
  def issue(tenant, sale) do
    with {:ok, credenciales} <- credenciales(),
         {:ok, respuesta} <- enviar("/invoices", payload(tenant, sale), credenciales) do
      {:ok,
       %{
         number: respuesta["number"],
         cufe: respuesta["cufe"],
         qr_url: respuesta["qr_url"],
         xml: respuesta["xml"],
         status: respuesta["status"] || "ISSUED",
         provider_reference: respuesta["id"]
       }}
    else
      {:error, %Error{} = error} -> {:error, error}
      {:error, razon} -> {:error, Error.new(:network, "El proveedor no respondio", inspec
    end
  end

  @impl true
  def issue_credit_note(tenant, sale, invoice, reason) do
    # Igual que issue/2 pero contra el endpoint de notas credito; devuelve cude.
    # ...
  end

  # Opcional: solo si el proveedor valida de forma asincrona.
  # @impl true
  # def refresh_status(invoice), do: ...

  defp credenciales do
    case {System.get_env("KUBO_PT_API_KEY"), System.get_env("KUBO_PT_BASE_URL")} do
      {nil, _} -> {:error, Error.new(:not_configured, "Falta KUBO_PT_API_KEY")}
      {_, nil} -> {:error, Error.new(:not_configured, "Falta KUBO_PT_BASE_URL")}
      {api_key, base} -> {:ok, %{api_key: api_key, base: base}}
    end
  end

  defp enviar(ruta, cuerpo, credenciales) do
    case Req.post("#{credenciales.base}#{ruta}",
           json: cuerpo,
           headers: [{"authorization", "Bearer #{credenciales.api_key}"}],
           receive_timeout: 15_000
         ) do
      {:ok, %{status: 200, body: body}} -> {:ok, body}
      {:ok, %{status: 422, body: body}} -> {:error, Error.new(:rejected, "El proveedor re
```

```
      {:ok, %{status: 401}} -> {:error, Error.new(:not_configured, "Credenciales del prov
      {:ok, %{status: status}} -> {:error, Error.new(:network, "Respuesta inesperada del
      {:error, razon} -> {:error, Error.new(:network, "No fue posible contactar al provee
    end
  end

  defp payload(tenant, sale) do
    %{
      # Los datos fiscales del emisor ya vienen verificados en `tenant`.
      issuer: %{nit: tenant.tax_id, dv: tenant.tax_id_dv, name: tenant.name, address: ten
      reference: sale.id,
      customer: %{name: sale.customer_name},
      payment_method: sale.payment_method,
      subtotal: sale.subtotal,
      tax: sale.tax,
      total: sale.total,
      items: Enum.map(sale.items, &%{name: &1.product_name, quantity: &1.quantity, unit_p
    }
  end
end
```

2. **Registra el adaptador** en `kubo-infra/.env` (y en el despliegue):

```bash

KUBO_BILLING_ADAPTER=KuboErp.Billing.MiProveedor

KUBO_BILLING_ENVIRONMENT=2 # habilitación; 1 cuando la DIAN lo apruebe

KUBO_PT_API_KEY=... # credenciales del PT, nunca en el repositorio

KUBO_PT_BASE_URL=https://sandbox.miproveedor.co

```

Añade también las variables al compose si son nuevas (patrón:

`KUBO_PT_*: ${KUBO_PT_*:-}`).

3. **Idempotencia**: envía `sale.id` como referencia al PT. Si la red falla y el

ERP reintenta, el PT no debe emitir dos veces (el ERP además ya no regenera:

si la venta tiene factura, devuelve la existente).

4. **Pruebas** con `Req.Test` (no necesitas el PT real):

```elixir

test "emite la factura con el proveedor" do
```

Req.Test.stub(KuboErp.Billing.MiProveedor, fn conn ->

Req.Test.json(conn, %{number: "FE-1", cufe: String.duplicate("a", 96), qr_url: "https://...", xml: "<Invoice/>", status: "ISSUED", id: "pt-1"})

end)

configurar Req con el stub y llamar Billing.issue/2

end

```

El adaptador debe pasar además el contrato común del sandbox (CUFE de 96

hexadecimales, XML UBL 2.1, nota crédito con CUDE) que ya vive en

`test/kubo_erp/billing/sandbox_test.exs`.

5. **Verifica en el ambiente de habilitación**: con `KUBO_BILLING_ENVIRONMENT=2`

emite facturas reales de prueba y revisa que la DIAN las acepte. Cuando todo

esté en verde, cambia a `1`.

## 5. Errores (contrato de la capa web)

| Código del adaptador | HTTP | Significado | | --- | --- | --- | | `:not_configured` | 503 | Faltan credenciales o no hay adaptador real | | `:not_supported` | 501 | El PT no ofrece esa operación (p. ej. estado en línea) | | `:network` | 503 | El PT no respondió; reintentable | | `:rejected` | 422 | El PT o la DIAN rechazaron el documento (trae detalle) | | `:validation` | 422 | Los datos no cumplen el contrato del PT |

## 6. Lo que **no** debe hacer el adaptador

- **No** firmar XAdES ni enviar a la DIAN: eso lo hace el PT (ADR-0014).

- **No** leer bases de otros servicios ni IAM: los datos fiscales llegan en el

token.

- **No** guardar secretos en el repositorio: van en `.env`/compose.

- **No** regenerar una factura existente: es inmutable (cambiarla cambiaría el

CUFE que ya recibió el cliente).

## 7. Checklist externo del negocio (paralelo al desarrollo)
```

1. RUT y responsabilidades al día.

2. Habilitación como facturador electrónico ante la DIAN (entrega la clave técnica y la resolución de facturación).

3. Contrato con el PT y sus credenciales de sandbox/producción.

4. Certificado digital (normalmente lo gestiona el PT).

5. Cargar los datos fiscales en Kubo (NIT, dirección, régimen, resolución, prefijo) y probar una factura de habilitación.

8. Estado actual (sandbox)

Con el adaptador por defecto, el humo comprueba la cadena completa: datos fiscales con DV calculado, cabeceras verificadas del gateway, NIT con DV en el XML, prefijo del negocio y estado `ISSUED`. Evidencia: `make smoke` **192/192**. La emisión es **de un solo emisor por venta**: factura y nota crédito toman un candado por venta (`pg_advisory_xact_lock`) antes de llamar al PT, así que un doble clic o dos reintentos simultáneos no emiten dos documentos ni cobran dos veces. La idempotencia por venta ya existía; el candado cierra la carrera entre peticiones concurrentes.

14 — Plan de la app móvil (Capacitor)

| Campo | Valor | | --- | --- | | Estado | **En ejecución** — activado por decisión del propietario (2026-10-07): vitrina móvil + preparación para clientes. Fase 0 completada; Fase 1 (envoltura Android) en curso | | Decisión marco | [ADR-0023](adr/ADR-0023-app-movil.md): la PWA primero; Capacitor antes que reescribir | | Disparadores | Cámara/código de barras · Notificaciones push · Biometría para el 2FA · Impresión térmica Bluetooth | | Esfuerzo estimado | ~20–30 días hábiles de desarrollo + 1 semana de piloto (por tramos; cada fase deja valor) | | Costo objetivo | $0 (APK directo; sin tiendas por defecto) |

1. Principios (heredados del ADR-0023)

1. **La PWA es la interfaz** mientras el navegador alcance: esto se activa con un caso real, no "porque sí".

2. **Capacitor antes que reescribir**: misma base de código (`kubo-web`), un solo contrato de API; nada de endpoints exclusivos (ADR-0011).

3. **La tienda no es requisito**: para un negocio autoalojado, el APK firmado desde su propio servidor es suficiente; publicar en Play/App Store es una decisión comercial posterior.

4. **Cada capacidad se encapsula**: los plugins viven detrás de interfaces propias para poder cambiarlos sin tocar el núcleo.

5. **iOS se difiere** hasta que un negocio lo pida y exista hardware Apple para firmar; la misma base lo habilitará.

2. Disparadores de activación (criterio de entrada)

#	Disparador	Señal de negocio que lo justifica	Fase
D1	Cámara y código de barras	Cargar inventario o vender escaneando (hoy: búsqueda manual)	2a
D2	Notificaciones push	El dueño no vive dentro del sistema; hoy revisa el buzón a mano	2b
D3	Biometría para el segundo factor	Fricción del TOTP con app externa (ADR-0015) en el día a día	2c
D4	Impresión térmica por Bluetooth	El POS necesita el ticket físico sin depender de impresora de red	2d

> Regla: **sin disparador documentado con un negocio real, el plan no arranca**
> (la Fase 0 exige esa evidencia).

3. Fases

Fase 0 — Activación y alcance (1–2 días)

Objetivo: convertir el disparador en un alcance firmado.

- Documentar el caso real (qué negocio, qué flujo, qué duele hoy).
- Mini-ADR de las decisiones nuevas que aparezcan (p. ej. transporte de push en D2; estrategia de origen/sesión en WebView, §5).
- Modo de empaquetado elegido: **empaquetado** (activos locales + servidor configurable en el primer arranque), decisión en [ADR-0031](adr/ADR-0031-empaquetado-movil.md); el modo remoto queda como plan B si la sesión nativa no resulta viable.

- Priorizar capacidades si hay más de una y fijar el tramo presupuestado.

Criterios de aceptación: caso real escrito · mini-ADR aceptado · alcance y tramo aprobados.

Estado: completada (2026-10-07) — activación por decisión del propietario (vitrina móvil + preparación para clientes); alcance del primer tramo: Fase 1 (envoltura Android).

Fase 1 — Envoltura Capacitor, Android primero (3–4 días)

Objetivo: la PWA corriendo como app Android firmada, sin tocar el contrato.

- Añadir Capacitor al repositorio `kubo-web` (proyecto `android/`; sin bifurcar el código).
- Validar plugins y versiones vigentes al activar (documentación actualizada, p. ej. con `context7`).
- Prueba temprana de sesión en dispositivo: cookie de refresco httpOnly (ADR-0007/0015) y origen del WebView; el ajuste es de configuración, **el contrato no cambia**.
- Build de depuración y **APK firmado de prueba**; keystore creado y respaldado cifrado desde este día (regla: sin keystore no hay actualización).
- Job de CI `android` (GitHub Actions + SDK de Android) que produce el APK por tag.
- Iconos, pantalla de arranque, safe areas y botón atrás.

Criterios: ingreso + POS + cola offline funcionan en el binario; APK instalable en un equipo real; E2E móvil inicial con **Maestro** (OSS) cubriendo ingreso y venta offline; contrato OpenAPI validado igual que hoy (verificación del ADR-0023).

Estado: completada (2026-10-07) — proyecto Android + APK en CI + modo nativo + **suite móvil completa en verde en el emulador (Maestro)**: sesión

(primer arranque, ingreso y restauración por cookie), **venta de POS**, **venta sin conexión a la cola local** y **sincronización automática al reconectar**.

La instalación en un equipo físico queda como verificación opcional.

Fase 2 — Capacidades nativas, por prioridad

2a — Cámara y código de barras (3–4 días)

- Escaneo en alta de producto (inventario) y en el POS (agregar al carrito).
- Permisos con degradación limpia (sin permiso, la app sigue usable).

Criterios: EAN-13/Code128 → carrito en < 2 s; sin red la venta se encola igual; sin permiso no se bloquea la app.

2b — Notificaciones push (3–5 días)

- Extender el **puerto de notificaciones (ADR-0017)** con un canal push detrás de una interfaz; el buzón in-app sigue siendo la fuente de verdad.
- Implementación Android (FCM) y **alternativa OSS (ntfy/UnifiedPush) evaluada** en el mini-ADR de F0 — decisión explícita por el costo de la dependencia.
- Opt-in por usuario; sin tokens en logs; revocación al cerrar sesión del dispositivo.

Criterios: venta sincronizada, stock bajo y factura lista llegan como push; si no hay permiso, todo sigue en el buzón; apagar la capacidad no deja rastro.

2c — Biometría para el segundo factor (2–3 días)

- **Primero un spike de WebAuthn/passkeys**: puede resolver el disparador **sin envoltura** (funciona en la PWA); si no alcanza, plugin biométrico nativo.
- Secreto TOTP (ADR-0015) en Keystore si es nativo; nunca fuera del dispositivo.

Criterios: segundo factor con huella en < 2 s; desactivación limpia; el secreto no sale del almacén seguro.

2d — Impresión térmica Bluetooth (3–5 días)

- ESC/POS por BLE desde el POS (impresoras 58 mm genéricas); perfil de equipos objetivo definido en F0.

Criterios: ticket impreso ≤ 5 s; reconexión tras pausa; "sin impresora" no bloquea la venta (fallback).

Fase 3 — Calidad y verificación móvil (3–4 días)

- Matriz de dispositivos: 2–3 equipos Android de gama baja reales (o emuladores equivalentes); mínimo Android 10.
- E2E móvil (Maestro) en CI con emulador; humo del binario contra el sistema real.
- Accesibilidad (TalkBack), rendimiento de listas largas, tamaño del APK.
- Seguridad móvil: checklist MASVS-lite (almacenamiento, red solo HTTPS, sin secretos en el binario, permisos mínimos).

Criterios: E2E móvil verde en CI · checklist sin hallazgos altos · APK razonable (< ~20 MB).

Fase 4 — Distribución y operación (2–3 días)

- Versionado alineado con el producto (tag `vX.Y.Z`); APK adjunto al release de GitHub.
- Descarga para el negocio: página autenticada del operador o instrucciones del runbook.
- Runbook móvil: instalar, actualizar, perder el equipo (revocar sesión), desinstalar.
- Actualización: APK manual por ahora; evaluar Play (internal testing) solo si un negocio lo pide.

Criterios: un negocio instala el APK desde su servidor en ≤ 10 min siguiendo el runbook; la revocación de sesión del dispositivo funciona.

Fase 5 — Piloto con negocio real y cierre (1 semana calendario)

- Piloto con un negocio (uso real ≥ 5 días); métricas: adopción, ventas desde

móvil, incidencias.

- Retrospectiva y decisión: cerrar, iterar o pausar; actualizar este plan y (si

cambia la decisión) el ADR-0023.

Criterios: piloto documentado con feedback · decisión firmada.

4. Estimación

| Fase | Esfuerzo | Deja valor por sí sola | | --- | --- | --- | | 0 — Activación | 1–2 d | Alcance firmado + mini-ADRs | | 1 — Envoltura | 3–4 d | APK firmado con sesión y offline | | 2a — Cámara | 3–4 d | Inventario y POS por escaneo | | 2b — Push | 3–5 d | Avisos fuera del sistema | | 2c — Biometría | 2–3 d | 2FA sin fricción | | 2d — Impresión | 3–5 d | Ticket físico | | 3 — Calidad | 3–4 d | Confianza de release | | 4 — Distribución | 2–3 d | Instalación por el negocio | | 5 — Piloto | 1 semana | Decisión con evidencia | | **Total** | **~20–30 d hábiles + piloto** | Ejecutable por tramos |

5. Riesgos y mitigaciones

| Riesgo | Mitigación | | --- | --- | | Sesión por cookie en WebView (origen distinto) | Prueba temprana en Fase 1; estrategia de origen elegida en F0; el contrato no cambia | | Fragmentación de WebView/permisos | Matriz de gama baja (F3) + mínimos soportados documentados | | Plugins comunitarios sin mantenimiento | Encapsulados tras interfaces propias + versiones fijadas | | Push = dependencia de terceros | Puerto con interfaz + alternativa OSS evaluada (mini-ADR) | | Pérdida del keystore de firma | Respaldo cifrado desde Fase 1; sin keystore no hay actualización | | Scope creep ("ya que estamos…") | Cada capacidad con disparador y criterio; lo demás, backlog | | iOS | Diferido hasta negocio + hardware Apple (misma base) |

6. Trazabilidad

- **ADR-0023** → disparadores D1–D4 y su verificación (E2E contra el binario).

- **ADR-0011** (BFF) → sin cambios de contrato; validado en F1 y F3.

- **ADR-0015** (TOTP) → Fase 2c · **ADR-0017** (puerto de notificaciones) → Fase 2b.

- **ADR-0007** (JWT/cookies) → prueba de sesión en Fase 1.

- Historias de usuario (INVEST) y criterios Gherkin por capacidad: se redactan

al activar cada tramo (Fase 0).

7. Estado

- **Activación**: 2026-10-07, por decisión explícita del propietario (vitrina
móvil + preparación para los primeros clientes; el primer objetivo de prueba
es la demo pública). Los disparadores D1–D4 siguen ordenando las capacidades
cuando existan negocios reales.
- **Avance**: Fase 0 completada (alcance + ADR-0031) · **Fase 1 completada**
(APK en CI, modo nativo y suite móvil completa en el emulador: sesión, venta
de POS, cola offline y sincronización). Hallazgos de la suite: el túnel
público gratuito a veces responde 502 en el primer cobro (la app conserva el
carrito y el flujo reintenta); backlog declarado: clave de idempotencia en
las ventas del POS para que un reintento tras un fallo ambiguo no pueda
duplicar la venta.

ADR-0001 — Microservicios con monolito modular interno

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada | | Decisores | Arquitectura Kubo |

Contexto

Kubo debe cubrir cuatro dominios bien distintos (identidad, clientes, operación
comercial e indicadores) y, al mismo tiempo, poder instalarse en el local de una
PYME con recursos limitados (2 vCPU / 4 GB de RAM). El equipo es pequeño y el
tiempo de entrega, corto.

Decisión

Se despliegan **cinco servicios independientes** (identidad, CRM, ERP,
analítica, documentos) más un API Gateway, cada uno con su propia base de datos.
Dentro de cada servicio no se subdivide en más servicios: se aplica

arquitectura limpia en capas (`domain`, `application`, `infrastructure`, `presentation`) y los subdominios conviven como módulos del mismo despliegue.

Alternativas consideradas

1. **Monolito único**: más simple de operar, pero incumple el requisito de microservicios y no demuestra aislamiento por dominio.
2. **Microservicios finos** (catálogo, inventario y ventas por separado): el costo de coordinación (transacciones, consistencia, red) supera el beneficio en un equipo de una persona.
3. **Microservicios con monolito modular interno** (elegida): cada servicio puede partirse más adelante sin reescribir la lógica, porque las fronteras entre módulos ya están trazadas.

Consecuencias

- **Positivas**: se cumple el requisito, el consumo de memoria es acotado (~900 MB para todo el sistema), y el dominio queda listo para escalar horizontalmente el servicio que lo necesite.
- **Negativas**: no hay transacciones distribuidas; la consistencia entre servicios es eventual y se resuelve con eventos (ver ADR-0005).
- **Regla derivada**: si dos módulos del mismo servicio se comunican con demasiada frecuencia, se refactorizan a un solo módulo; si un módulo cambia por razones distintas a su servicio, se evalúa extraerlo.

ADR-0002 — Polyrepo con contratos como fuente de verdad

Campo	Valor
Fecha	2026-09-23
Estado	Aceptada

Contexto

El requisito del proyecto exige un repositorio separado por cada parte del software. Además, cada servicio usa un lenguaje y un ecosistema distinto, con herramientas de construcción incompatibles entre sí.

Decisión

Un repositorio por componente (nueve en total): gateway, cuatro servicios, PWA, infraestructura, documentación y el repositorio raíz de workspace. La comunicación entre repos se gobierna por **contratos explícitos**:

- HTTP: contrato documentado en `kubo-docs/03-api.md` con ejemplos ejecutables. La **especificación OpenAPI 3.1 en archivo** (y las pruebas de contrato automatizadas) quedan para la Fase 2: hoy el contrato se verifica con la prueba de humo end-to-end, que ejerce todos los endpoints desde el gateway.
- Eventos: sobre estable versionado (`event_id`, `event_type`, `version`, `occurred_at`, `tenant_id`, `data`).
- Base de datos: cada servicio es dueño exclusivo de su esquema; **ningún servicio consulta la base de otro**.

Consecuencias

- **Positivas**: cada servicio se construye, versiona y despliega por separado; un fallo de toolchain en un lenguaje no bloquea a los demás; los equipos pueden crecer sin fricción de monorepo.
- **Negativas**: los cambios de contrato exigen coordinación entre repos. Se mitiga con versionado semántico del contrato y una prueba de humo que valida el flujo completo antes de cada entrega.
- **Regla derivada**: un cambio incompatible en un contrato obliga a subir la versión del evento o del endpoint, nunca a modificar el significado del existente.

ADR-0003 — Multi-tenancy híbrido: `tenant_id` + RLS activo

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Actualizado | 2026-09-24 (RLS activado en la Fase 1) | | Estado | Aceptada |

Contexto

Kubo se vende como software autoalojable: cada negocio instala su propia copia y no paga suscripción. Sin embargo, el mercado exige más adelante una versión en la nube con varias empresas en un mismo despliegue. Cambiar de modelo después es costosísimo si el modelo de datos no lo previó.

Decisión

Modelo híbrido: el código y el esquema son multi-tenant desde el día uno (todas las tablas de negocio llevan `tenant_id` y todas las consultas filtran por él), pero **el despliegue por defecto es de un solo negocio por instalación**.

Como segunda barrera, **RLS está activo con `FORCE`** en las tres bases PostgreSQL. Cada petición abre una transacción y fija `app.tenant_id` mediante un interceptor propio de cada lenguaje; las operaciones que cruzan negocios por diseño (autenticación por correo, cadena de auditoría global, semilla) usan la marca `app.system`. El detalle y las consecuencias están en [ADR-0010](ADR-0010-rls-activo.md).

Activación de RLS (Fase 1)

La activación se pospuso en el MVP por una razón concreta: activar RLS sin fijar la variable de sesión en cada transacción deja las consultas sin resultados —un fallo silencioso—. La Fase 1 aportó lo que faltaba:

1. Interceptores de transacción en los tres servicios (Java, Rails y Phoenix).

2. Políticas versionadas en migraciones (`V3__enable_rls.sql` en IAM,

`20260923000003_enable_rls.rb` en CRM y `20260923000004_enable_rls.exs` en ERP).

3. La distinción explícita entre contexto de negocio y contexto de sistema.

4. Verificación automática en `make smoke`: sin contexto **0 filas**, con

contexto > 0, en las tres bases.

Consecuencias

- **Positivas**: el mismo binario sirve para autoalojamiento y para nube; no hay

que migrar el esquema para cambiar de modelo de negocio. El aislamiento ya no

depende de la disciplina del código: lo impone el motor.

- **Negativas**: toda transacción nueva debe fijar su contexto; los

interceptores cubren el camino normal y los caminos de sistema lo hacen

explícito. Es la regla a recordar al agregar código.

- **Siguiente paso**: zona horaria por negocio en la tabla `tenants` (ADR-0012,

Fase 4) para el despliegue SaaS multi-tenant.

ADR-0004 — Poliglotismo deliberado: un lenguaje por contexto

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

El proyecto exige al menos cuatro microservicios con un mínimo de tres

tecnologías distintas. La tentación es elegir un lenguaje y repetirlo por

comodidad, pero eso convierte el requisito en un ejercicio vacío.

Decisión

Cada servicio se implementa en el lenguaje cuya **forma natural** encaja con su

problema, y esa elección se documenta:

| Servicio | Lenguaje | Por qué ese y no otro | | --- | --- | --- | | Identidad | Java 21 + Spring Boot 4 | El ecosistema de seguridad más maduro: Spring Security, validación, JPA, migraciones. Aquí la corrección importa más que la novedad. | | CRM | Ruby 3.4 + Rails 8 | Velocidad de evolución del modelo comercial. ActiveRecord y las migraciones permiten iterar el esquema de clientes en minutos. | | ERP | Elixir 1.17 + Phoenix 1.8 | La venta es la operación crítica: concurrencia masiva, latencia predecible y tolerancia a fallos con árboles de supervisión de OTP. | | Analítica | Python 3.13 + FastAPI | Agregaciones y, más adelante, pronóstico de demanda. El ecosistema de datos y el motor de agregación de MongoDB encajan sin esquema rígido. | | Gateway | TypeScript + NestJS | Capa de borde con validación de JWT, límites de tasa y enrutamiento: el mismo lenguaje del frontend reduce el cambio de contexto. | | PWA | React 19 + Vite | Interfaz instalable, offline-first y con tipado compartido. |

Consecuencias

- **Positivas**: el equipo aprende cinco ecosistemas reales; cada servicio usa

la herramienta idónea; el requisito se cumple con sentido y no por relleno.

- **Negativas**: cinco toolchains, cinco pipelines, cinco formas de hacer

pruebas. Se mitiga con un contrato común (ADR-0002), con la misma estructura

de carpetas y con la prueba de humo que valida el sistema completo.

- **Regla derivada**: ningún servicio se reescribe en otro lenguaje sin un ADR

que lo justifique con métricas (latencia, costo, mantenibilidad).

ADR-0005 — Eventos con RabbitMQ y publicador tolerante a fallos

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada (outbox implementado en la Fase 1; ver ADR-0009) |

Contexto

Cuando se registra una venta, varios procesos deben reaccionar: analítica

proyecta el indicador, notificaciones avisa al cliente, cartera actualiza el

saldo. Si el ERP llamara a cada uno por HTTP, una caída de analítica rompería la

venta y el acoplamiento crecería con cada consumidor nuevo.

Decisión

1. El ERP publica **eventos de dominio** en el exchange de tipo *topic* `kubo.events`, con sobre estable y versionado (`sale.created` v1).
2. Los consumidores **deduplican por `event_id`**: analítica usa un índice único en MongoDB, de modo que una reentrega del broker no cuenta la venta dos veces.
3. Los mensajes que no se pueden procesar van a una **cola de mensajes muertos** (DLQ), nunca se pierden ni bloquean la cola principal.
4. **La venta nunca depende del bus**: la publicación es asíncrona y tolerante a fallos. Si RabbitMQ está caído, se registra la advertencia y el negocio sigue operando.

Lo que queda pendiente y por qué

El patrón *transactional outbox* (guardar el evento en la misma transacción de la venta y publicarlo después desde una tabla de salida) elimina la ventana en la que un evento puede perderse si el proceso muere justo después del `commit`. En el MVP se asume esa pérdida eventual y se documenta; la implementación del outbox es el primer trabajo de la Fase 1 (P-01, ADR-0009).

Consecuencias

- **Positivas**: desacoplamiento real (analítica puede caer sin afectar la caja), reacción en segundos y camino abierto a nuevos consumidores sin tocar el ERP.
- **Negativas**: consistencia eventual (el tablero tarda milisegundos en reflejar la venta) y, en el MVP, posibilidad de perder un evento ante una caída en el instante exacto del `commit`.
- **Nota técnica**: se usa `amqp` 4.x; la serie 3.x depende de `rabbit_common` 3.12, que no compila en Erlang/OTP 27.

ADR-0006 — Cifrado de datos personales con AES-256-GCM e índice ciego

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

El CRM guarda datos personales de clientes (documento, teléfono), protegidos por la Ley 1581 de 2012 (habeas data). Si alguien obtiene una copia del respaldo o accede a la base, esos datos no deben quedar legibles. Al mismo tiempo, el negocio necesita **buscar por documento** con precisión.

Decisión

1. **Cifrado a nivel de campo** con **AES-256-GCM** (confidencialidad + autenticidad: cualquier manipulación del texto cifrado se detecta y el valor se rechaza). Formato almacenado: `base64(iv ‖ tag ‖ ciphertext)`.
2. **Índice ciego** con HMAC-SHA256 sobre el valor normalizado, en una columna aparte. Permite buscar por igualdad sin descifrar y sin exponer el dato.
3. **Minimización**: los listados devuelven el valor enmascarado (`******432`); solo el detalle de un cliente concreto revela el valor completo.
4. Las claves llegan por variable de entorno (`KUBO_FIELD_ENCRYPTION_KEY`, `KUBO_BLIND_INDEX_KEY`) y **nunca** están en el repositorio.

Alternativas consideradas

- **Cifrar toda la base** (TDE o disco cifrado): no está disponible en PostgreSQL comunitario y no protege contra un volcado lógico.
- **Guardar el dato en claro y confiar en los permisos**: insuficiente ante un respaldo filtrado, que es el escenario real más frecuente.
- **Cifrado determinista** (mismo texto cifrado para el mismo valor): permitiría

buscar, pero filtra información por comparación. Se prefiere el índice ciego.

Consecuencias

- **Positivas**: un volcado de la tabla no revela documentos ni teléfonos; el GCM detecta manipulaciones; la búsqueda sigue siendo exacta y rápida.
- **Negativas**: no se puede buscar por coincidencia parcial sobre el campo cifrado (solo por igualdad) y rotar la clave exige re-cifrar los registros.
- **Siguiente paso**: envoltura de claves (KEK/DEK por negocio) con un KMS para la versión en la nube.

ADR-0007 — JWT RS256 con JWKS: la llave privada no sale del servicio de identidad

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

Varios servicios necesitan saber quién es el usuario y a qué negocio pertenece. La opción cómoda es que cada servicio comparta un secreto simétrico y verifique los tokens por su cuenta, pero eso reparte el secreto por todo el sistema: quien comprometa un servicio puede **emitir** tokens válidos para cualquier usuario.

Decisión

1. El servicio de identidad genera un par de llaves **RSA** y firma los access tokens con **RS256**. La llave privada vive únicamente ahí (o llega por `KUBO_JWT_PRIVATE_KEY` en producción).
2. Publica la llave pública en un documento **JWKS**.
3. El **API Gateway** es el único que valida la firma, contra el JWKS, con caché y refresco automático ante rotación de llave.
4. Tras validar, el gateway **elimina cualquier cabecera `X-User-*` enviada por

el cliente** e inyecta la identidad real antes de reenviar la petición.

5. Los servicios internos confían en esas cabeceras porque **no son alcanzables desde fuera**: solo existen en la red privada de contenedores.

Consecuencias

- **Positivas**: comprometer un servicio de negocio no permite emitir tokens; el gateway centraliza la política de acceso; los servicios no cargan librerías de criptografía ni conocen el formato del token.
- **Negativas**: el gateway es un punto único de fallo y un salto adicional de red. Se mitiga con healthchecks, límites de memoria y la posibilidad de escalarlo horizontalmente sin estado (los límites de tasa viven en Redis).
- **Siguiente paso**: mTLS entre gateway y servicios para la versión en clúster.

ADR-0008 — Alcance declarado del MVP y recortes conscientes

| Campo | Valor | | --- | --- | | Fecha | 2026-09-23 | | Estado | Aceptada |

Contexto

La entrega del MVP ocurre en una jornada. Es imposible cubrir un ERP/CRM de producción completo, y fingir lo contrario sería peor que recortar con criterio.

Decisión

Se prioriza lo que **demuestra arquitectura funcionando de extremo a extremo** y se documenta lo que queda fuera, con su ruta de implementación.

Incluido y funcionando

- Cuatro microservicios + gateway + PWA, con base de datos por servicio.
- JWT RS256 con JWKS, rotación de refresh tokens con detección de reuso, contraseñas con BCrypt, auditoría encadenada.

- Cifrado AES-256-GCM de datos personales con índice ciego y enmascarado.

- Venta transaccional con bloqueo pesimista de inventario y kardex.

- Eventos de dominio por RabbitMQ con consumidor idempotente y DLQ.

- Tablero con agregaciones sobre MongoDB y PWA instalable con cola offline.

- Aislamiento por negocio verificado por prueba automática.

Fuera del MVP (con ruta definida)

| Recorte | Por qué | Cuándo | | --- | --- | --- | | *Transactional outbox* | Requiere tabla de salida y publicador de barrido; el MVP ya no bloquea la venta por el bus | Fase 1 (P-01) | | Activación de RLS | Necesita interceptor de transacción probado con concurrencia | Fase 1 (P-02) | | Refresh token en cookie httpOnly | Exige BFF con cookies; hoy vive en el navegador (documentado) | Fase 1 (P-03) | | Recuperación de contraseña por correo | Requiere proveedor de correo y plantillas | Fase 1 (P-04) | | Facturación electrónica DIAN (UBL 2.1, CUFE, QR) | Requiere ser Proveedor Tecnológico autorizado; el puerto de facturación queda preparado | Fase 4 (P-18) | | Multi-rubro (Vertical Packs) | El MVP demuestra un vertical (retail) completo | Fase 4 (P-17) | | Kubernetes y Terraform | El despliegue objetivo es un local con Docker Compose | Fase 5 |

Consecuencias

- **Positivas**: el sistema se entrega funcionando y verificado, con un mapa

honesto de lo que falta; nada se presenta como terminado sin estarlo.

- **Negativas**: el producto no está listo para un negocio real sin antes

completar la Fase 1 (outbox, RLS, cookie httpOnly, recuperación de contraseña).

- **Regla derivada**: ningún recorte se oculta en la documentación; cada uno

aparece en este ADR y en la matriz de trazabilidad.

ADR-0009 — Outbox transaccional para la entrega de eventos

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 1) | | Relacionada | ADR-0005 (eventos con RabbitMQ) |

Contexto

La venta se confirmaba en PostgreSQL y **después** se publicaba `sale.created` en

RabbitMQ. Entre el `commit` y la publicación existe una ventana en la que el proceso puede morir: la venta ya se cobró al cliente, pero el evento se pierde y el tablero jamás se entera. Es el único punto del sistema donde se podía perder un dato que el usuario ya confirmó.

Publicar antes del `commit` tampoco sirve: un rollback dejaría un evento de una venta que no ocurrió.

Decisión

Patrón **transactional outbox**:

1. La venta y su evento se guardan en la **misma transacción**: `Sales.insert_sale` inserta el sobre completo en `outbox_events` (`PENDING`) antes del `commit`. Si la transacción se revierte, el evento desaparece con ella.
2. Un **publicador de barrido** (`KuboErp.Events.Publisher`) toma lotes con `FOR UPDATE SKIP LOCKED` —seguro con varias instancias del ERP— y los entrega a RabbitMQ. Una venta lo despierta (`kick/0`); además barre cada 500 ms.
3. Entrega **al-menos-una-vez**: si el proceso muere después de publicar y antes de marcar la fila, el evento se reentrega. Los consumidores ya deduplican por `event_id` (índice único en MongoDB).
4. Reintentos con espera exponencial (2, 4, 8… hasta 300 s). Tras 10 intentos el evento queda `FAILED`, visible en la sonda de salud: un evento que no se puede entregar es un incidente, no un detalle.
5. La conexión AMQP se **monitorea** (`Process.monitor`): si el bus cae, el publicador vuelve a estado desconectado y los eventos esperan en la bandeja.

Alternativas descartadas

Alternativa	Por qué no
Reintentos en memoria tras el fallo de publicación	No sobreviven a la muerte del proceso: la ventana seguía abierta
Transacción distribuida (2PC/XA)

con el broker | Complejidad alta y soporte limitado en los cuatro lenguajes | | CDC (Debezium) sobre el WAL | Una pieza más de infraestructura para el mismo resultado |

Consecuencias

- **Positivas**: cero pérdida de eventos confirmados; la caja nunca depende del

bus; el tablero se recupera solo tras una caída; trazabilidad de reintentos.

- **Negativas**: tabla operativa que crece (requiere política de purga en Fase 2);

entrega duplicada posible, asumida por consumidores idempotentes.

- `outbox_events` queda fuera de RLS: es una tabla operativa que el publicador lee

cruzando negocios para entregar los eventos.

Verificación

- `make bus-drill` (4/4): con RabbitMQ detenido la venta se registra, el evento

queda `PENDING`, y al volver el bus se publica y llega a analítica.

- `make smoke`: el evento de la venta queda en la bandeja y se publica.

- La sonda de salud del ERP expone `outbox: {pending, published, failed}`.

ADR-0010 — RLS activo con interceptor de transacción

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 1) | | Relacionada | ADR-0003 (multi-tenancy híbrido) |

Contexto

El aislamiento entre negocios dependía de que **toda** consulta recordara filtrar

por `tenant_id`. Un olvido no se detecta: no falla, simplemente muestra datos de

otro negocio. La política de *row level security* existía escrita desde el MVP,

pero desactivada porque activarla sin un interceptor rompe autenticación y

semilla.

Decisión

RLS activo en las tres bases PostgreSQL, con **`FORCE`** (el rol de la aplicación es dueño de las tablas; sin FORCE PostgreSQL lo eximiría de las políticas).

1. Cada transacción fija su contexto con `select set_config('app.tenant_id', '<uuid>', true)` — local a la transacción, imposible de filtrar a otra petición que reutilice la conexión.

2. **Interceptores por lenguaje**:

- IAM (Java): `TenantRlsFilter` envuelve cada petición en una transacción.
- CRM (Rails): `around_action` en `ApplicationController`.
- ERP (Phoenix): `action/2` en `KuboErpWeb.controller`.

3. **Marca `app.system`** para operaciones que cruzan negocios por diseño: autenticación por correo (el negocio aún no se conoce), rotación de tokens, cadena de auditoría global, semilla y respuesta a incidentes de seguridad.
Sin ella, login y recuperación de contraseña serían imposibles.

4. Alcance por tabla:

- `sale_items` no tiene `tenant_id`: su política hereda el negocio por la venta.
- `outbox_events` queda fuera: tabla operativa que el publicador lee cruzando negocios (ver ADR-0009).
- `tenants` queda sin RLS: el registro debe comprobar la unicidad global del slug.

5. Las transacciones `REQUIRES_NEW` **no heredan** el contexto de la exterior: `SecurityIncidentService` y `AuditService` fijan su propia marca. Sin esto, la revocación de la familia de tokens actualizaba cero filas **en silencio** —el sistema detectaba el robo y no cerraba nada—.

Consecuencias

- El aislamiento pasa a ser garantía del motor, no del programador: una consulta sin contexto devuelve **cero filas**.

- Toda transacción nueva debe fijar contexto; es la regla a recordar al agregar código. Los interceptores cubren el camino normal; lo explícito queda en los caminos de sistema.
- El modo sistema es una capacidad deliberada, acotada a identidad y auditoría.
- El rendimiento no se degrada de forma perceptible (las políticas son comparaciones por índice sobre `tenant_id`).

Verificación

- `make smoke`: consulta sin contexto = 0 filas y con contexto > 0 en IAM, CRM y ERP.
- El humo comprueba además que el reuso de un token revoca la **familia completa**.
- `04-seguridad.md` §4 documenta las políticas vigentes.

ADR-0011 — BFF como capa formal del gateway

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 2) | | Relacionada | ADR-0007 (JWT RS256 con JWKS) |

Contexto

El patrón *Backend For Frontend* apareció dos veces en el gateway sin haberse declarado como decisión: primero con `GET /api/v1/dashboard/overview` (la vista compuesta del tablero) y después con el BFF de autenticación, que guarda el refresh token en una cookie `httpOnly`. Sin una decisión explícita, cada nueva necesidad de la PWA corría el riesgo de resolverse con lógica de presentación repartida entre el navegador y los microservicios.

Decisión

El gateway es la **capa BFF formal** del sistema. Toda composición o adaptación pensada para la PWA vive ahí, no en el navegador ni en los servicios de negocio:

1. **Composición de lecturas** (`/dashboard/overview`): el gateway consulta los

servicios en paralelo (`Promise.allSettled`) y entrega una sola respuesta. Si una vista falla, el tablero llega parcial y lo informa en `unavailable`.

2. **Adaptación de credenciales** (`login`/`refresh`/`logout`): el refresh token se transforma en cookie `httpOnly` + `SameSite=Strict` y desaparece del cuerpo. El navegador nunca lo ve.

3. **Reglas del BFF**:

- Los servicios de negocio conservan contratos limpios y sin conocimiento de la interfaz.
- El BFF no duplica reglas de dominio: solo orquesta y adapta forma.
- Las rutas compuestas se declaran en `GATEWAY_OWNED_PATHS` y no se reenvían.

Consecuencias

- **Positivas**: un solo lugar para la composición y la seguridad de sesión; los microservicios pueden evolucionar sin romper a la PWA; la cookie `httpOnly` es posible sin tocar los servicios de negocio.
- **Negativas**: el gateway gana responsabilidad y debe probarse como capa propia (contratos OpenAPI + E2E + humo); una caída del gateway afecta a todas las vistas compuestas (mitigado con entrega parcial).

Verificación

- `make smoke`: la vista compuesta trae las 7 vistas en una petición y el login no expone el refresh token.
- `make contracts`: el contrato OpenAPI declara `additionalProperties: false` en `LoginResponse`; un `refreshToken` filtrado rompería la validación.
- `make e2e`: el navegador inicia sesión y navega sin leer nunca el token de refresco.

ADR-0012 — Zona horaria por negocio

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 4) | | Relacionada | ADR-0007 (JWT RS256 con JWKS) · ADR-0011 (BFF del gateway) |

Contexto

El defecto A-01 de la auditoría: el ERP calculaba el «día comercial» en UTC, así que una venta de las 20:00 en Colombia aparecía en el día siguiente. Se corrigió calculando en la zona del negocio, pero la zona quedó **fija por configuración** (`KUBO_TIMEZONE`, valor por defecto `America/Bogota`). Kubo es un producto para la región: un negocio en México, Ecuador o España no puede compartir la zona del despliegue, y la fecha de cierre, los reportes y el comprobante deben hablar en la hora del negocio, no en la del servidor.

La zona horaria es un atributo del **negocio**, no del despliegue: es el mismo razonamiento que llevó el aislamiento a RLS (ADR-0010) en lugar de confiar en que cada consulta recuerde filtrar.

Decisión

La zona horaria vive en el negocio y viaja con la identidad, sin que el ERP tenga que consultarla a IAM en cada petición:

1. **IAM es la fuente de verdad**: `tenants.timezone` (columna `NOT NULL` con valor por defecto `America/Bogota` y restricción de no vacío). El negocio nuevo nace con la zona por defecto; un cambio de zona es un cambio de datos, no de configuración.
2. **Viaja en el token**: `TokenService` agrega el claim `tenant_timezone` al token de acceso. El ERP no conoce la tabla `tenants` (database-per-service): el claim es la forma de que reciba el dato sin acoplarse a IAM.
3. **El gateway la propaga**: `auth.middleware.ts` añade `x-tenant-timezone` a

las cabeceras de identidad. Como el resto, la cabecera que envíe el cliente se **borra primero** y se reescribe con el valor verificado del token: nadie puede cambiar su día comercial para falsear un cierre.

4. **El ERP la aplica con respaldo**: el plug `Identity` valida el nombre contra la base de zonas (`tzdata`) y rechaza con 400 `INVALID_TIMEZONE` una zona desconocida —una zona inválida no debe degradar el cálculo a UTC en silencio—. `Sales.stats/2` y `Reports.sales_csv/3` reciben la zona de la petición y, si no llega (llamada interna, publicador de eventos), usan el respaldo configurado.

Consecuencias

- **Positivas**: el día comercial, los reportes y el comprobante son correctos para cualquier negocio de la región sin reconfigurar el despliegue; el dato viaja firmado (no es manipulable por el cliente); el ERP no consulta IAM en cada petición.
- **Negativas**: el token de acceso carga un dato más y un cambio de zona no aplica hasta el siguiente inicio de sesión (aceptable: la zona casi nunca cambia). El respaldo por configuración sigue existiendo y podría enmascarar un claim ausente en llamadas internas; por eso el camino de la petición siempre pasa la zona explícita.

Verificación

- `make ci` → `kubo-iam`: prueba de que el token firmado incluye `tenant_timezone` con el valor del negocio.
- `kubo-gateway` (`npm test`): la cabecera `x-tenant-timezone` se reescribe con el valor del token y nunca conserva el del cliente.
- `kubo-erp` (integración con PostgreSQL real): `Sales.stats/2` respeta la zona

recibida y `Sales.stats/1` cae al respaldo.

- `make smoke` (**109/109**): el ERP responde con la zona del negocio, honra una zona distinta recibida por cabecera y rechaza (400) una zona desconocida.

ADR-0013 — Vertical Packs: el vertical como dato del negocio

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 4) | | Relacionada | ADR-0012 (zona horaria por negocio) · ADR-0011 (BFF del gateway) |

Contexto

Kubo nace para el comercio de barrio, pero «comercio» abarca negocios con operaciones distintas: una tienda vende productos con inventario y código de barras, un restaurante vende platillos y maneja mesas, un prestador de servicios no tiene inventario y el agro trabaja con insumos y bodegas. El plan de cierre (P-17) pide «dos verticales activables sin tocar el núcleo».

La tentación es bifurcar el producto (ramas por vertical, campos `if restaurante` repartidos por el código). Eso convierte cada vertical nuevo en un cambio del núcleo — exactamente lo que el criterio de aceptación prohíbe.

Decisión

Un **vertical es un dato del negocio** y un **paquete de configuración** es declarativo:

1. **El catálogo de paquetes vive en el ERP** (`KuboErp.Packs`): el ERP es el dueño del núcleo comercial. Cada paquete declara terminología (`product_label`, `product_label_plural`), impuesto por defecto, si lleva inventario y el flujo del punto de venta (`sale`, `table`).
2. **IAM guarda cuál tiene activo el negocio** (`tenants.vertical`, valor por defecto `retail`) y lo valida contra la lista de paquetes conocidos: es un

contrato entre servicios, igual que los roles.

3. **Viaja con la identidad**: el vertical va en el token (`tenant_vertical`) y el gateway lo propaga como `x-tenant-vertical`, reescribiéndolo siempre con el valor verificado (mismo patrón que la zona horaria, ADR-0012).

4. **El núcleo no pregunta por el vertical**: el ERP resuelve el paquete en un único punto (`Packs.get/1`) y la PWA adapta etiquetas con un contexto (`usePack`). Añadir un vertical es agregar una entrada al catálogo, no tocar la venta, el kardex ni la caja.

5. **Un vertical desconocido cae al paquete por defecto con aviso**, a diferencia de la zona horaria (que se rechaza): el paquete solo afecta etiquetas y valores por defecto —no la integridad de los datos—, así que un token viejo no debe tumbar la operación. El cambio aplica al siguiente refresco de sesión, que la propia interfaz dispara al guardar.

Consecuencias

- **Positivas**: activar un vertical es un `PATCH /tenants/me` y un refresco de token; el núcleo permanece igual para todos los negocios; la terminología deja de estar cableada en la interfaz.

- **Negativas**: la lista de verticales se conoce en dos servicios (IAM valida, ERP aplica) y debe mantenerse en sincronía; se acepta porque cambia muy poco y el desajuste degrada a etiquetas, no a datos. La siembra de arranque crea productos de ejemplo: un negocio real deberá editarlos o borrarlos, y por eso es una acción explícita del usuario (nunca automática al registrar).

Verificación

- `kubo-iam`: prueba del claim `tenant_vertical` y de la validación (`INVALID_VERTICAL`) en el cambio de perfil del negocio.

- `kubo-gateway`: la cabecera `x-tenant-vertical` se reescribe con el valor del

token, nunca con el del cliente.

- `kubo-erp`: prueba pura del catálogo (cuatro verticales con etiquetas propias

y respaldo ante un vertical desconocido).

- `make smoke` (**120/120**): el catálogo ofrece los cuatro verticales, la demo

arranca en `retail`, un vertical desconocido responde `INVALID_VERTICAL`, el

cambio a `restaurantes` se refleja en el token nuevo (`Platillo`) y el

comportamiento también cambia: un producto con `tracks_stock: false` se vende

sin existencias, no deja kardex, la venta guarda la mesa y `POST /packs/apply`

siembra el catálogo de arranque sin duplicar al repetirse.

- `make e2e`: la pantalla de configuración pasa la auditoría de accesibilidad.

ADR-0014 — Facturación electrónica DIAN como puerto enchufable

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 4) | | Relacionada | ADR-0009 (outbox transaccional) · ADR-0013 (vertical packs) |

Contexto

La facturación electrónica en Colombia exige ser **Proveedor Tecnológico

autorizado** por la DIAN o contratar uno: el trámite es externo y no puede

bloquear el producto. Además, el documento tiene reglas propias (UBL 2.1, CUFE,

QR del catálogo, numeración, notas crédito) que cambian con la normativa.

Si el núcleo comercial llamara directamente a un proveedor concreto, cada

cambio de proveedor —o el paso de habilitación a producción— tocaría la venta,

el reporte y el comprobante.

Decisión

La facturación es un **puerto** (`KuboErp.Billing`) con adaptadores

intercambiables:

1. **El contrato**: `issue(tenant, sale)` devuelve la representación emitida (`number`, `cufe`, `qr_url`, `xml`) o un error. El núcleo no conoce al proveedor.
2. **Adaptador por defecto** (`KuboErp.Billing.Sandbox`): genera UBL 2.1 con el CUFE calculado como lo define la DIAN (SHA-384 de la cadena de datos, 96 hexadecimales), el QR del catálogo y el escapado XML. Sirve para desarrollo y para el ambiente de habilitación; **no tiene validez fiscal**. El NIT y la clave técnica son marcadores documentados: registrarlos ante la DIAN es parte del trámite externo.
3. **Documento inmutable**: la factura se persiste con su CUFE y su XML (`invoices`, con RLS como el resto de tablas de negocio). Emitir es **idempotente**: si la venta ya tiene factura se devuelve la existente, porque regenerarla cambiaría el CUFE que ya recibió el cliente.
4. **Regla de negocio**: una venta anulada no se factura (409 `SALE_VOIDED`); el camino correcto es una **nota crédito**, que será un documento nuevo.
5. **Datos del emisor**: el nombre del negocio viaja en el token (`tenant`) y el gateway lo propaga como `x-tenant-name`; el ERP no consulta IAM.

Consecuencias

- **Positivas**: el núcleo no depende del proveedor; se puede desarrollar y probar sin habilitación; el paso a producción es configuración (`KUBO_BILLING_ADAPTER`); la factura emitida queda auditable y aislada por negocio.
- **Negativas**: el adaptador sandbox puede dar una falsa sensación de cumplimiento —por eso el plan declara la habilitación como cierre externo y el

módulo lo advierte—. Las notas crédito **ya están** (ver «Implementado») y la firma XAdES queda del lado del proveedor tecnologico: el puerto entrega el XML y el proveedor lo firma y lo envia; implementarla nosotros sin poder validarla ante la DIAN seria un adorno. El detalle de por que aún no está implementadas: son el siguiente paso de P-18. El NIT real del emisor exigirá un campo en el negocio (hoy marcador).

Verificación

- **Nota crédito (implementada)**: anular una venta facturada emite el documento que corrige la factura —la factura es inmutable: no se edita ni se borra— con su propio **CUDE** (SHA-384, 96 hexadecimales), tipo 91 y la referencia a la factura (número y CUFE) en UBL 2.1; su XML queda como documento `CREDIT_NOTE_XML`. La emisión es idempotente por venta y exige la venta anulada y facturada (`SALE_NOT_VOIDED`, `INVOICE_NOT_FOUND`). Evidencia: ERP puras **32/32** e integración **16/16**, humo **184/184** (la nota referencia la factura, el XML es CreditNote 91, reemitir devuelve la misma y sin factura no hay nota).
- `kubo-erp` puras **32/32**: CUFE hexadecimal de 96 caracteres, estable con los mismos datos y distinto al cambiar el total; XML UBL 2.1 con totales, NIT y datos escapados.
- `kubo-erp` integración **9/9**: emitir dos veces devuelve la misma factura, RLS la aísla por negocio y una venta anulada responde `:sale_voided`.
- `make smoke` (**125/125**): CUFE válido, idempotencia, XML UBL 2.1 con el nombre del negocio propagado por el gateway y 409 al facturar una venta anulada.

Actualización (2026-10-03)

El puerto quedó **listo para enchufar un proveedor tecnológico** sin tocar el
núcleo:

- El adaptador se elige por configuración (`KUBO_BILLING_ADAPTER`) y el ambiente DIAN por `KUBO_BILLING_ENVIRONMENT` (1 producción, 2 habilitación); el ERP valida al arrancar que el módulo implemente el contrato.
- Los **datos fiscales del emisor** (NIT con DV calculado, dirección, régimen, resolución y prefijo) viven en `tenants`, se editan en Configuración y viajan en el token como el resto de la configuración del negocio.
- Los adaptadores reportan errores tipados (`KuboErp.Billing.Error`) y pueden implementar `refresh_status/1` si su proveedor valida de forma asíncrona; la factura guarda `status`, `provider_reference` y `status_detail`.
- Guía paso a paso: [`13-guia-adaptador-facturacion.md`](../13-guia-adaptador-facturacion.md).
- Evidencia: humo **192/192** (incluye el NIT con DV, el régimen y el prefijo en el XML del sandbox).

ADR-0015 — Segundo factor TOTP del propietario

Campo	Valor
Fecha	2026-09-24
Estado	Aceptada (Fase 4)
Relacionada	ADR-0007 (JWT RS256 con JWKS) · ADR-0011 (BFF del gateway)

Contexto

El propietario de un negocio concentra los permisos: gestiona usuarios, cambia el perfil del negocio y ve toda la operación. Una contraseña filtrada —reutilizada de otro sitio, vista por encima del hombro en el mostrador— entrega el negocio completo. El plan de cierre (P-30) pide el segundo factor del propietario, y el estándar de facto para una app de barrio es TOTP: lo entienden Google Authenticator, Authy y 1Password, y no depende de SMS (costoso y vulnerable a SIM swapping).

Dos decisiones no obvias hay que registrar: **dónde vive el secreto** y **cómo entra el segundo paso en un login que hoy emite tokens de una vez**.

Decisión

1. **El secreto se cifra en reposo** (AES-256-GCM, llave en `KUBO_TOTP_ENCRYPTION_KEY`). El secreto es la llave para generar códigos válidos: en claro, una copia de la base convierte el segundo factor en decorativo. Sin llave configurada el servicio **no arranca** (fail-fast), y la llave marcador de la plantilla se rechaza, igual que en el cifrado de campos del CRM.
2. **Implementación propia y sin dependencias** (`TotpService`): HMAC-SHA1, 6 dígitos, paso de 30 s y **ventana de ±1 paso**. Se verifican los vectores del RFC 6238 en las pruebas para no validar la implementación contra sí misma. Cada paso extra de tolerancia multiplicaría las oportunidades de un atacante con un código observado.
3. **El acceso se parte en dos**: con el segundo factor activo, `POST /auth/login` responde `{totpRequired: true, challengeToken}` en lugar de la sesión. El desafío es un JWT firmado de 5 minutos con **`typ=totp`**, y el gateway solo acepta `typ=access`: sin esa comprobación, un desafío serviría como credencial de API. `POST /auth/totp/verify` (ruta pública) cambia desafío + código por la sesión.
4. **La activación exige un código**: `setup` genera el secreto y lo deja *pendiente*; `enable` lo confirma con el primer código válido (así se detecta un escaneo mal hecho antes de bloquear el acceso); `disable` exige un código vigente. Todo queda en la auditoría (`TOTP_CHALLENGED`, `TOTP_ENABLED`, `TOTP_FAILED`, `TOTP_DISABLED`).

5. **El estado viaja en el perfil**: `UserResponse.totpEnabled` permite que la interfaz muestre el estado real sin adivinar.

Consecuencias

- **Positivas**: el segundo factor es real (secreto cifrado, desafío de un solo uso, tolerancia acotada) y no agrega dependencias al servicio; el flujo de login normal no cambia para quien no lo activa; el gateway queda blindado contra el uso cruzado de tokens por tipo.
- **Negativas**: si el propietario pierde el autenticador no hay recuperación automática —el camino es un administrador con acceso a la base o un procedimiento de soporte, que el plan deja como manual—; el desafío dura 5 minutos y un código observado en ese lapso sigue siendo válido por la ventana de tolerancia. El segundo factor es opcional: se recomienda, no se impone.

Verificación

- `kubo-iam` **46 pruebas**: vectores del RFC 6238, ventana de tolerancia, cifrado del secreto (round-trip y rechazo de la llave marcador), desafío firmado/verificado de verdad, ciclo setup → enable → disable y accesos inválidos.
- `kubo-gateway` **9/9**: el desafío no sirve como token de acceso (el `typ` se comprueba) y `/auth/totp/verify` es pública.
- `make smoke` (**130/130**): con un código TOTP real calculado en Python (implementación independiente) se activa, se entra en dos pasos, se rechaza un código inválido y se desactiva; el usuario de prueba queda restaurado.
- `make e2e`: el ingreso normal (sin segundo factor) sigue pasando la auditoría de accesibilidad y llegando al tablero.

ADR-0016 — Stock por bodega y transferencias

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 4) | | Relacionada | ADR-0010 (RLS activo) · ADR-0013 (vertical packs) |

Contexto

Hasta hoy el inventario del ERP es **un solo número por producto** (`products.stock`), con el kardex (`stock_movements`) como historia. Eso alcanza para una tienda con una trastienda, pero no para el negocio que creció: una bodega y el local, dos locales, o el camión que reparte. El plan de cierre (P-22) pide multi-bodega y transferencias.

El riesgo del cambio es que **todo el sistema lee `products.stock`**: el POS valida existencias, el tablero cuenta stock bajo, los reportes valorizan el inventario, el kardex calcula `stock_after`. Cambiar el modelo sin romper esas lecturas es la restricción de diseño.

Decisión

1. **`warehouses`** (por negocio, borrado lógico): una bodega es un lugar donde hay existencia. Exactamente **una por defecto** por negocio, garantizada con índice único parcial (`WHERE is_default AND deleted_at IS NULL`).
2. **`stock_levels` es la fuente de verdad por bodega**: `(tenant_id, warehouse_id, product_id, stock)` con índice único. Reemplaza a los movimientos como fuente de verdad (el kardex sigue siendo la **historia auditable**, no el saldo).
3. **`products.stock` se conserva como total denormalizado**, mantenido en la misma transacción. Es una decisión consciente: las lecturas existentes (POS, reportes, tablero, alertas) siguen funcionando sin cambios y el total es trivialmente consistente porque se escribe junto al detalle. Cuando el POS

permita elegir bodega, la lectura por bodega saldrá de `stock_levels`.

4. **El kardex gana `warehouse_id`** (NOT NULL tras el backfill) y su índice de idempotencia pasa a incluir la bodega: una transferencia mueve el **mismo producto dos veces** con la misma referencia (sale de una bodega y entra en otra), y el índice anterior lo habría rechazado.

5. **Transferencia = par de movimientos atómicos**: `stock_transfers` + `stock_transfer_items`; al completarse, en **una sola transacción**, se descuenta el origen (`OUT`) y se suma el destino (`IN`), ambos con `reference_type = "TRANSFER"` y el id de la transferencia. Reglas: origen distinto del destino, cantidades positivas, producto con inventario y existencia suficiente en el origen. Si algo falla no se mueve nada.

6. **Venta, compra, ajuste e importación operan en la bodega por defecto**. El POS todavía no elige bodega: `Catalog.move_stock/3` recibe la bodega y resuelve la por defecto cuando no llega, en un solo punto.

7. **RLS `FORCE`** en `warehouses`, `stock_levels`, `stock_transfers` y `stock_transfer_items`, como el resto de tablas de negocio.

Consecuencias

- **Positivas**: el inventario por bodega es real y auditable; las lecturas existentes no cambian; la transferencia es atómica por construcción; el negocio de una sola bodega no nota la diferencia (todo cae en la bodega por defecto).
- **Negativas**: hay **dos representaciones** del stock (el total del producto y el detalle por bodega) que deben escribirse juntas; se mitiga manteniéndolas en la misma transacción y con una prueba de integración que las compara. El total denormalizado deja de ser válido si alguien escribe `stock_levels` sin

pasar por `Catalog`. El POS aún no permite elegir bodega ni hay mínimos por bodega: son los siguientes incrementos de P-22.

Verificación

- `kubo-erp` integración **10/10** (PostgreSQL real): la transferencia mueve origen y destino y deja **dos movimientos** con la misma referencia (uno por bodega); sin existencia en el origen **no se mueve nada** (niveles intactos); el total del producto no cambia con la transferencia y sigue siendo la suma de las bodegas; la venta, la compra y el ajuste descuentan de la bodega por defecto.
- `make smoke` (**137/137**): la bodega por defecto existe, la transferencia mueve el producto, **el total no cambia**, el kardex deja los dos movimientos, transferir más de lo disponible responde 409 y la bodega por defecto no se puede borrar.
- `make contracts` (**18/18**): `WarehouseList` y `TransferList` validados contra la API viva.
- **Defectos reales corregidos durante la implementación**: el esquema del kardex no conocía `warehouse_id` (los movimientos fallaban con `not_null_violation`); el runner de pruebas montaba `lib/` y `config/` pero no `priv/` (las migraciones nuevas no se aplicaban); la segunda pata de la transferencia usaba el producto **obsoleto**, inflando el total (10 → 14); la lista de transferencias no precargaba las líneas (`NotLoaded`, 500); y —el más serio— **el backfill de la migración era invisible para RLS**: `products` y `stock_movements` ya tenían `FORCE ROW LEVEL SECURITY`, así que el `SELECT` del backfill devolvía cero filas y el `SET NOT NULL` fallaba en bucle de arranque. La migración suspende `FORCE` mientras corre (el dueño de la tabla

puede) y lo restaura al terminar; el backfill es **idempotente**

(`WHERE warehouse_id IS NULL`, `ON CONFLICT DO NOTHING`).

- **Siguiente incremento de P-22**: pantalla de bodegas/transferencias en la PWA

y elección de bodega en el POS.

ADR-0017 — Puerto de notificaciones y buzón del negocio

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 4) | | Relacionada | ADR-0009 (outbox transaccional) · ADR-0014 (puerto de facturación) |

Contexto

El plan de cierre (P-19 · P-25) pide notificaciones (WhatsApp/correo) y un

servicio de Documentos. La parte difícil no es enviar: es **no acoplar el

núcleo** a un proveedor —WhatsApp Business, SMTP, un agregador— ni a su

disponibilidad. Una venta no puede fallar porque la pasarela de mensajes esté

caída, y cambiar de proveedor no debe tocar la venta.

Además, el negocio necesita **enterarse**: que el sistema avise «se está

agotando el arroz» o «entró una compra» es más útil que un panel que hay que

recordar mirar.

Decisión

1. **Puerto** (`KuboErp.Notifications`): el núcleo dice *qué* pasó; el

adaptador decide *cómo* se entrega (`deliver/1`). El adaptador por defecto

(`Log`) escribe en el **buzón del negocio** (`notifications`): sirve de

demostración, de auditoría de lo enviado y de punto de partida para un

proveedor real.

2. **El buzón es una tabla de negocio**: `notifications` (tenant, tipo, canal,

destinatario, asunto, cuerpo, estado, referencia, fecha de envío) con RLS

`FORCE`, como el resto. La PWA puede leerlo (`GET /notifications`).

3. **El aviso se encola en la transacción que lo provoca** (`PENDING`): es una escritura local y rápida, así que un canal lento o caído no puede frenar ni revertir una venta. Si la operación se revierte, el aviso desaparece con ella.
El **entregador** (`Notifications.Deliverer`) barre pendientes cada 5 s como proceso de sistema (marca `app.system`), entrega por el adaptador y anota el resultado: estado, intentos y último error. Tras 5 intentos queda `FAILED` para que un humano lo revise.
4. **Aviso de stock bajo con histéresis**: se notifica cuando el producto **cruza** el mínimo (`antes > mínimo` y `después <= mínimo`), no en cada venta por debajo. Sin esa regla, un producto agotado genera un aviso por venta y el buzón se vuelve ruido que nadie lee.
5. **Configurable**: `KUBO_NOTIFICATIONS_ADAPTER` (`log` por defecto, `smtp` para correo real con Swoosh y la familia `KUBO_SMTP_*` de IAM). Un adaptador de WhatsApp implementa el mismo contrato.

Consecuencias

- **Positivas**: el núcleo no conoce al proveedor ni depende de su disponibilidad; el buzón deja rastro auditable y es aislado por negocio; añadir un canal es escribir un adaptador.
- **Negativas**: el adaptador por defecto **no entrega nada fuera del sistema** —es una demostración, no un canal real— y el plan lo declara. La entrega es de al-menos-una-vez: si el entregador muere tras enviar y antes de marcar, el aviso se reenvía (aceptable para un correo interno; un canal con costo por mensaje necesitaría idempotencia en el proveedor).

Verificación

- `make smoke` (**146/146**): una venta que cruza el mínimo deja un aviso `LOW_STOCK` con las unidades restantes y el entregador lo marca **`SENT`** en segundo plano.
- `kubo-erp` integración **12/12**: el aviso nace `PENDING` y el barrido de sistema lo entrega; el adaptador SMTP se valida construyendo el correo (sin enviarlo).
- Buzón en la PWA con filtro por stock bajo; auditoría axe del E2E en verde.
- `kubo-erp` integración: la notificación se escribe en la misma transacción que la venta y respeta RLS.
- `make contracts` (**19/19**): `NotificationList` validado contra la API viva.
- **Pendiente del paso**: adaptador de WhatsApp, notificaciones de compra y de resumen diario, y soportes de compra como documentos (P-25).

ADR-0018 — Documentos: en el ERP, con puerto de almacenamiento

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 4) | | Relacionada | ADR-0002 (database-per-service) · ADR-0014 (facturación) · ADR-0017 (notificaciones) |

Contexto

El plan de cierre (P-25) deja abierto **dónde** viven los documentos —el XML de la factura, el comprobante, los soportes de una compra—: ¿un quinto microservicio de Documentos, como figuraba en la trazabilidad, o dentro del ERP?

Los hechos que enmarcan la decisión:

- **El ERP ya es el dueño de los documentos**: emite la factura (XML + CUFE), imprime el comprobante y exporta los reportes. Un servicio aparte solo recibiría bytes sin conocer su negocio ni su vigencia.
- **Kubo es autoalojable para un negocio de barrio**: cada servicio extra es un

contenedor más que el dueño (o su técnico) debe operar, actualizar y respaldar.

- **El volumen es pequeño**: un negocio emite decenas de documentos al día, no millones de objetos.
- **La nube es opcional, no un supuesto**: un despliegue puede querer guardar los documentos en un bucket (S3, Cloudinary) y otro en el disco del servidor.

Decisión

1. **Los documentos viven en el ERP** (`documents`), no en un quinto servicio. El ERP es el dueño del dato de negocio que los origina y puede aplicar la misma RLS, la misma auditoría y el mismo respaldo que el resto.
2. **El almacenamiento es un puerto** (`KuboErp.Documents.Storage`): el adaptador por defecto escribe en el sistema de archivos (`KUBO_DOCUMENTS_PATH`, un volumen del despliegue); un adaptador de objeto (S3/Cloudinary) implementa el mismo contrato. La fila de `documents` guarda los metadatos y la **clave** del objeto, no los bytes: la base no engorda y el respaldo de la base y el de los archivos pueden tener políticas distintas.
3. **Integridad**: cada documento guarda el **SHA-256** de su contenido. Un documento fiscal no se edita: si el contenido cambia, es otro documento.
4. **La descarga pasa por el ERP** (`GET /documents/:id`), que resuelve la clave en el almacenamiento y responde con su tipo de contenido. Así el aislamiento por negocio se aplica igual que en el resto del API y el almacenamiento no necesita ser público.
5. **Primer documento: el XML de la factura**, guardado al emitirla. Un documento se crea como **consecuencia de un hecho de negocio**, no por una subida manual: eso deja la trazabilidad cerrada (factura ↔ documento).

Consecuencias

- **Positivas**: cero servicios nuevos que operar; el aislamiento, la auditoría y el respaldo son los del ERP; cambiar a almacenamiento de objetos es escribir un adaptador; la integridad queda verificable por el hash.
- **Negativas**: el disco del ERP pasa a ser estado que hay que respaldar (se documenta en el runbook; el `backup.sh` deberá incluir el volumen); si algún día los documentos crecen a millones de objetos o varios servicios necesitan consumirlos, este ADR se revisa —la migración es barata porque los metadatos y la clave ya están separados—.
- **Enmienda (misma fecha)**: el comprobante **también se genera en el servidor** con un **escritor mínimo** (`ReceiptPdf`: objetos PDF, `xref` con desplazamientos reales y Courier con WinAnsi para los acentos), sin introducir un motor de PDF —sin tipografías, imágenes ni varias páginas—. Así el comprobante queda como documento descargable y archivado, y el navegador sigue pudiendo imprimirlo. El **adaptador de almacenamiento de objetos** (Cloudinary, firma SHA-1 y URL determinista) cierra el puerto por el otro extremo.

Verificación

- `kubo-erp` integración **12/12**: emitir una factura deja un documento `INVOICE_XML` con el hash de su contenido, y el contenido leído coincide con el XML emitido.
- `make smoke` (**149/149**): la factura de la venta de prueba produce un documento descargable con su **SHA-256** verificado, y la compra acepta un soporte adjunto (rechazando un formato no soportado).
- `make contracts` (**20/20**): `DocumentList` validado contra la API viva.
- `make smoke` (**154/154**): la venta deja su **comprobante en PDF** (cabecera `%PDF-1.4`, número de la venta dentro del archivo) además del XML de la

factura; la CA de la malla se verifica con 30 días de vigencia.

- **Soportes de compra**: `POST /purchases/:id/documents` acepta el archivo

(multipart, hasta 5 MB, formatos de documento e imagen) y lo liga a la compra;

un formato no soportado responde `INVALID_EXTENSION`.

- **Sin pendientes en este ADR**: el comprobante en servidor y el adaptador de

objetos quedaron implementados; los soportes de compra adjuntos también.

ADR-0019 — Rotación de claves de cifrado de campo (KEK/DEK)

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 5) | | Relacionada | ADR-0010 (RLS activo) · ADR-0015 (secreto TOTP cifrado) |

Contexto

El CRM cifra los datos personales (documento y teléfono) con AES-256-GCM y una

llave en configuración (`KUBO_FIELD_ENCRYPTION_KEY`). Esa llave **nunca rota**:

si se filtra, si un empleado con acceso al servidor se va, o si la normativa

exige rotación periódica, hoy no hay camino —y re-cifrar todo de una sentada

dejaría el sistema inconsistente si algo falla a mitad.

Rotar una llave de cifrado de datos vivos tiene tres problemas clásicos:

1. **Ventana de inconsistencia**: mientras se re-cifra, conviven valores viejos y

nuevos; el sistema debe poder leer ambos.

2. **Indisponibilidad**: no se puede parar la operación para re-cifrar.

3. **Pérdida de datos**: una rotación mal hecha (o interrumpida) destruye datos

que no se pueden recuperar.

Decisión

Envelope encryption con anillo de llaves y formato versionado (KEK/DEK):

1. **El texto cifrado dice con qué llave se escribió**:

`v1:<key_id>:base64(iv|tag|ciphertext)`. Sin prefijo (valores anteriores al anillo) se descifra con la llave actual: **compatible hacia atrás**.

2. **Anillo de llaves**: `KUBO_FIELD_ENCRYPTION_KEYS` = `id:hex,id:hex,...` con la llave **nueva al frente** (la que cifra) y las viejas detrás (las que descifran). Sin esa variable se usa `KUBO_FIELD_ENCRYPTION_KEY` con el id `default`, así que el despliegue actual sigue funcionando sin cambios.

3. **La rotación es una tarea de sistema, por lotes e idempotente** (`rake kubo:rotate_field_keys`): recorre los clientes y re-cifra solo lo que está en una llave vieja; el índice ciego se recalcula en el mismo guardado (los setters del modelo lo hacen). Se puede repetir tras una interrupción y se puede correr en caliente.

4. **Nunca se destruye un dato ilegible**: si un valor no se puede descifrar (llave ausente, contenido manipulado), la fila **no se toca** y se cuenta en el resumen (`unreadable`). Una rotación no es una purga.

5. **La tarea cruza negocios con la marca `app.system`** (política de RLS del CRM, migración `000004`), igual que el entregador de notificaciones del ERP. El camino de la petición sigue aislado por negocio.

Consecuencias

- **Positivas**: rotar es agregar una llave al frente del anillo y correr una tarea; no hay ventana de indisponibilidad ni de inconsistencia; una interrupción se reanuda repitiendo la tarea; los datos ilegibles se preservan y se reportan.

- **Negativas**: el anillo crece con cada rotación y hay que retirar las llaves viejas cuando ya no queden valores con su id (el resumen de la tarea lo permite saber: cuando `rotated` es 0, las viejas sobran). La llave raíz (KEK)

sigue viviendo en la configuración: este ADR resuelve la rotación de las llaves de datos, no el gobierno de la KEK (un KMS o un cofre de secretos es el paso siguiente). El IAM tiene su propio cifrado (secreto TOTP, ADR-0015) que puede adoptar el mismo anillo cuando haga falta.

Verificación

- `kubo-crm` puras **12/12**: el formato lleva la llave, un valor sin prefijo se descifra con la actual, una llave desconocida no descifra nada y rotar no pierde el valor.
- `kubo-crm` integración **3/3** (PostgreSQL real): un cliente guardado con la llave vieja queda con la nueva tras la rotación, se sigue leyendo y el índice ciego se recalcula (la búsqueda por documento sigue encontrándolo).
- `make ci` **10/10** con la tarea integrada en el runner del CRM.

ADR-0020 — mTLS en la malla interna

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 5) | | Relacionada | ADR-0007 (JWT RS256) · ADR-0010 (RLS) · ADR-0011 (BFF) |

Contexto

Los servicios internos (IAM, CRM, ERP y analítica) se hablan por HTTP plano dentro de la red del compose: la única defensa es que el gateway **borra las cabeceras de identidad** del cliente y las reescribe con el token verificado. Eso protege contra un cliente externo, pero no contra un contenedor comprometido o añadido a la red: cualquiera que alcance `kubo-erp:8083` puede enviar `x-tenant-id` y `x-user-role` y suplantar a un negocio entero. El plan de cierre (P-28) pide mTLS entre el gateway y los servicios.

En un producto autoalojable, la malla interna no sale del host del dueño; el riesgo realista no es un atacante en internet, sino el aislamiento entre

contenedores y el principio de menor privilegio.

Decisión

1. **Una CA interna propia** (`kubo-infra/certs/`, generada con `scripts/gen-internal-certs.sh`): no se usa una CA pública porque los nombres son de la red del compose (`kubo-erp`, `kubo-iam`, …) y no deben existir en internet. Las llaves **no se versionan** (`.gitignore`); cada despliegue genera las suyas.
2. **Cada servicio presenta su certificado y exige el del cliente** (`client-auth: need` / `verify_mode: peer` / `verify: :verify_peer` con `fail_if_no_peer_cert`). Un contenedor sin certificado de la CA no puede hablar con la malla, aunque esté en la red.
3. **El gateway es el único cliente** de la malla: presenta su certificado en cada reenvío y **verifica el del servicio** (`rejectUnauthorized: true` con la CA interna). El navegador sigue entrando por HTTP/HTTPS público al gateway; el salto interno es el que se cifra y autentica.
4. **Los SAN incluyen `localhost`**: los *healthchecks* corren dentro del contenedor y usan `https://localhost:PUERTO` con la CA, de modo que la verificación de nombre también se ejerce ahí.
5. **La CA es un artefacto de despliegue**: `make certs` la genera; el runbook documenta su rotación (regenerar y recrear la malla) y que **no** entra al repositorio.

Consecuencias

- **Positivas**: la suplantación por cabeceras deja de ser posible desde fuera de la malla; el tráfico interno viaja cifrado; la rotación de la CA es explícita y local; el patrón es el mismo para cualquier servicio nuevo (un certificado más

en el bucle del script).

- **Negativas**: un certificado vencido o una CA regenerada tumba la malla completa (se mitiga con la vigencia de 825 días, un `make certs` idempotente y la verificación de `make ci`); los *healthchecks* y las llamadas directas de los scripts pasan por la CA (`--cacert`), lo que agrega un parámetro a las herramientas de operación. En un despliegue con orquestador (Kubernetes) esto se reemplaza por una malla de servicio (Istio/Linkerd) o `cert-manager`: este ADR cubre el despliegue de compose.

Verificación

- `make ci` (**10/10**) y humo (**150/150**) con la malla cifrada: el gateway habla con los cuatro servicios por HTTPS mutuo.
- Un cliente **sin** certificado no obtiene respuesta de la malla (comprobación negativa en el humo: `curl` con la CA pero sin certificado → error de TLS).
- `make certs` genera la CA y los certificados de forma idempotente; `make up` depende de él, así que un clon nuevo no puede arrancar sin malla.
- **Hallazgos de la implementación**: `fetch` nativo de Node **no** presenta certificado de cliente, así que el gateway usa un agente de `undici` compartido (`src/lib/internal-fetch.ts`) para el BFF de autenticación, el tablero compuesto y la descarga del JWKS; `jose` no permite inyectar un `fetch` propio en su variante remota, de modo que el gateway **descarga el JWKS con el agente** y verifica con `createLocalJWKSet` y caché (refresco ante un `kid` desconocido, que es una rotación de llave); y el agente mTLS se aplica **solo a destinos HTTPS** (pasarlo en un salto HTTP rompía la conexión).

ADR-0021 — Multi-tenant SaaS: modo de despliegue, no un producto aparte

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 5) | | Relacionada | ADR-0010 (RLS activo) · ADR-0012 (zona horaria por negocio) · ADR-0013 (vertical packs) |

Contexto

Kubo nace **autoalojable**: cada negocio instala su copia y es dueño de sus datos. La arquitectura ya es multi-negocio de forma nativa —RLS con `FORCE` en las tres bases, cada petición con su `app.tenant_id`, y por negocio la zona horaria, el vertical y los usuarios—, pero eso es aislamiento **técnico**, no un modelo comercial: no hay planes, límites, suspensión ni facturación.
Ofrecer Kubo como SaaS (una instalación, muchos negocios) es tentador y peligroso: si se bifurca el código, se mantienen dos productos. El plan de cierre lo deja como backlog declarado justamente por eso.

Decisión

1. **El SaaS es un MODO de despliegue del mismo producto**, no un fork: la misma imagen, la misma base de código. Lo que cambia es la operación (quién aprovisiona, quién paga, cuánto se permite), no la arquitectura.
2. **El plan es un dato del negocio** (`tenants.plan`, ya existente), con un catálogo declarativo —igual que los vertical packs (ADR-0013)— y viaja en el token como `tenant_plan`, de modo que cada servicio aplica sus límites sin consultar a IAM en cada petición.
3. **Los límites son del plan, no del código**: usuarios, bodegas, productos y almacenamiento. Se aplican como **límites suaves con aviso** (el servicio responde `PLAN_LIMIT_REACHED` con el límite y el uso) y nunca borrando datos: un negocio que creció no pierde nada por bajar de plan.
4. **La suspensión es un estado del negocio** (`tenants.status`): suspendido no puede iniciar sesión (`TENANT_SUSPENDED`), pero sus datos quedan intactos y

exportables. La morosidad no puede convertirse en rehén de los datos.

5. **El cobro es un puerto** (como la facturación DIAN y las notificaciones): el producto registra el estado comercial y un adaptador lo sincroniza con la pasarela o con la cobranza manual. Para el mercado de barrio, el adaptador por defecto es **manual** (transferencia/Nequi y activación por el operador): la pasarela automática es un adaptador más, no un requisito del núcleo.

6. **Los datos del negocio son suyos**: exportación completa (documentos, catálogo, ventas) y borrado con confirmación explícita al terminar la relación; el operador no retiene copias más allá del respaldo.

Consecuencias

- **Positivas**: un solo producto para autoalojado y SaaS; los límites y la suspensión no tocan el núcleo comercial (son datos + validaciones); el aislamiento ya está probado (RLS) y auditado.
- **Negativas**: el SaaS exige operación que el autoalojado no —aprovisionar, medir uso, cobrar, atender—; el plan viaja en el token, así que un cambio de plan aplica al siguiente refresco de sesión (aceptable: no es un dato transaccional). La facturación automática queda declarada como adaptador pendiente; hoy el ciclo es manual y así se documenta al cliente.

Verificación

- `kubo-iam`: el plan viaja en el token; crear usuarios por encima del límite responde `PLAN_LIMIT_REACHED`; un negocio suspendido no inicia sesión (`TENANT_SUSPENDED`) y sus datos siguen consultables por el operador.
- `make smoke`: el negocio de la demo (plan `community`) mantiene sus límites y el catálogo de planes es consultable.
- **Pendiente declarado**: medir uso agregado, pasarela de pago automática y

panel del operador (aprovisionar/suspender desde una interfaz).

ADR-0022 — Operador de respaldos: contenedor con verificación de restauración

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 5) | | Relacionada | ADR-0018 (documentos) · ADR-0020 (malla interna) |

Contexto

El respaldo existe desde la Fase 0 (`make backup`): `pg_dump` de las tres bases, `mongodump` de analítica, la configuración y retención por días, con un `systemd` timer sugerido para un servidor. Pero es **manual y sin verificación**, y tiene un hueco real: **no incluye el volumen de documentos** (P-25) —los XML de las facturas y los comprobantes en PDF viven en disco, no en la base—, así que un respaldo "completo" perdería los documentos fiscales.

Un respaldo que nunca se restaura no es un respaldo: es una carpeta con fechas.

Decisión

1. **Operador en contenedor** (`kubo-backup`, perfil `backup` de compose): un servicio que corre el ciclo (respaldo → retención → verificación) sin depender del host ni del socket de Docker. Habla con Postgres y Mongo **por la red del compose** con los clientes oficiales; los documentos los lee del volumen montado en solo lectura. El `systemd` timer sigue valiendo para un servidor sin el perfil.
2. **El respaldo incluye los documentos**: el volumen `documents` se empaqueta junto a las bases. Un respaldo sin los documentos no restaura un negocio.
3. **Verificación obligatoria**: después de cada respaldo, el operador **restaura** los dumps en bases de usar y tirar, compara **conteos clave** (productos, ventas, clientes, usuarios) contra el origen y deja el resultado

en el manifiesto. Si la verificación falla, el respaldo se marca `FALLIDO` y el ciclo termina con error (un respaldo no verificado no debe pasar por bueno).

4. **Manifiesto con integridad**: fecha, versión del esquema, conteos, resultado de la verificación y **SHA-256** de cada archivo. Permite comprobar un respaldo viejo sin restaurarlo entero.

5. **Retención declarada**: `KUBO_BACKUP_RETENTION_DAYS` (por defecto 14) y `KUBO_BACKUP_KEEP_MIN` (por defecto 3): se conservan al menos los últimos N aunque la retención por días los alcance —un negocio que estuvo cerrado una semana no debe quedarse sin ningún respaldo—.

6. **El destino es un puerto**: por defecto el volumen local; una copia fuera del sitio (otro disco montado o un bucket) se configura con `KUBO_BACKUP_OFFSITE_DIR` (copia posterior al ciclo). Un adaptador de objeto (S3/Cloudinary) es el siguiente paso natural, igual que en documentos.

Consecuencias

- **Positivas**: el respaldo pasa a ser un ciclo verificable y completo (documentos incluidos); la verificación detecta dumps corruptos o migraciones rotas antes de necesitarlos; el manifiesto con hashes permite auditar sin restaurar.
- **Negativas**: la verificación consume recursos (restaura las bases en un contenedor pequeño): se ejecuta con `pg_restore --jobs=2` y bases temporales que se borran al terminar; en un servidor muy modesto puede desactivarse con `KUBO_BACKUP_VERIFY=false`, y entonces el manifiesto lo declara (`verificacion=omitida`) en lugar de mentir.

Verificación

- `make backup-operator`: ejecuta un ciclo completo y deja el manifiesto con

`verificacion=ok` y los conteos; un respaldo con `verificacion=fallida`
termina con código de salida distinto de cero.
- `make restore-drill`: sigue siendo la prueba de restauración **completa** en
bases de prueba (el operador verifica conteos en cada ciclo; el simulacro
restaura y valida la aplicación contra los datos).
- El volumen de documentos aparece en el archivo del respaldo y en el
manifiesto.

ADR-0023 — App móvil: la PWA primero, Capacitor cuando el negocio lo pida

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 5) | | Relacionada | ADR-0011 (BFF) · ADR-0013 (vertical packs) |

Contexto

El plan de cierre deja la app móvil nativa como backlog declarado. La PWA ya
resuelve buena parte del caso móvil: es instalable, funciona sin conexión (el
service worker encola ventas y sincroniza al reconectar, P-12) y pasa la
auditoría de accesibilidad. Un negocio de barrio vende desde el mismo equipo del
mostrador; el caso "móvil" realista es el celular del dueño para mirar el
tablero, o un vendedor en la calle con el catálogo.
Escribir una app nativa (Kotlin/Swift) significa mantener **dos** interfaces
sobre el mismo API: el doble de trabajo para la misma función.

Decisión

1. **La PWA es la interfaz** mientras el negocio no pida capacidades que el
navegador no dé. No se escribe una app nativa "porque sí": cada interfaz
nueva es deuda de mantenimiento.
2. **Cuando haga falta, se envuelve la PWA con Capacitor** antes que reescribir:

la misma base de código, empaquetada como app para Android/iOS, con acceso a las capacidades nativas que el navegador no expone. Criterios objetivos para dar el paso:

- **Cámara y código de barras** para cargar inventario y vender.
- **Notificaciones push** (además del buzón actual).
- **Biometría** para el segundo factor (hoy TOTP con app externa, ADR-0015).
- **Impresión térmica** por Bluetooth en el POS.

3. **El API no cambia**: la app (envuelta o nativa) consume el mismo gateway; nada de endpoints exclusivos que bifurquen el contrato.
4. **La tienda de aplicaciones no es un requisito del producto**: una app interna (distribución directa del APK) sirve a un negocio autoalojado; la publicación en las tiendas es una decisión comercial posterior.

Consecuencias

- **Positivas**: cero trabajo hoy y una ruta de escape concreta y barata (Capacitor) cuando aparezca una necesidad real; el contrato del API y el BFF ya están preparados (ADR-0011); la cola offline de la PWA es reutilizable.
- **Negativas**: una PWA no tiene el rendimiento de una app nativa en listas largas ni el acceso a todos los periféricos; se acepta porque el caso de uso (catálogo de barrio, tablero, POS de mostrador) está cubierto. El segundo factor por biometría queda pendiente hasta que exista la app envuelta.

Verificación

- Hoy: `make e2e` (Playwright + axe) sobre la PWA en viewport móvil y la cola offline verificada en el humo (una venta sin conexión se sincroniza).
- Al envolver con Capacitor (cuando se decida): la suite E2E debe correr contra el binario empaquetado y el contrato OpenAPI debe validarse igual que hoy.

ADR-0024 — Superficie del operador: script sobre la base, panel cuando haya rol

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 5) | | Relacionada | ADR-0021 (multi-tenant SaaS) · ADR-0015 (segundo factor) |

Contexto

El corte SaaS (ADR-0021) dejó el estado comercial del negocio —plan, cupos y suspensión— pero **sin forma de operarlo**: suspender un negocio por mora exigía abrir `psql` y escribir el `UPDATE` a mano. El plan de cierre lo declara como pendiente: «panel del operador (listar negocios, suspender/activar) con un endpoint protegido».

Ese endpoint exige antes decidir **el rol de operador**: hoy todos los roles (`OWNER`, `ADMIN`, `SELLER`, `ACCOUNTANT`, `VIEWER`) son **de un negocio**; el operador de la plataforma está por encima de los negocios. Un rol así es la credencial más poderosa del sistema: puede suspender a cualquier cliente y, si se descuida, leer cualquier dato.

Decisión

1. **Hoy: script de operación** (`kubo-infra/scripts/tenant-admin.sh`) contra la base de identidad: `list`, `suspend` y `activate` por slug o correo. El operador de un despliegue autoalojado (o del SaaS pequeño) ya tiene acceso al servidor; un script **no agrega superficie de ataque** y las acciones son raras (una mora, un abuso).
2. **La suspensión es reversible y no toca datos**: cambia `tenants.status`; el negocio no entra (`TENANT_SUSPENDED`) y todo lo demás queda intacto. El script imprime a quién afectó para que la acción quede en el registro de quien la ejecuta.

3. **El panel HTTP llega cuando haya rol de plataforma**, y ese rol tendrá su propio ADR con: ámbito separado (`/platform/*`, nunca `/tenants/me`), segundo factor **obligatorio** (ADR-0015), auditoría en tabla propia y un token con `platform: true` que el gateway valide. No se improvisa un rol omnipotente por comodidad.
4. **Ni el script ni el futuro panel pueden leer datos de negocio**: el script solo consulta `tenants` y `users` (identidad) y jamás las bases del CRM o del ERP; el aislamiento por RLS sigue siendo la garantía, no la buena voluntad del operador.

Consecuencias

- **Positivas**: cero superficie nueva hoy; la suspensión queda verificada de extremo a extremo (humo); el día que haya varios operadores o soporte delegado, el ADR del rol acota el poder antes de construirlo.
- **Negativas**: el operador necesita acceso SSH al servidor (aceptable en autoalojado y en un SaaS pequeño; un SaaS con soporte delegado necesitará el panel y el rol, con las salvaguardas del punto 3). El script no valida reglas de negocio (por ejemplo, avisar al cliente antes de suspender): es una herramienta de operación, no un flujo comercial.

Verificación

- `make smoke`: el negocio aislado se suspende con el script, su ingreso responde `TENANT_SUSPENDED` y al reactivarlo vuelve a entrar — el ciclo completo, no solo la unidad.
- El script lista plan, estado y usuarios activos de cada negocio (lo que el operador necesita para decidir).

ADR-0025 — Rol de plataforma y panel del operador

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada (Fase 6) | | Relacionada | ADR-0015 (TOTP) · ADR-0021 (multi-tenant SaaS) · ADR-0024 (superficie del operador) |

Contexto

La superficie del operador es hoy un script contra la base (ADR-0024), que basta mientras el operador sea uno y tenga acceso al servidor. Cuando haya **varios operadores o soporte delegado**, hace falta un panel con acceso propio —y eso exige el rol de plataforma que ADR-0024 dejó explícitamente para después.
Ese rol es la credencial más poderosa del sistema: está **por encima de los negocios**. Un error de diseño aquí no se arregla con un parche: se convierte en la forma de leer los datos de todos los clientes.

Decisión

1. **El rol de plataforma está separado de los roles del negocio**: una tabla propia (`platform_admins`) con sus credenciales y su segundo factor. Nunca es un usuario de un tenant: un usuario de negocio no puede tener poder de plataforma, y un administrador de plataforma **no puede entrar a un negocio como si fuera su dueño**.
2. **Segundo factor obligatorio** (TOTP, ADR-0015): sin código no hay sesión de plataforma. No es opcional para este rol.
3. **Reino de token separado**: el token de plataforma lleva `platform: true` (y `typ=access`); el gateway enruta `/platform/*` **solo** con ese claim y rechaza un token de negocio (y al revés: un token de plataforma no lee `/products`, `/sales` ni `/customers`). El aislamiento no depende de la interfaz.
4. **Poder mínimo**: el panel lista negocios, muestra su uso, plan y estado, y

permite **suspender, reactivar y renovar**. No puede leer datos de negocio —ventas, clientes, documentos—: para eso está el dueño. El aislamiento por RLS sigue siendo la garantía, no la buena voluntad del operador.

5. **Auditoría propia** (`platform_audit`): cada acción con quién, qué, cuándo y desde dónde; el panel la muestra. Un poder sin registro no es auditable.
6. **La interfaz es una ruta de la PWA** (`/plataforma`) con su propio guardia por claim: reutiliza la pila (React, tokens, axe) sin abrir una aplicación aparte. El script (ADR-0024) queda como acceso de emergencia y automatización.
7. **Límite de tasa propio** para `/platform/*`: es una superficie pública más, y la más sensible.

Consecuencias

- **Positivas**: el poder de plataforma queda acotado, auditado y separado del producto; un token de negocio comprometido no abre el panel; el operador ve lo que necesita (uso, plan, estado) sin poder espiar la operación de nadie.
- **Negativas**: hay una credencial más que proteger (por eso 2FA obligatorio, poder mínimo y auditoría propia) y un reino de token que el gateway debe validar con cuidado (una confusión de claims sería grave; se cubre con pruebas negativas en el humo). El soporte «entrar a mirar el negocio del cliente» queda **fuera** por diseño: se resuelve pidiendo al dueño, no suplantándolo.

Verificación

- `make smoke` (**167/167**): el acceso de plataforma **siempre** pide el código (`totpRequired`), un código inválido responde `INVALID_TOTP` y un token de negocio contra `/platform/tenants` responde **403**.
- `kubo-iam` (**69 pruebas**, cobertura cumplida): el flujo positivo con el secreto bajo control (acceso → código vigente → token), la contraseña

incorrecta auditada, la suspensión, la renovación (extiende desde la fecha vigente), el listado con la marca de sistema y la rotación del segundo factor (el código del secreto viejo deja de verificar).
- `make e2e` (**5/5**): la página del panel pasa la auditoría de accesibilidad.
- **Uso agregado del ERP en el panel (cerrado)**: el ERP expone `GET /api/v1/internal/usage` —sin plug de identidad, alcanzable solo por la malla mTLS y con la lista de negocios que le pasa el gateway— y consulta cada negocio **con su propia marca `app.tenant_id`** (no se amplía la política RLS). El gateway compone la vista (`/api/v1/platform/usage`, patrón BFF) y el panel muestra productos, bodegas, ventas del mes y documentos por negocio: **solo conteos, nunca datos de negocio**. Evidencia: 3 pruebas del gateway (composición y degradación parcial), el endpoint interno inalcanzable sin certificado y el humo 177/177 (un token de negocio responde 403 en `/platform/usage`).
- **Rotación del segundo factor del operador (cerrada)**: `POST /platform/totp/rotate` exige sesión de plataforma (haber pasado el código), invalida el secreto anterior en el acto y entrega la URI `otpauth` **una sola vez**; queda auditado como `TOTP_ROTATED`. Si el operador pierde el autenticador, el camino de emergencia es borrar su fila de `platform_admins` y reiniciar IAM: el seeder recrea el operador y registra la URI una vez en el log. Evidencia: prueba de rotación (el código del secreto viejo deja de verificar) y el panel con el aviso de «se muestra una sola vez».

ADR-0026 — Puerto de cobro y pasarela de pago

| Campo | Valor | | --- | --- | | Fecha | 2026-09-24 | | Estado | Aceptada e implementada (Fase 6) | | Relacionada | ADR-0021 (multi-tenant SaaS) · ADR-0025 (rol de plataforma) |

Contexto

El cobro es **manual** (ADR-0021): el negocio paga por transferencia o Nequi y el operador registra el pago renovando la fecha (F6.2). Funciona para los primeros clientes, pero no escala: cada renovación depende de que alguien mire quién debe, y no hay rastro del pago más allá de una fecha.

Una pasarela automática (Wompi, MercadoPago o ePayco en Colombia) resuelve eso, pero mete al producto en el negocio de los pagos: webhooks públicos, firmas, idempotencia, conciliación. Como en la facturación DIAN y las notificaciones, la decisión es **puerto + adaptador**, no acoplar el núcleo a un proveedor.

Decisión

1. **Puerto de cobro en IAM** (`Payments`): `create_checkout(tenant, plan, ciclo)` devuelve el enlace/token de pago; `confirm(evento)` aplica el resultado. Adaptador por defecto: **manual** (el operador registra el pago, que es lo que hoy hace `renew`); un proveedor real implementa el mismo contrato.

2. **Los pagos son datos**: tabla `payment_intents` (negocio, plan, ciclo, monto, moneda, proveedor, referencia, estado `PENDING|PAID|FAILED|EXPIRED`, fecha). La renovación del plan se deriva del pago **pagado**, no de la buena voluntad: un pago manual también crea su intento (historia completa).

3. **Idempotencia por referencia del proveedor**: un webhook repetido (los proveedores reintentan) **no extiende dos veces** el plan. La clave es la transacción del proveedor, no el evento.

4. **Los webhooks son la fuente de verdad**, con **verificación de firma** y límite de tasa propio; el retorno del navegador solo muestra el estado (un usuario puede cerrar la pestaña antes de volver).

5. **La suspensión sigue siendo humana**: la pasarela registra pagos; el corte por mora lo decide el operador con la lista de vencidos delante (ADR-0024). Ningún webhook suspende un negocio en plena jornada.
6. **Los precios son datos del plan** (por moneda y ciclo), en el catálogo, no en el código.

Consecuencias

- **Positivas**: cambiar de proveedor —o volver al cobro manual— es cambiar el adaptador; la historia de pagos queda auditada y conciliable; el núcleo no conoce la pasarela.
- **Negativas**: aparece un endpoint público de webhooks que hay que proteger (firma, tasa, idempotencia) y una conciliación que alguien debe mirar cuando un pago no cuadra. Los precios en varias monedas agregan una tabla que mantener. El adaptador por defecto sigue siendo manual: la pasarela es una mejora, no un requisito para operar.

Verificación

- `kubo-iam`: un evento firmado válido marca el pago `PAID` y extiende la renovación; un evento repetido **no** la extiende otra vez; una firma inválida responde 401 y no toca nada.
- `make smoke` (F6.6 implementado): el negocio pide pagar (`POST /tenants/me/payments`) y el monto sale del catálogo; el webhook sin firma responde 401; el webhook firmado confirma el pago y extiende la vigencia; un reintento responde igual y **no** extiende otra vez; un monto distinto al de la intención se rechaza (`AMOUNT_MISMATCH`). 176/176.
- El panel del operador lista los pagos pendientes con el negocio y registra el pago manual (`POST /platform/payments/{id}/confirm`), que es el mismo camino

que usa un proveedor: una sola historia de pagos.

ADR-0027 — Sincronización de los 4 entornos: repositorio canónico y espejos con huella

| Campo | Valor | | --- | --- | | Fecha | 2026-10-04 | | Estado | Aceptada e implementada | | Relacionada | ADR-0002 (polyrepo con contratos) · ADR-0018 (documentos) |

Contexto

El proyecto vive en cuatro entornos con roles distintos: el **directorio del proyecto** (repos git locales, fuente de trabajo), **GitHub** (código e historial publicados), **Notion** (documentación de consulta con diseño: portadas, iconos, callouts y diagramas) y **Obsidian** (knowledge base con grafo y wikilinks). Mantenerlos a mano produce el riesgo clásico: versiones que se contradicen (un humo 192 en un lado y 189 en otro, una decisión que solo existe en un espejo).

La documentación se regenera de fuentes Markdown, así que la pregunta no es "cómo editar cuatro veces" sino **cuál es la fuente de verdad y cómo detectar que un espejo quedó desviado**.

Decisión

1. **El directorio del proyecto es la fuente canónica.** GitHub, Notion y Obsidian son **espejos derivados**; nunca se edita contenido en un espejo (la presentación propia de cada uno — portadas, iconos, wikilinks — sí es suya).
2. **Huella de sincronización**: un `sha256` (16 hex) de las fuentes canónicas (README raíz, 13 documentos, ADRs, evidencia, diagramas y los README de los 9 repos). Cada espejo guarda la huella con la que fue generado: Notion en el intro de la página raíz; Obsidian en el frontmatter del hub.

3. **Orquestador `kubo-infra/scripts/kubo-sync.sh`** con tres comandos: `status` (compara huellas y estado git; no cambia nada), `run` (propaga solo lo desviado: PDF si las fuentes de docs cambiaron → Notion → Obsidian → push de commits pendientes) y `watch` (vigila y propaga al detectar cambios).
4. **La presentación se preserva**: el sincronizador de Notion inyecta los callouts de resumen y reemplaza los bloques Mermaid por los diagramas archify ya subidos (mapa persistente de `file-upload` ids), de modo que re-sincronizar no degrada el diseño.
5. **Los cambios manuales en un espejo se detectan como desvío** (la huella no coincide) y `run` lo regenera desde el repositorio. La política es explícita: el contenido se corrige en la fuente, no en el espejo.
6. **Git nunca se automatiza a ciegas**: `run` empuja commits que ya existen; auto-commitear cambios locales requiere `--commit` explícito.

Consecuencias

- **Positivas**: los cuatro entornos quedan alineados sin incongruencias; la sincronización es idempotente y regenerable (un espejo corrupto se reconstruye); cada entorno conserva su fortaleza (GitHub: código; Notion: presentación; Obsidian: grafo).
- **Negativas**: la huella es gruesa (un cambio en cualquier README dispara la re-sincronización completa de Notion); los PNG de diagramas no se re-suben automáticamente si cambian (requieren el kit de diseño); Notion y Obsidian dependen de un token y de una ruta local respectivamente.

Verificación

- `make sync-status` → **SINCRONIZADO** con las cuatro huellas iguales.

- Ciclo completo ejecutado dos veces: Notion 53 operaciones OK / 0 errores,

Obsidian regenerado, PDF al día.

- Prueba de detección: un cambio temporal en `07-pruebas.md` marca PENDIENTE;

al revertirlo, los espejos vuelven a `ok`.

- Preservación del diseño: tras dos sincronizaciones, `01 — Arquitectura`

conserva su callout y sus 5 diagramas (0 bloques Mermaid) y `02` mantiene

sus 2 diagramas ER.

ADR-0028 — Despliegue público de demostración 100 % open source

| Campo | Valor | | --- | --- | | Fecha | 2026-10-04 | | Estado | Aceptada e implementada (en ejecución) | | Relacionada | ADR-0027 (sincronización de los 4 entornos) · ADR-0020 (mTLS) · ADR-0022 (respaldos) |

Contexto

El producto necesita **mostrarse a cualquier persona con internet** (clientes,

jurados, reclutadores) sin presupuesto inicial. Restricciones del propietario:

$0, sin tarjeta de crédito, **tecnologías 100 % open source**, dominio gratis y

sin Kubernetes por ahora. El sistema no es un sitio estático: son 6 servicios,

4 almacenes, malla mTLS y un outbox, pensados para un servidor único con Docker

Compose (ADR-0002, ADR-0021).

La conexión del hogar tiene **IP pública real** (no CGNAT), lo que habilita la

ruta más pura: el tráfico no necesita pasar por ningún tercero.

Decisión

1. **Host: el portátil**, directo (sin equipo de repuesto y sin presupuesto).

Los puertos internos siguen en loopback; solo se publica el borde TLS.

2. **Borde: Caddy + Let's Encrypt** (ya en el proyecto) con **HTTP-01** y la

imagen oficial (sin builds custom). El perfil público vive en
`Caddyfile.public` y se selecciona con `KUBO_CADDYFILE`.
3. **Dominio gratis: DuckDNS** (`kubo-app.duckdns.org`) — API de actualización para IP dinámica (`ddns-duckdns.sh`). Se descartaron deSEC (`dedyn.io` pausado) y dynv6 (dominio padre caído en DNS); sus herramientas se eliminaron del repositorio.
4. **Respaldo offsite: restic/rclone hacia Storj** (S3, 25 GB gratis, tecnología abierta) como destino del operador de respaldos (ADR-0022).
5. **Descartes**: Vercel (solo estático; rompe el proxy mismo-origen y la cookie `SameSite=Strict`), PaaS gratuitos (512 MB, expiran o hibernan), nubes con tarjeta de verificación (Oracle/GCP/AWS), Kubernetes por ahora.
6. **Plan B**: si el ISP bloquea los puertos entrantes o no hay acceso al router, se usa un **túnel open source** (zrok/OpenZiti) — el compose no cambia, solo el borde.
7. **KVM/virt-manager** queda para la **VM de ensayo** (paridad de entornos y snapshots) antes de presentaciones a clientes; no es el host del demo.

Consecuencias

- **Positivas**: la ruta de datos es 100 % propia y open source (sin túneles ni intermediarios); costo $0; la migración a un VPS pagado es lift-and-shift (mismo compose, mismo Ansible, solo cambia el host); el ensayo en VM permite probar todo antes de exponerlo.
- **Negativas**: el portátil debe estar encendido durante el demo (sin SLA); la IP es dinámica (mitigado con DDNS); el router del ISP debe permitir el reenvío de puertos (mitigado con el plan B); `KUBO_COOKIE_SECURE=true` implica que el acceso local de desarrollo debe ser por HTTPS o vía la URL

pública.

Verificación

- La ruta directa (`https://kubo-app.duckdns.org`) sirve la PWA con

certificado Let's Encrypt válido; el túnel (`https://kubo.shares.zrok.io`)

cubre el demo actual.

- `check-host.net` confirma 80/443 abiertos desde nodos externos; los puertos

9080–9084 y las bases **no** responden desde internet.

- Login por HTTPS con cookie de refresco `Secure`; el límite de autenticación

sigue operando por IP real (sin túnel que la enmascare).

- `make backup` con copia offsite en Storj y `make restore-drill` en verde.

ADR-0029 — Repositorios públicos con licencia MIT y CI en GitHub Actions

| Campo | Valor | | --- | --- | | Fecha | 2026-10-06 | | Estado | Aceptada e implementada | | Relacionada | ADR-0002 (polyrepo) · ADR-0027 (sincronización de los 4 entornos) · ADR-0028 (despliegue público) |

Contexto

El MVP está operable y verificado en local (`make ci`: 13 verificaciones), pero

los 9 repositorios vivían en GitHub **privados**, sin licencia explícita, sin

integración continua en la nube y sin un camino de clonado para terceros. El

proyecto cumple además una función de **vitrina profesional** (jurados,

reclutadores, clientes) y necesita: que cualquiera pueda leer el código y la

documentación; que un clon desde cero funcione en un solo paso; que cada cambio

se verifique automáticamente en GitHub (no solo en la máquina del autor); y que

la rama `main` quede protegida contra reescrituras y borrados.

Restricciones: costo $0 (plan gratuito de GitHub); la demo pública usa datos de

ejemplo (ADR-0028); el despliegue real de cada negocio ocurre en su propio

servidor, nunca en la nube del proyecto.

Decisión

1. **Repositorios públicos** (workspace + 8 hijos). Antes de cambiar la visibilidad se ejecutó el escaneo de secretos (6 verificaciones, 0 hallazgos) y se confirmó que no hay `.env`, claves ni tokens en el árbol ni en el historial de los 9 repos.
2. **Licencia MIT** en los 9 (titular: Nelson Fabián Gallego Sánchez), la que ya declaraban el README y la portada del PDF; ahora es explícita y detectada por GitHub.
3. **CI en GitHub Actions** por repositorio (`.github/workflows/ci.yml`), espejo del gate local: gateway (typecheck + pruebas con Redis real), IAM (`mvn verify` + Testcontainers), CRM y ERP (PostgreSQL real con RLS), analítica (pytest + cobertura + MongoDB), web (typecheck + vitest), docs (enlaces relativos con el workspace completo) e infra (sintaxis de scripts + compose válido con `.env.example`).
4. **Protección de `main`** en los 9: sin force-push ni borrado de la rama; en los 8 con CI, el *status check* del job correspondiente es obligatorio para fusionar pull requests (`strict`). Los pushes directos del mantenedor siguen permitidos mientras el proyecto esté en manos de una sola persona.
5. **Clonado en un paso**: `scripts/clonar-todos.sh` + `make clone` en el workspace; el Ansible del despliegue clona los 9.
6. **Versionado**: tag `v0.3.0` en los 9 y release publicado en el workspace.

Opciones consideradas

Opción	Por qué no
Mantener los repos privados	El proyecto pierde su función de vitrina y el portafolio no es verificable
Públicos sin licencia explícita	Sin licencia rige "todos los derechos reservados", contra el README y la adopción
CI solo local (`make ci`)	La verificación no es

reproducible por terceros ni visible en PRs | | GitLab CI | Descartado antes: migración completa a GitHub, un solo proveedor | | Exigir PR y revisiones al mantenedor | Fricción innecesaria para un equipo de una persona; se revisará al sumar colaboradores |

Consecuencias

- **Positivas**: portafolio público verificable (badge de CI por repositorio);
ningún PR externo se fusiona con la CI en rojo; `main` no puede reescribirse
ni borrarse; el clonado desde cero está probado; Actions es ilimitado en
repositorios públicos; la licencia MIT habilita el uso y la contribución.
- **Negativas**: el código y el historial quedan visibles de forma permanente
(mitigado con el escaneo previo y la disciplina de secretos); dependencia del
plan gratuito de GitHub (si el producto se comercializa, se evaluará
open-core: núcleo público + módulos privados); los checks obligatorios añaden
una espera al fusionar PRs.
- **Revisión**: al entrar un segundo colaborador se activará "requerir PR +
aprobación" y se incluirá al administrador en las protecciones.

Verificación

- `gh api repos/NFGS/<repo>` reporta visibilidad pública y licencia MIT en los 9.
- 8 flujos de CI en verde (gateway, IAM, CRM, ERP, analítica, web, docs, infra)
y el gate local sigue en 13/13.
- Clon público desde cero: workspace + `make clone` trae los 8 hijos.
- Protección activa en `main` (sin force-push ni borrado) con los checks
exigidos por repositorio.
- Tag `v0.3.0` en los 9 repos y release publicado en el workspace.

ADR-0030 — Sincronización bidireccional de los 4 entornos con reconciliación

| Campo | Valor | | --- | --- | | Fecha | 2026-10-06 | | Estado | Aceptada e implementada | | Relacionada | ADR-0027 (sincronización de los 4 entornos) · ADR-0002 (polyrepo) · ADR-0028 (despliegue público) |

Contexto

El ADR-0027 dejó el directorio del proyecto como fuente canónica y tres espejos derivados (GitHub, Notion, Obsidian) con una huella global. Ese diseño detecta los cambios del canónico, pero **no los cambios hechos en un espejo**: una edición manual en Notion o en el vault quedaba como "desvío" y la siguiente sincronización la sobrescribía sin aviso. El dueño del proyecto pidió lo contrario: que un cambio en **cualquiera** de los 4 entornos se detecte y se propague a los demás, manteniendo la coherencia (sin incongruencias ni contradicciones) y respetando la naturaleza de cada entorno (GitHub: código e historial; Notion: presentación; Obsidian: grafo y wikilinks; local: fuente).

Decisión

1. **Líneas base por ítem** en `~/.config/opencode/kubo-sync/`: `files-baseline.json` (sha256 por archivo canónico), `notion-baseline.json` (`last_edited_time` por página) y `obsidian-baseline.json` (sha256 por nota). Los manifiestos (`notion-manifest.json`, `obsidian-manifest.json`) mapean cada página/nota con su archivo de origen y la marcan `importable` o `generado`.
2. **Detección y reconciliación** (`kubo-sync.sh check|pull`):
- GitHub: `git fetch` + fast-forward cuando el local está limpio y solo detrás; divergencia o árbol sucio ⇒ conflicto (sin auto-merge).
- Notion: página editada desde la línea base ⇒ se importa al archivo canónico (sin la línea de callout que agrega el diseño).
- Obsidian: nota editada ⇒ se importa revirtiendo frontmatter y wikilinks a

enlaces Markdown relativos.

3. **Política de conflictos explícita**: si el mismo ítem cambió en dos entornos (o el viaje de ida y vuelta perdería información, p. ej. diagramas Mermaid en Notion), **no se pisa nada**: la versión del espejo queda en `.sync/conflictos/` (ignorado por git) y el ciclo reporta `CONFLICTOS`. Las páginas generadas (índices, intro) se regeneran, no se importan.

4. **Ciclo automático desplegado**: `kubo-sync.service` + `kubo-sync.timer` (systemd de usuario, cada 15 min, `auto --commit`) con las unidades en `kubo-infra/systemd/user/` e instalador `instalar-unidades-usuario.sh`. `auto` = detección → importación → propagación → push, con registro en `auto.log` y notificación de escritorio si hay conflictos.

5. **Commits acotados**: el modo `--commit` solo agrega los archivos del alcance del sync (README, `kubo-docs/`, PDF y su huella); el trabajo ajeno al sync queda intacto y se reporta. Las ediciones importadas de un espejo se commitean con el mensaje `sync: documentacion actualizada (kubo-sync)`.

6. **Trazabilidad**: cada ciclo deja resumen en `auto.log`; el journal del servicio usa salida sin subprocesos cortos (journald atribuye mal las líneas de procesos efímeros).

Opciones consideradas

Opción	Por qué no
Bidireccional total con merge automático	Sin revisión humana puede fabricar contenido híbrido incorrecto; el staging de conflictos es verificable
Solo detección (avisar sin importar)	Incumple el requisito: el cambio en un espejo debe llegar a los demás
Comparar el contenido completo en vez de líneas base	Costoso (56 páginas × markdown completo) y frágil ante normalizaciones de Notion; `last_edited_time` + hashes es exacto y barato
Editar solo en local (estado anterior)	El usuario trabaja también en Notion y Obsidian; prohibirlo no era viable

Consecuencias

- **Positivas**: los 4 entornos convergen vengan de donde vengan los cambios;

nada se destruye sin dejar rastro (staging + git); el ciclo corre solo cada 15 minutos; el costo es bajo (≈50 s por ciclo, API de Notion a 3 req/s).

- **Negativas**: el viaje de ida y vuelta de Notion no es perfecto para archivos con Mermaid (se marca conflicto en vez de importar); una edición concurrente en local y espejo exige resolución manual; el timer commitea documentación automáticamente (decisión explícita del dueño).
- **Revisión**: si el proyecto suma colaboradores, se evaluará requerir PR para los commits del sync y ampliar el staging a notificaciones externas.

Verificación

- Ciclo completo probado en las 4 direcciones: local → espejos, Notion → local, Obsidian → local y GitHub → local (fast-forward), con reversión y re-sincronización.
- Conflicto real probado: edición simultánea local+Notion ⇒ staging en `.sync/conflictos/`, local intacto, `RESULTADO: CONFLICTOS`.
- `kubo-sync.timer` activo (cada 15 min) con `auto --commit`; `auto.log` registra cada ciclo y su resumen.
- `make sync-check` → sin deriva; `make sync-status` → SINCRONIZADO.

Actualización — 2026-10-07 (incidente de carrera y fidelidad)

La operación real expuso dos fallas, ya corregidas:

1. **Carrera timer ↔ sync manual**: el ciclo del timer se disparó mientras corría un `make sync` manual; la detección de deriva leyó páginas a medio escribir e importó 15 archivos con contenido redondeado. **Corrección**: `kubo-sync.sh` toma un candado (`flock`) — un solo ciclo a la vez; las ejecuciones concurrentes se omiten con aviso.
2. **El redondeo de Notion no es fiel**: su export markdown convierte tablas a

HTML, escapa enlaces dentro de formato y usa tabs. La importación automática escribió esas versiones en los canónicos (detectado, revertido con `git revert` en los 9 repos y espejos regenerados). **Corrección**: `notion-pull.sh` solo importa páginas cuyo redondeo es limpio; con pérdida, la versión queda en `.sync/conflictos/` y se reporta (nada se pisa en silencio).

3. **Alcance del commit acotado**: en el repo `kubo-docs` solo se agregaba su README; ahora el commit del sync incluye todo `kubo-docs/` (documentos, ADRs, evidencia y diagramas).

4. **Verificación en operación real (mismo día)**: un `make sync` manual durante un ciclo del timer se omitió con aviso (el candado funcionó); una edición hecha a mitad de ciclo dejó la huella del estado un paso atrás y el siguiente `make sync` la convergió sin intervención.

ADR-0031 — Empaquetado móvil: assets locales y servidor configurable en el primer arranque

| Campo | Valor | | --- | --- | | Fecha | 2026-10-07 | | Estado | Aceptada e implementada (Fase 1 del plan 14) | | Relacionada | ADR-0023 (app móvil) · ADR-0007 (JWT/cookies) · ADR-0011 (BFF) |

Contexto

El plan [`14-plan-app-movil.md`](../14-plan-app-movil.md) activa la envoltura de la PWA con Capacitor. Kubo es **autoalojado**: cada negocio tiene su propia URL (el APK debería ser el mismo para todos), y el POS convive con el servidor en la red local. La decisión clave de la Fase 0 es cómo sirve la app sus activos y cómo llega al servidor de cada negocio sin tocar el contrato del API.

Decisión

1. **Assets locales (modo empaquetado)**: la PWA se compila dentro del APK

(`webDir: dist`); la app abre sin red desde el primer arranque.

2. **URL del servidor configurable en el primer arranque**: la app pide la URL del negocio (p. ej. `https://kubo-app.duckdns.org`), la guarda en el dispositivo y la usa como base de `/api`.

3. **HTTP nativo (CapacitorHttp) para el API**: evita CORS y conserva la cookie de refresco httpOnly (ADR-0007) **sin cambiar el contrato**; se valida en la prueba de sesión de la Fase 1. Si la cookie no sobreviviera en el puente nativo, se activa el plan B (modo remoto).

4. **Un solo binario por versión** (no builds por negocio): la configuración vive en el dispositivo, no en el build.

5. **La PWA no cambia su comportamiento**: los ajustes son aditivos y solo se activan en plataforma nativa (`Capacitor.isNativePlatform()`).

Opciones consideradas

| Opción | Por qué no (o cuándo sí) | | --- | --- | | Modo remoto (`server.url` al servidor del negocio) | Un APK por negocio (URL estática en build) y el primer arranque sin red falla; queda como **plan B** si la sesión nativa no resulta viable | | App nativa (Kotlin/Swift) | Reescribir la interfaz: doble mantenimiento (ADR-0023) | | Tiendas como distribución | No requerido para autoalojado; decisión comercial posterior (ADR-0023) |

Consecuencias

- **Positivas**: un APK para todos los negocios; offline desde el primer arranque; contrato intacto; el trabajo web es aditivo y testeable; la cola offline de la PWA se reutiliza tal cual.

- **Negativas**: la app necesita una pantalla de configuración inicial; el HTTP nativo debe validarse en dispositivo real (riesgo declarado, mitigado con la prueba temprana de la Fase 1).

- **Revisión**: al validar la sesión en dispositivo se confirma este ADR o se cambia al modo remoto mediante un mini-ADR.

Verificación

- APK de depuración compilado en CI (job `android` de `kubo-web`) y artefacto subido por cada tag/ejecución manual.
- Fase 1 (siguiente tramo): ingreso + POS + cola offline funcionando en el binario; contrato OpenAPI validado igual que hoy (verificación del ADR-0023).

Actualización — 2026-10-07 (prueba de sesión en emulador)

- La **prueba de sesión pasó en el emulador Android de CI** (Maestro, flujo `.maestro/sesion.yaml`): primer arranque, conexión a la demo, ingreso real y **restauración de la sesión por cookie tras relanzar la app** (cookie httpOnly a través del puente nativo).
- Dos hallazgos implementados en el camino: (1) en la app móvil el **service worker se desregistra** — interceptaba los GET y devolvía el `index.html` local; (2) las llamadas del API usan **`CapacitorHttp` directo**, porque el proxy del parche de `fetch` resultó frágil en el WebView (trataba el GET como documento y servía el `index.html`). El contrato no cambió.
- Queda como verificación opcional instalar el APK en un equipo físico del negocio (el APK se descarga del artefacto de CI o del release).

Suite móvil completa (2026-10-07)

El job `Android` corre en cada push la suite `e2e-android/run-sesion.sh` (cuatro flujos Maestro sobre el emulador): sesión con restauración por cookie, **venta de POS con conexión**, **venta sin conexión a la cola local** (red cortada y verificada con `ping` desde el propio emulador) y **sincronización automática al reconectar** (la cola reintenta cada 30 s). Notas de operación: el túnel público gratuito puede responder 502 en el primer cobro (la app conserva el carrito y el flujo reintenta el cobro); y queda como backlog una

clave de idempotencia en las ventas del POS para que un reintento tras un

fallo ambiguo no pueda duplicar la venta.